[PRIVATE RFC] swap priority queue v2 patches and KMB validation package

"Lian Wang (ProcessMission)" <lianux.mm@gmail.com>
August 7, 2026 at 2:35 PM
To: "Kunwu Chan" <kunwu.chan@linux.dev>
Cc: "Lian Wang (ProcessMission)" <lianux.mm@gmail.com>, "Kairui Song" <kasong@tencent.com>, "Baoquan He" <bhe@redhat.com>, "Youngjun Park" <youngjun.park@lge.com>

Hi Kunwu,

I'm taking over the next iteration of Kairui's swap priority queue work and would like to share the current v2 package with you privately before we move toward another upstream RFC.

The package is based on mm-unstable commit 94f9b3980dd4 and ends at d5c8964cf19f. It preserves Kairui's and Youngjun's original authorship. My substantive follow-up changes are recorded with the corresponding development trailers in the affected patches.

I have ordered the material as requested:

1. the v2 cover letter and 13 kernel patches;
2. the v1-to-v2 range-diff;
3. the four-patch KMB support series;
4. the original Chinese VM raw report;
5. the Chinese test plan and current status;
6. the English KMB reproduction guide; and
7. the English handoff summary.

The main v2 changes are the stable task-local retry cursor, the CPU migration boundary around the queue walk, exactly-once same-ring peer traversal, same-priority large-folio retry, tagged full/disabled entries, tighter swapon/swapoff ordering, bounded discard work, and removal of the transitional plist infrastructure.

Current validation status:

- the current v2 builds and boots;
- the functional KMB suite passed 5/5;
- an extreme 2 GiB memcg / make -j96 / 8-ZRAM smoke completed with exit 0, all eight devices used, no OOM, and no kernel failure signature;
- the A/B/C/D/E five-kernel build-only gate completed successfully, with all five images, initrds, and the ordered inventory installed; and
- the A/B/C/D/E performance benchmarks have not started yet.

The 10-CPU VM is useful for correctness and severe-contention evidence, but it cannot reproduce Kairui's original absolute performance numbers. One current v2 2-GiB warm-up took 37 minutes, so the original 12-run 2-GiB/3-GiB matrix and the 48-GiB BRD workload are assigned to the later server phase. Real multi-SSD claims will require at least two independent SSD/NVMe devices.

Could you please focus your review on:

- whether one same-priority group should remain the queue/ring definition;
- the task-local cursor and migrate_disable() boundary;
- queue publication, tag updates, and swapoff lifetime ordering;
- exactly-once same-ring retry, especially for large folios;
- the synchronous discard bound; and
- whether the A/B/C/D/E plus real multi-SSD plan covers the important risks.

This is a private review package, not a request to post the series upstream yet. After your review, Kairui's review, and the remaining performance work, we can decide how to shape the public RFC v2 and coordinate with Youngjun.

Thanks,
Lian

```
==== Attachment 0 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0000-cover-letter.patch) ====
From d5c8964cf19fcffd7eea75dc9f28c928b46503bf Mon Sep 17 00:00:00 2001
From: "Lian Wang (Processmission)" <lianux.mm@gmail.com>
Date: Fri, 7 Aug 2026 13:56:50 +0800
Subject: [RFC PATCH v2 00/13] mm/swap: introduce per-priority allocation queues

This private-review v2 continues Kairui's swap priority queue work on top
of mm-unstable commit 94f9b3980dd4.  It keeps the original authorship and
Youngjun's per-device percpu-cluster patch while incorporating the review
and validation changes made during the handoff.

The series replaces the allocation-time priority plist rotation with
per-priority device rings and per-CPU readers.  Device selection remains
priority ordered, while devices at the same priority are rotated without
serializing every cluster transition on swap_avail_lock.

Compared with v1, the main allocator changes are:

- keep a task-local cursor stable across retries;
- disable task migration across the queue walk so queue accounting and
  per-CPU cluster allocation stay on the same CPU;
- visit every peer in the selected priority ring exactly once before
  considering a lower priority;
- retry a same-priority peer for large folios before returning -E2BIG;
- keep full/disabled state in tagged ring entries with serialized writers
  and READ_ONCE()/WRITE_ONCE() lockless readers;
- tighten swapon/swapoff publication and disable ordering;
- bound synchronous discard work in the allocation path; and
- remove the transitional active/available plist infrastructure after the
  queue becomes authoritative.

This package is for private review and joint testing.  It is not the final
upstream submission.  The accompanying range-diff, VM raw report, KMB guide,
and follow-up test plan are provided after the patch files.

Kairui Song (12):
  mm/swap: remove unused parameter for reading swap header
  mm/swap: slightly cleanup the code for hibernation error handling
  mm/swap: cleanup and document swap device availability flag usage
  mm/swap: introduce swap device iteration helper
  mm/swap: change the swapon lock into a percpu rwsem
  mm/swap: remove swapon mutex and update proc reader
  mm/swap: consolidate swap inuse accounting helpers
  mm/swap: add priority queue for swap device allocation
  mm/swap: remove available list
  mm/swap: bound synchronous discard during allocation
  mm/swap: drop swap active plist
  lib/plist.c: remove requeue function

Youngjun Park (1):
  mm/swap: change back to use each swap device's percpu cluster

 include/linux/plist.h |    2 -
 include/linux/swap.h  |   43 +-
 lib/plist.c           |   64 --
 mm/swap.h             |   12 +-
 mm/swapfile.c         | 1301 ++++++++++++++++++++++++++++++++----------------
 5 files changed, 817 insertions(+), 605 deletions(-)

--
2.55.0


==== Attachment 1 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0001-mm-swap-remove-unused-parameter-for-reading-swap-hea.patch) ====
From 22ec384c63f514b50bba818dc0050702b046e361 Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:39 +0800
```

```
Subject: [RFC PATCH v2 01/13] mm/swap: remove unused parameter for reading
 swap header

No feature change, just a minor cleanup.

Signed-off-by: Kairui Song <kasong@tencent.com>
---
 mm/swapfile.c | 7 +++----
 1 file changed, 3 insertions(+), 4 deletions(-)

diff --git a/mm/swapfile.c b/mm/swapfile.c
index 70b90fa9c2a0..e33786578334 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -3460,9 +3460,8 @@ __weak unsigned long arch_max_swapfile_size(void)
        return generic_max_swapfile_size();
 }

-static unsigned long read_swap_header(struct swap_info_struct *si,
-                                       union swap_header *swap_header,
-                                       struct inode *inode)
+static unsigned long read_swap_header(union swap_header *swap_header,
+                                     struct inode *inode)
 {
        int i;
        unsigned long maxpages;
@@ -3686,7 +3685,7 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
        }
        swap_header = kmap_local_folio(folio, 0);

-       maxpages = read_swap_header(si, swap_header, inode);
+       maxpages = read_swap_header(swap_header, inode);
        if (unlikely(!maxpages)) {
                error = -EINVAL;
                goto bad_swap_unlock_inode;
--
2.55.0



==== Attachment 2 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0002-mm-swap-slightly-cleanup-the-code-for-hibernation-er.patch) ====
From a940e5611be055436bf0ec7fd163f1f7a55c446b Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:40 +0800
Subject: [RFC PATCH v2 02/13] mm/swap: slightly cleanup the code for
 hibernation error handling

Restructure the error paths in pin_hibernation_swap_type() using a
goto out pattern to avoid repeated spin_unlock() calls and simplify
the return flow.  Also simplify unpin_hibernation_swap_type() by
making the si NULL check inline, and clean up find_first_swap() to
avoid an early return inside the loop.

Signed-off-by: Kairui Song <kasong@tencent.com>
---
 mm/swapfile.c | 39 +++++++++++++++++----------------------
 1 file changed, 17 insertions(+), 22 deletions(-)

diff --git a/mm/swapfile.c b/mm/swapfile.c
index e33786578334..bc7349faae11 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -2253,21 +2253,18 @@ static int __find_hibernation_swap_type(dev_t device, sector_t
offset)
  */
 int pin_hibernation_swap_type(dev_t device, sector_t offset)
 {
```

```
-       int type;
+       int ret;
        struct swap_info_struct *si;

        spin_lock(&swap_lock);
+       ret = __find_hibernation_swap_type(device, offset);
+       if (ret < 0)
+               goto out;

-       type = __find_hibernation_swap_type(device, offset);
-       if (type < 0) {
-               spin_unlock(&swap_lock);
-               return type;
-       }
-
-       si = swap_type_to_info(type);
+       si = swap_type_to_info(ret);
        if (WARN_ON_ONCE(!si)) {
-               spin_unlock(&swap_lock);
-               return -ENODEV;
+               ret = -ENODEV;
+               goto out;
        }

        /*
@@ -2276,14 +2273,15 @@ int pin_hibernation_swap_type(dev_t device, sector_t offset)
         * the same session.
         */
        if (WARN_ON_ONCE(si->flags & SWP_HIBERNATION)) {
-               spin_unlock(&swap_lock);
-               return -EBUSY;
+               ret = -EBUSY;
+               goto out;
        }

        si->flags |= SWP_HIBERNATION;

+out:
        spin_unlock(&swap_lock);
-       return type;
+       return ret;
 }

 /**
@@ -2302,11 +2300,8 @@ void unpin_hibernation_swap_type(int type)

        spin_lock(&swap_lock);
        si = swap_type_to_info(type);
-       if (!si) {
-               spin_unlock(&swap_lock);
-               return;
-       }
-       si->flags &= ~SWP_HIBERNATION;
+       if (si)
+               si->flags &= ~SWP_HIBERNATION;
        spin_unlock(&swap_lock);
 }

@@ -2341,7 +2336,7 @@ int find_hibernation_swap_type(dev_t device, sector_t offset)

 int find_first_swap(dev_t *device)
 {
-       int type;
+       int type, ret = -ENODEV;

        spin_lock(&swap_lock);
        for (type = 0; type < nr_swapfiles; type++) {
@@ -2350,11 +2345,11 @@ int find_first_swap(dev_t *device)
                if (!(sis->flags & SWP_WRITEOK))
```

```
                        continue;
                *device = sis->bdev->bd_dev;
-               spin_unlock(&swap_lock);
-               return type;
+               ret = type;
+               break;
        }
        spin_unlock(&swap_lock);
-       return -ENODEV;
+       return ret;
 }

 /*
--
2.55.0



==== Attachment 3 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0003-mm-swap-cleanup-and-document-swap-device-availabilit.patch) ====
From f7ad0bf804906088430f718ffddc7aa15938abaa Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:41 +0800
Subject: [RFC PATCH v2 03/13] mm/swap: cleanup and document swap device
 availability flag usage

Rework swap device flag and metadata handling and swapon/swapoff to
be cleaner and better documented, in preparation for locking cleanup.

Consolidate existing routines and introduce swap_device_enable() and
swap_device_disable() as the clean boundary of exposing or isolating a
swap device. swap_device_enable() sets the proper flags and exposes the
device as an allocation candidate, while swap_device_disable() clears
related flags and ensures no more allocations will happen.

Keep the mapping lookup and disable transition in the same swap_lock
critical section. Otherwise, a concurrent swapoff can release the
selected slot and swapon can reuse it for a different device before the
first syscall disables the swap_info_struct it found. Drain the cluster
allocators in the same helper after dropping swap_lock, so the identity
transition remains atomic without holding the lock over all cluster
locks.

Add comment blocks documenting the lifetime and locking rules for swap
device flags and their locking conventions.

Apart from closing that lifecycle race, the remaining changes are code
rearrangement and documentation.

Signed-off-by: Kairui Song <kasong@tencent.com>
Co-developed-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 include/linux/swap.h |  27 ++++--
 mm/swapfile.c        | 207 +++++++++++++++++++++++++++----------------
 2 files changed, 115 insertions(+), 119 deletions(-)

diff --git a/include/linux/swap.h b/include/linux/swap.h
index 45f301d73e2a..5e1109010c89 100644
--- a/include/linux/swap.h
+++ b/include/linux/swap.h
@@ -194,6 +194,22 @@ struct swap_extent {
        ((offsetof(union swap_header, magic.magic) - \
          offsetof(union swap_header, info.badpages)) / sizeof(int))

+/*
+ * Swap device flags, except the ones documented below, all are immutable
+ * after exposed by swap_device_enable, and until the device is freed again
+ * (SWP_USED unset). The exceptions:
```

```
+ * - SWP_USED: Protected by swap_lock. Indicates the device is inuse. Once
+ *   set, won't be cleared unless all reference to this device is freed and
+ *   swapoff finished.
+ * - SWP_WRITEOK: Protected by both swap_lock and swap_avail_lock, clearing
+ *   this flag also waits for all current cluster lock users to exit so
+ *   checking this flag while holding any of these locks ensures the device
+ *   is safe to use at the moment. Note: clearing this flag doesn't affect
+ *   pending IO or async requests, it only prevents further entry allocation
+ *   or new async request (e.g. discard) from initiating.
+ * - SWP_HIBERNATION: Protected by swap_lock. Indicates if the device
+ *   is pinned for hibernation.
+ */
 enum {
        SWP_USED        = (1 << 0),     /* is slot in swap_info[] used? */
        SWP_WRITEOK     = (1 << 1),     /* ok to write to this swap?    */
@@ -262,14 +278,9 @@ struct swap_info_struct {
        struct file *swap_file;         /* seldom referenced */
        struct completion comp;         /* seldom referenced */
        spinlock_t lock;                /*
-                                        * protect map scan related fields like
-                                        * inuse_pages and all cluster lists.
-                                        * Other fields are only changed
-                                        * at swapon/swapoff, so are protected
-                                        * by swap_lock. changing flags need
-                                        * hold this lock and swap_lock. If
-                                        * both locks need hold, hold swap_lock
-                                        * first.
+                                        * Protect cluster lists. Other fields
+                                        * are only changed at swapon/swapoff,
+                                        * so are protected by swap_lock.
                                         */
        struct work_struct discard_work; /* discard worker */
        struct work_struct reclaim_work; /* reclaim worker */
diff --git a/mm/swapfile.c b/mm/swapfile.c
index bc7349faae11..7b8523df42a1 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -1199,28 +1199,20 @@ static void del_from_avail_list(struct swap_info_struct *si,
bool swapoff)

        spin_lock(&swap_avail_lock);

-       if (swapoff) {
-               /*
-                * Forcefully remove it. Clear the SWP_WRITEOK flags for
-                * swapoff here so it's synchronized by both si->lock and
-                * swap_avail_lock, to ensure the result can be seen by
-                * add_to_avail_list.
-                */
-               lockdep_assert_held(&si->lock);
-               si->flags &= ~SWP_WRITEOK;
-               atomic_long_or(SWAP_USAGE_OFFLIST_BIT, &si->inuse_pages);
-       } else {
-               /*
-                * If not called by swapoff, take it off-list only if it's
-                * full and SWAP_USAGE_OFFLIST_BIT is not set (strictly
-                * si->inuse_pages == pages), any concurrent slot freeing,
-                * or device already removed from plist by someone else
-                * will make this return false.
-                */
+       /*
+        * Force remove it only for swapoff. Else, take it off-list only if
+        * it's full and SWAP_USAGE_OFFLIST_BIT is not set (strictly
+        * si->inuse_pages == pages), so concurrent slot freeing, or
+        * concurrent list removal will make the cmpxchg fail and skip
+        * the removal.
+        */
+       if (!swapoff) {
                pages = si->pages;
```

```
                if (!atomic_long_try_cmpxchg(&si->inuse_pages, &pages,
                                             pages | SWAP_USAGE_OFFLIST_BIT))
                        goto skip;
+       } else {
+               atomic_long_or(SWAP_USAGE_OFFLIST_BIT, &si->inuse_pages);
        }

        plist_del(&si->avail_list, &swap_avail_head);
@@ -1230,21 +1222,21 @@ static void del_from_avail_list(struct swap_info_struct *si,
bool swapoff)
 }

 /* SWAP_USAGE_OFFLIST_BIT can only be cleared by this helper. */
-static void add_to_avail_list(struct swap_info_struct *si, bool swapon)
+static void add_to_avail_list(struct swap_info_struct *si)
 {
        long val;
        unsigned long pages;

        spin_lock(&swap_avail_lock);

-       /* Corresponding to SWP_WRITEOK clearing in del_from_avail_list */
-       if (swapon) {
-               lockdep_assert_held(&si->lock);
-               si->flags |= SWP_WRITEOK;
-       } else {
-               if (!(READ_ONCE(si->flags) & SWP_WRITEOK))
-                       goto skip;
-       }
+       /*
+        * Add the device to the avail list if SWP_WRITEOK is set and
+        * SWAP_USAGE_OFFLIST_BIT is still set. Swapoff clears
+        * SWP_WRITEOK first, so the device won't be re-added after
+        * swapoff starts unless swap_device_enable resurrects it.
+        */
+       if (!(si->flags & SWP_WRITEOK))
+               goto skip;

        if (!(atomic_long_read(&si->inuse_pages) & SWAP_USAGE_OFFLIST_BIT))
                goto skip;
@@ -1300,7 +1292,7 @@ static void swap_usage_sub(struct swap_info_struct *si, unsigned
int nr_entries)
         * add it to the plist.
         */
        if (unlikely(val & SWAP_USAGE_OFFLIST_BIT))
-               add_to_avail_list(si, false);
+               add_to_avail_list(si);
 }

 static void swap_range_alloc(struct swap_info_struct *si,
@@ -1354,7 +1346,7 @@ static bool get_swap_device_info(struct swap_info_struct *si)
         * up to dated.
         *
         * Paired with the spin_unlock() after setup_swap_info() in
-        * enable_swap_info(), and smp_wmb() in swapoff.
+        * swap_device_enable(), and smp_wmb() in swapoff.
         */
        smp_rmb();
        return true;
@@ -2970,58 +2962,87 @@ static int setup_swap_extents(struct swap_info_struct *sis,
        return generic_swapfile_activate(sis, swap_file, span);
 }

-static void _enable_swap_info(struct swap_info_struct *si)
+/*
+ * Mark a fully initialized swap device writable and expose it to the
+ * allocator. The caller must have resurrected its percpu ref first.
+ */
+static void swap_device_enable(struct swap_info_struct *si)
```

```
 {
-       atomic_long_add(si->pages, &nr_swap_pages);
-       total_swap_pages += si->pages;
+       spin_lock(&swap_lock);

-       assert_spin_locked(&swap_lock);
+       spin_lock(&swap_avail_lock);
+       si->flags |= SWP_WRITEOK;
+       spin_unlock(&swap_avail_lock);

+       atomic_long_add(si->pages, &nr_swap_pages);
+       total_swap_pages += si->pages;
        plist_add(&si->list, &swap_active_head);
+       spin_unlock(&swap_lock);

-       /* Add back to available list */
-       add_to_avail_list(si, true);
+       add_to_avail_list(si);
 }

-/*
- * Called after the swap device is ready, resurrect its percpu ref, it's now
- * safe to reference it. Add it to the list to expose it to the allocator.
- */
-static void enable_swap_info(struct swap_info_struct *si)
+static int swap_device_disable(struct address_space *mapping,
+                              struct swap_info_struct **swap_info)
 {
-       percpu_ref_resurrect(&si->users);
-       spin_lock(&swap_lock);
-       spin_lock(&si->lock);
-       _enable_swap_info(si);
-       spin_unlock(&si->lock);
-       spin_unlock(&swap_lock);
-}
+       struct swap_info_struct *si;
+       struct swap_cluster_info *ci;
+       unsigned long offset, end;
+       int err = -EINVAL;

-static void reinsert_swap_info(struct swap_info_struct *si)
-{
        spin_lock(&swap_lock);
-       spin_lock(&si->lock);
-       _enable_swap_info(si);
-       spin_unlock(&si->lock);
-       spin_unlock(&swap_lock);
-}
+       plist_for_each_entry(si, &swap_active_head, list) {
+               if ((si->flags & SWP_WRITEOK) &&
+                   si->swap_file->f_mapping == mapping) {
+                       err = 0;
+                       break;
+               }
+       }
+       if (err)
+               goto unlock;

-/*
- * Called after clearing SWP_WRITEOK, ensures cluster_alloc_range
- * see the updated flags, so there will be no more allocations.
- */
-static void wait_for_allocation(struct swap_info_struct *si)
-{
-       unsigned long offset;
-       unsigned long end = ALIGN(si->max, SWAPFILE_CLUSTER);
-       struct swap_cluster_info *ci;
+       /*
+        * Refuse swapoff while the device is pinned for hibernation.
```

```
+		 */
+		if (si->flags & SWP_HIBERNATION) {
+			err = -EBUSY;
+			goto unlock;
+		}

-		BUG_ON(si->flags & SWP_WRITEOK);
+		if (security_vm_enough_memory_mm(current->mm, si->pages)) {
+			err = -ENOMEM;
+			goto unlock;
+		}
+		vm_unacct_memory(si->pages);

+		spin_lock(&swap_avail_lock);
+		si->flags &= ~SWP_WRITEOK;
+		spin_unlock(&swap_avail_lock);
+
+		plist_del(&si->list, &swap_active_head);
+		total_swap_pages -= si->pages;
+		atomic_long_sub(si->pages, &nr_swap_pages);
+
+		end = ALIGN(si->max, SWAPFILE_CLUSTER);
+unlock:
+		spin_unlock(&swap_lock);
+		if (err)
+			return err;
+
+		del_from_avail_list(si, true);
+
+		/*
+		 * The swap allocator doesn't take swap_lock. Looping through every
+		 * cluster lock after clearing SWP_WRITEOK ensures that allocators see
+		 * the updated flag and that no allocation remains in flight.
+		 */
		for (offset = 0; offset < end; offset += SWAPFILE_CLUSTER) {
			ci = swap_cluster_lock(si, offset);
			swap_cluster_unlock(ci);
		}
+
+		*swap_info = si;
+		return 0;
 }

 static void free_swap_cluster_info(struct swap_cluster_info *cluster_info,
@@ -3066,7 +3087,6 @@ static void flush_percpu_swap_cluster(struct swap_info_struct *si)
		}
 }

-
 SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
 {
		struct swap_info_struct *p = NULL;
@@ -3075,7 +3095,7 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
		struct address_space *mapping;
		struct inode *inode;
		unsigned int maxpages;
-		int err, found = 0;
+		int err;

		if (!capable(CAP_SYS_ADMIN))
			return -EPERM;
@@ -3088,44 +3108,11 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
			return PTR_ERR(victim);

		mapping = victim->f_mapping;
-		spin_lock(&swap_lock);
-		plist_for_each_entry(p, &swap_active_head, list) {
-			if (p->flags & SWP_WRITEOK) {
-				if (p->swap_file->f_mapping == mapping) {
```

```
-                               found = 1;
-                               break;
-                       }
-               }
-       }
-       if (!found) {
-               err = -EINVAL;
-               spin_unlock(&swap_lock);
-               goto out_dput;
-       }
-
-       /* Refuse swapoff while the device is pinned for hibernation */
-       if (p->flags & SWP_HIBERNATION) {
-               err = -EBUSY;
-               spin_unlock(&swap_lock);
-               goto out_dput;
-       }
-
-       if (!security_vm_enough_memory_mm(current->mm, p->pages))
-               vm_unacct_memory(p->pages);
-       else {
-               err = -ENOMEM;
-               spin_unlock(&swap_lock);
-               goto out_dput;
-       }
-       spin_lock(&p->lock);
-       del_from_avail_list(p, true);
-       plist_del(&p->list, &swap_active_head);
-       atomic_long_sub(p->pages, &nr_swap_pages);
-       total_swap_pages -= p->pages;
-       spin_unlock(&p->lock);
-       spin_unlock(&swap_lock);
+       err = swap_device_disable(mapping, &p);
+       filp_close(victim, NULL);

-       wait_for_allocation(p);
+       if (err)
+               return err;

        set_current_oom_origin();
        err = try_to_unuse(p->type);
@@ -3133,8 +3120,8 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)

        if (err) {
                /* re-insert swap space back into swap_list */
-               reinsert_swap_info(p);
-               goto out_dput;
+               swap_device_enable(p);
+               return err;
        }

        /*
@@ -3194,13 +3181,10 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
        p->flags = 0;
        spin_unlock(&swap_lock);

-       err = 0;
        atomic_inc(&proc_poll_event);
        wake_up_interruptible(&proc_poll_wait);

-out_dput:
-       filp_close(victim, NULL);
-       return err;
+       return 0;
 }

 #ifdef CONFIG_PROC_FS
@@ -3622,7 +3606,7 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
```

```
        /*
         * Allocate or reuse existing !SWP_USED swap_info. The returned
         * si will stay in a dying status, so nothing will access its content
-        * until enable_swap_info resurrects its percpu ref and expose it.
+        * until swap_device_enable resurrects its percpu ref and expose it.
         */
        si = alloc_swap_info();
        if (IS_ERR(si))
@@ -3780,7 +3764,8 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
        si->swap_file = swap_file;

        /* Sets SWP_WRITEOK, resurrect the percpu ref, expose the swap device */
-       enable_swap_info(si);
+       percpu_ref_resurrect(&si->users);
+       swap_device_enable(si);

        pr_info("Adding %uk swap on %s.  Priority:%d extents:%d across:%lluk
%s%s%s%s\n",
                K(si->pages), name->name, si->prio, nr_extents,
--
2.55.0
```

```
==== Attachment 4 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0004-mm-swap-introduce-swap-device-iteration-helper.patch) ====
From da61e75e47484d79a0fccd15aa2616c48da93e4a Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:42 +0800
Subject: [RFC PATCH v2 04/13] mm/swap: introduce swap device iteration helper

There are many users that access the swap info array just to iterate
through the swap devices to find usable ones. Introduce a generic helper
for this to prepare for dropping the lock convention.

Also slightly adjust __swap_type_to_info to allow access of swapped off
devices by moving the sanity check into its only current caller, and this
way makes more sense too: the only caller is __swap_entry_to_info, where
we should never see an entry referring to a dead device.

Signed-off-by: Kairui Song <kasong@tencent.com>
---
 mm/swap.h     | 12 ++++----
 mm/swapfile.c | 77 +++++++++++++++++++++++++++++++-------------------------
 2 files changed, 53 insertions(+), 36 deletions(-)

diff --git a/mm/swap.h b/mm/swap.h
index d077e5893a42..00af0dc6dd3c 100644
--- a/mm/swap.h
+++ b/mm/swap.h
@@ -127,16 +127,16 @@ static inline unsigned int swp_cluster_offset(swp_entry_t entry)
  */
 static inline struct swap_info_struct *__swap_type_to_info(int type)
 {
-       struct swap_info_struct *si;
-
-       si = READ_ONCE(swap_info[type]); /* rcu_dereference() */
-       VM_WARN_ON_ONCE(percpu_ref_is_zero(&si->users)); /* race with swapoff */
-       return si;
+       return READ_ONCE(swap_info[type]); /* rcu_dereference() */
 }

 static inline struct swap_info_struct *__swap_entry_to_info(swp_entry_t entry)
 {
-       return __swap_type_to_info(swp_type(entry));
+       struct swap_info_struct *si;
+
+       si = __swap_type_to_info(swp_type(entry)); /* rcu_dereference() */
```

```
+       VM_WARN_ON_ONCE(percpu_ref_is_zero(&si->users)); /* race with swapoff */
+       return si;
 }

 static inline struct swap_cluster_info *__swap_offset_to_cluster(
diff --git a/mm/swapfile.c b/mm/swapfile.c
index 7b8523df42a1..327ab1e5803d 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -106,6 +106,34 @@ static DEFINE_SPINLOCK(swap_avail_lock);

 struct swap_info_struct *swap_info[MAX_SWAPFILES];

+static inline struct swap_info_struct *__swap_iter(int *i, unsigned long flag)
+{
+       lockdep_assert_held(&swap_lock);
+       while (*i < nr_swapfiles) {
+               struct swap_info_struct *si = __swap_type_to_info(*i);
+
+               VM_WARN_ON(!si);
+               (*i)++;
+               if (flag && !((si->flags & flag) == flag))
+                       continue;
+               return si;
+       }
+       return NULL;
+}
+
+#define __for_each_swap(si, flag)                      \
+       for (int __i = 0; ((si) = __swap_iter(&__i, flag));)
+
+/*
+ * for_each_swap - iterate through all allocated and inuse swap devices
+ * @si: the iterator
+ *
+ * Context: The caller must hold swap_lock. The lock may be dropped during
+ * the loop body but must be re-acquired before the next iteration.
+ */
+#define for_each_swap(si) __for_each_swap(si, SWP_USED)
+#define for_each_avail_swap(si) __for_each_swap(si, SWP_USED | SWP_WRITEOK)
+
 static struct kmem_cache *swap_table_cachep;

 /* Protects si->swap_file for /proc/swaps usage */
@@ -2203,26 +2231,18 @@ void swap_free_hibernation_slot(swp_entry_t entry)

 static int __find_hibernation_swap_type(dev_t device, sector_t offset)
 {
-       int type;
-
-       lockdep_assert_held(&swap_lock);
+       struct swap_info_struct *si;

        if (!device)
                return -EINVAL;

-       for (type = 0; type < nr_swapfiles; type++) {
-               struct swap_info_struct *sis = swap_info[type];
-
-               if (!(sis->flags & SWP_WRITEOK))
-                       continue;
-
-               if (device == sis->bdev->bd_dev) {
-                       struct swap_extent *se = first_se(sis);
-
-                       if (se->start_block == offset)
-                               return type;
+       for_each_avail_swap(si) {
+               if (device == si->bdev->bd_dev) {
```

```
+                       if (first_se(si)->start_block == offset)
+                               return si->type;
                }
        }
+
        return -ENODEV;
 }

@@ -2328,16 +2348,13 @@ int find_hibernation_swap_type(dev_t device, sector_t offset)

 int find_first_swap(dev_t *device)
 {
-       int type, ret = -ENODEV;
+       int ret = -ENODEV;
+       struct swap_info_struct *si;

        spin_lock(&swap_lock);
-       for (type = 0; type < nr_swapfiles; type++) {
-               struct swap_info_struct *sis = swap_info[type];
-
-               if (!(sis->flags & SWP_WRITEOK))
-                       continue;
-               *device = sis->bdev->bd_dev;
-               ret = type;
+       for_each_avail_swap(si) {
+               *device = si->bdev->bd_dev;
+               ret = si->type;
                break;
        }
        spin_unlock(&swap_lock);
@@ -2823,12 +2840,14 @@ static int try_to_unuse(unsigned int type)
  */
 static void drain_mmlist(void)
 {
+       struct swap_info_struct *si;
        struct list_head *p, *next;
-       unsigned int type;

-       for (type = 0; type < nr_swapfiles; type++)
-               if (swap_usage_in_pages(swap_info[type]))
+       for_each_swap(si) {
+               if (swap_usage_in_pages(si))
                        return;
+       }
+
        spin_lock(&mmlist_lock);
        list_for_each_safe(p, next, &init_mm.mmlist)
                list_del_init(p);
@@ -3813,14 +3832,12 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)

 void si_swapinfo(struct sysinfo *val)
 {
-       unsigned int type;
+       struct swap_info_struct *si;
        unsigned long nr_to_be_unused = 0;

        spin_lock(&swap_lock);
-       for (type = 0; type < nr_swapfiles; type++) {
-               struct swap_info_struct *si = swap_info[type];
-
-               if ((si->flags & SWP_USED) && !(si->flags & SWP_WRITEOK))
+       for_each_swap(si) {
+               if (!(si->flags & SWP_WRITEOK))
                        nr_to_be_unused += swap_usage_in_pages(si);
        }
        val->freeswap = atomic_long_read(&nr_swap_pages) + nr_to_be_unused;
--
2.55.0
```

```
==== Attachment 5 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0005-mm-swap-change-the-swapon-lock-into-a-percpu-rwsem.patch) ====
From 893fb7195cbe964162c7cb9efef72105afac041a Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:43 +0800
Subject: [RFC PATCH v2 05/13] mm/swap: change the swapon lock into a percpu
 rwsem

Reading swap metadata is a common hot path, while swapon and swapoff
are costly and very rare.  Convert the existing swap_lock spinlock
into a percpu rwsem so that readers can run concurrently without
contention.  This is a first step toward cleaning up the swap lock
model and we might already see a performance gain for existing
readers.

Also update the comments in swap_info_struct to reflect the lock
name change.

Signed-off-by: Kairui Song <kasong@tencent.com>
Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 include/linux/swap.h |  10 ++--
 mm/swapfile.c        | 115 +++++++++++++++++++++++++++++++-------------------
 2 files changed, 66 insertions(+), 59 deletions(-)

diff --git a/include/linux/swap.h b/include/linux/swap.h
index 5e1109010c89..13f732bd40bf 100644
--- a/include/linux/swap.h
+++ b/include/linux/swap.h
@@ -197,17 +197,17 @@ struct swap_extent {
 /*
  * Swap device flags, except the ones documented below, all are immutable
  * after exposed by swap_device_enable, and until the device is freed again
- * (SWP_USED unset). The exceptions:
- * - SWP_USED: Protected by swap_lock. Indicates the device is inuse. Once
+ * (SWP_USED unset). The exceptions are all protected by swapon_rwsem:
+ * - SWP_USED: Protected by swapon_rwsem. Indicates the device is inuse. Once
  *   set, won't be cleared unless all reference to this device is freed and
  *   swapoff finished.
- * - SWP_WRITEOK: Protected by both swap_lock and swap_avail_lock, clearing
+ * - SWP_WRITEOK: Protected by both swapon_rwsem and swap_avail_lock, clearing
  *   this flag also waits for all current cluster lock users to exit so
  *   checking this flag while holding any of these locks ensures the device
  *   is safe to use at the moment. Note: clearing this flag doesn't affect
  *   pending IO or async requests, it only prevents further entry allocation
  *   or new async request (e.g. discard) from initiating.
- * - SWP_HIBERNATION: Protected by swap_lock. Indicates if the device
+ * - SWP_HIBERNATION: Protected by swapon_rwsem. Indicates if the device
  *   is pinned for hibernation.
  */
 enum {
@@ -280,7 +280,7 @@ struct swap_info_struct {
        spinlock_t lock;                /*
                                         * Protect cluster lists. Other fields
                                         * are only changed at swapon/swapoff,
-                                        * so are protected by swap_lock.
+                                        * so are protected by swapon_rwsem.
                                         */
        struct work_struct discard_work; /* discard worker */
        struct work_struct reclaim_work; /* reclaim worker */
diff --git a/mm/swapfile.c b/mm/swapfile.c
index 327ab1e5803d..37bcce77b2cd 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -57,23 +57,29 @@ static void move_cluster(struct swap_info_struct *si,
                         enum swap_cluster_flags new_flags);
```

```
 /*
- * Protects the swap_info array, and the SWP_USED flag. swap_info contains
- * lazily allocated & freed swap device info structs, and SWP_USED indicates
- * which device is used, ~SWP_USED devices and can be reused.
- *
- * Also protects swap_active_head total_swap_pages, and the SWP_WRITEOK flag.
+ * Serializes swapon/swapoff (writers) and protects the swap_info
+ * array, nr_swapfiles, total_swap_pages, and part of swap device
+ * info content (see comment of swap_info_struct).  Readers
+ * (allocation, /proc/swaps, etc.) take percpu_down_read() which
+ * is cheap as the hot path.
+ */
+DEFINE_STATIC_PERCPU_RWSEM(swapon_rwsem);
+/*
+ * swap_info contains lazily allocated swap device info structs, and
+ * SWP_USED indicates which device is used, ~SWP_USED devices can be
+ * reused. Protected by swapon_rwsem, but reading could be lockless.
  */
-static DEFINE_SPINLOCK(swap_lock);
+struct swap_info_struct *swap_info[MAX_SWAPFILES];
 static unsigned int nr_swapfiles;
-atomic_long_t nr_swap_pages;
+long total_swap_pages;
+
 /*
  * Some modules use swappable objects and may try to swap them out under
  * memory pressure (via the shrinker). Before doing so, they may wish to
  * check to see if any swap space is available.
  */
+atomic_long_t nr_swap_pages;
 EXPORT_SYMBOL_GPL(nr_swap_pages);
-/* protected with swap_lock. reading in vm_swap_full() doesn't need lock */
-long total_swap_pages;
 #define DEF_SWAP_PRIO  -1
 unsigned long swapfile_maximum_size;
 #ifdef CONFIG_MIGRATION
@@ -85,7 +91,7 @@ static const char Bad_offset[] = "Bad swap offset entry ";

 /*
  * all active swap_info_structs
- * protected with swap_lock, and ordered by priority.
+ * protected with swapon_rwsem, and ordered by priority.
  */
 static PLIST_HEAD(swap_active_head);

@@ -95,20 +101,18 @@ static PLIST_HEAD(swap_active_head);
  * This is used by folio_alloc_swap() instead of swap_active_head
  * because swap_active_head includes all swap_info_structs,
  * but folio_alloc_swap() doesn't need to look at full ones.
- * This uses its own lock instead of swap_lock because when a
+ * This uses its own lock instead of swapon_rwsem because when a
  * swap_info_struct changes between not-full/full, it needs to
  * add/remove itself to/from this list, but the swap_info_struct->lock
- * is held and the locking order requires swap_lock to be taken
+ * is held and the locking order requires swapon_rwsem to be taken
  * before any swap_info_struct->lock.
  */
 static PLIST_HEAD(swap_avail_head);
 static DEFINE_SPINLOCK(swap_avail_lock);

-struct swap_info_struct *swap_info[MAX_SWAPFILES];
-
 static inline struct swap_info_struct *__swap_iter(int *i, unsigned long flag)
 {
-       lockdep_assert_held(&swap_lock);
+       lockdep_assert_held(&swapon_rwsem);
        while (*i < nr_swapfiles) {
                struct swap_info_struct *si = __swap_type_to_info(*i);
```

```
@@ -128,7 +132,7 @@ static inline struct swap_info_struct *__swap_iter(int *i, unsigned
long flag)
  * for_each_swap - iterate through all allocated and inuse swap devices
  * @si: the iterator
  *
- * Context: The caller must hold swap_lock. The lock may be dropped during
+ * Context: The caller must hold swapon_rwsem. The lock may be dropped during
  * the loop body but must be re-acquired before the next iteration.
  */
 #define for_each_swap(si) __for_each_swap(si, SWP_USED)
@@ -1371,10 +1375,10 @@ static bool get_swap_device_info(struct swap_info_struct *si)
        /*
         * Guarantee the si->users are checked before accessing other
         * fields of swap_info_struct, and si->flags (SWP_WRITEOK) is
-        * up to dated.
+        * up to date.
         *
-        * Paired with the spin_unlock() after setup_swap_info() in
-        * swap_device_enable(), and smp_wmb() in swapoff.
+        * Paired with percpu_up_write() in swap_device_enable(), and
+        * smp_wmb() after clearing SWP_WRITEOK in swapoff.
         */
        smp_rmb();
        return true;
@@ -1459,10 +1463,10 @@ static bool swap_sync_discard(void)
        bool ret = false;
        struct swap_info_struct *si, *next;

-       spin_lock(&swap_lock);
+       percpu_down_read(&swapon_rwsem);
 start_over:
        plist_for_each_entry_safe(si, next, &swap_active_head, list) {
-               spin_unlock(&swap_lock);
+               percpu_up_read(&swapon_rwsem);
                if (get_swap_device_info(si)) {
                        if (si->flags & SWP_PAGE_DISCARD)
                                ret = swap_do_scheduled_discard(si);
@@ -1471,11 +1475,11 @@ static bool swap_sync_discard(void)
                if (ret)
                        return true;

-               spin_lock(&swap_lock);
+               percpu_down_read(&swapon_rwsem);
                if (plist_node_empty(&next->list))
                        goto start_over;
        }
-       spin_unlock(&swap_lock);
+       percpu_up_read(&swapon_rwsem);

        return false;
 }
@@ -2268,7 +2272,7 @@ int pin_hibernation_swap_type(dev_t device, sector_t offset)
        int ret;
        struct swap_info_struct *si;

-       spin_lock(&swap_lock);
+       percpu_down_write(&swapon_rwsem);
        ret = __find_hibernation_swap_type(device, offset);
        if (ret < 0)
                goto out;
@@ -2292,7 +2296,7 @@ int pin_hibernation_swap_type(dev_t device, sector_t offset)
        si->flags |= SWP_HIBERNATION;

 out:
-       spin_unlock(&swap_lock);
+       percpu_up_write(&swapon_rwsem);
        return ret;
 }
```

```
@@ -2310,11 +2314,11 @@ void unpin_hibernation_swap_type(int type)
 {
        struct swap_info_struct *si;

-       spin_lock(&swap_lock);
+       percpu_down_write(&swapon_rwsem);
        si = swap_type_to_info(type);
        if (si)
                si->flags &= ~SWP_HIBERNATION;
-       spin_unlock(&swap_lock);
+       percpu_up_write(&swapon_rwsem);
 }

 /**
@@ -2339,9 +2343,9 @@ int find_hibernation_swap_type(dev_t device, sector_t offset)
 {
        int type;

-       spin_lock(&swap_lock);
+       percpu_down_read(&swapon_rwsem);
        type = __find_hibernation_swap_type(device, offset);
-       spin_unlock(&swap_lock);
+       percpu_up_read(&swapon_rwsem);

        return type;
 }
@@ -2351,13 +2355,13 @@ int find_first_swap(dev_t *device)
        int ret = -ENODEV;
        struct swap_info_struct *si;

-       spin_lock(&swap_lock);
+       percpu_down_read(&swapon_rwsem);
        for_each_avail_swap(si) {
                *device = si->bdev->bd_dev;
                ret = si->type;
                break;
        }
-       spin_unlock(&swap_lock);
+       percpu_up_read(&swapon_rwsem);
        return ret;
 }

@@ -2386,7 +2390,7 @@ unsigned int count_swap_pages(int type, int free)
 {
        unsigned int n = 0;

-       spin_lock(&swap_lock);
+       percpu_down_read(&swapon_rwsem);
        if ((unsigned int)type < nr_swapfiles) {
                struct swap_info_struct *sis = swap_info[type];

@@ -2398,7 +2402,7 @@ unsigned int count_swap_pages(int type, int free)
                }
                spin_unlock(&sis->lock);
        }
-       spin_unlock(&swap_lock);
+       percpu_up_read(&swapon_rwsem);
        return n;
 }
 #endif /* CONFIG_HIBERNATION */
@@ -2710,10 +2714,10 @@ static unsigned int find_next_to_unuse(struct swap_info_struct
*si,
        unsigned long swp_tb;

        /*
-        * No need for swap_lock here: we're just looking
+        * No need for swapon_rwsem here: we're just looking
         * for whether an entry is in use, not modifying it; false
```

```
          * hits are okay, and sys_swapoff() has already prevented new
-         * allocations from this area (while holding swap_lock).
+         * allocations from this area (while holding swapon_rwsem).
          */
         for (i = prev + 1; i < si->max; i++) {
                 swp_tb = swap_table_get(__swap_offset_to_cluster(si, i),
@@ -2834,8 +2838,8 @@ static int try_to_unuse(unsigned int type)

  /*
   * After a successful try_to_unuse, if no swap is now in use, we know
- * we can empty the mmlist.  swap_lock must be held on entry and exit.
- * Note that mmlist_lock nests inside swap_lock, and an mm must be
+ * we can empty the mmlist. swapon_rwsem must be held on entry and exit.
+ * Note that mmlist_lock nests inside swapon_rwsem, and an mm must be
   * added to the mmlist just after page_duplicate - before would be racy.
   */
  static void drain_mmlist(void)
@@ -2987,8 +2991,7 @@ static int setup_swap_extents(struct swap_info_struct *sis,
   */
  static void swap_device_enable(struct swap_info_struct *si)
  {
-       spin_lock(&swap_lock);
-
+       percpu_down_write(&swapon_rwsem);
         spin_lock(&swap_avail_lock);
         si->flags |= SWP_WRITEOK;
         spin_unlock(&swap_avail_lock);
@@ -2996,7 +2999,7 @@ static void swap_device_enable(struct swap_info_struct *si)
         atomic_long_add(si->pages, &nr_swap_pages);
         total_swap_pages += si->pages;
         plist_add(&si->list, &swap_active_head);
-       spin_unlock(&swap_lock);
+       percpu_up_write(&swapon_rwsem);

         add_to_avail_list(si);
  }
@@ -3009,7 +3012,7 @@ static int swap_device_disable(struct address_space *mapping,
         unsigned long offset, end;
         int err = -EINVAL;

-       spin_lock(&swap_lock);
+       percpu_down_write(&swapon_rwsem);
         plist_for_each_entry(si, &swap_active_head, list) {
                 if ((si->flags & SWP_WRITEOK) &&
                     si->swap_file->f_mapping == mapping) {
@@ -3044,14 +3047,14 @@ static int swap_device_disable(struct address_space *mapping,

         end = ALIGN(si->max, SWAPFILE_CLUSTER);
  unlock:
-       spin_unlock(&swap_lock);
+       percpu_up_write(&swapon_rwsem);
         if (err)
                 return err;

         del_from_avail_list(si, true);

         /*
-         * The swap allocator doesn't take swap_lock. Looping through every
+         * The swap allocator doesn't take swapon_rwsem. Looping through every
          * cluster lock after clearing SWP_WRITEOK ensures that allocators see
          * the updated flag and that no allocation remains in flight.
          */
@@ -3165,7 +3168,7 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
                 atomic_dec(&nr_rotate_swap);

         mutex_lock(&swapon_mutex);
-       spin_lock(&swap_lock);
+       percpu_down_write(&swapon_rwsem);
         spin_lock(&p->lock);
```

```
         drain_mmlist();

@@ -3176,7 +3179,7 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
         p->max = 0;
         p->cluster_info = NULL;
         spin_unlock(&p->lock);
-        spin_unlock(&swap_lock);
+        percpu_up_write(&swapon_rwsem);
         arch_swap_invalidate_area(p->type);
         zswap_swapoff(p->type);
         mutex_unlock(&swapon_mutex);
@@ -3195,10 +3198,14 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
          * Clear the SWP_USED flag after all resources are freed so that swapon
          * can reuse this swap_info in alloc_swap_info() safely.  It is ok to
          * not hold p->lock after we cleared its SWP_WRITEOK.
+         *
+         * The write lock ensures the flag clear is visible to lockless
+         * readers of swap_type_to_info() before alloc_swap_info() reuses
+         * this slot.
          */
-        spin_lock(&swap_lock);
+        percpu_down_write(&swapon_rwsem);
         p->flags = 0;
-        spin_unlock(&swap_lock);
+        percpu_up_write(&swapon_rwsem);

         atomic_inc(&proc_poll_event);
         wake_up_interruptible(&proc_poll_wait);
@@ -3358,13 +3365,13 @@ static struct swap_info_struct *alloc_swap_info(void)
                 return ERR_PTR(-ENOMEM);
         }

-        spin_lock(&swap_lock);
+        percpu_down_write(&swapon_rwsem);
         for (type = 0; type < nr_swapfiles; type++) {
                 if (!(swap_info[type]->flags & SWP_USED))
                         break;
         }
         if (type >= MAX_SWAPFILES) {
-                spin_unlock(&swap_lock);
+                percpu_up_write(&swapon_rwsem);
                 percpu_ref_exit(&p->users);
                 kvfree(p);
                 return ERR_PTR(-EPERM);
@@ -3389,7 +3396,7 @@ static struct swap_info_struct *alloc_swap_info(void)
         plist_node_init(&p->list, 0);
         plist_node_init(&p->avail_list, 0);
         p->flags = SWP_USED;
-        spin_unlock(&swap_lock);
+        percpu_up_write(&swapon_rwsem);
         if (defer) {
                 percpu_ref_exit(&defer->users);
                 kvfree(defer);
@@ -3815,9 +3822,9 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
          * Clear the SWP_USED flag after all resources are freed so
          * alloc_swap_info can reuse this si safely.
          */
-        spin_lock(&swap_lock);
+        percpu_down_write(&swapon_rwsem);
         si->flags = 0;
-        spin_unlock(&swap_lock);
+        percpu_up_write(&swapon_rwsem);
         if (inced_nr_rotate_swap)
                 atomic_dec(&nr_rotate_swap);
         if (swap_file)
@@ -3835,14 +3842,14 @@ void si_swapinfo(struct sysinfo *val)
         struct swap_info_struct *si;
         unsigned long nr_to_be_unused = 0;
```

```
-       spin_lock(&swap_lock);
+       percpu_down_read(&swapon_rwsem);
        for_each_swap(si) {
                if (!(si->flags & SWP_WRITEOK))
                        nr_to_be_unused += swap_usage_in_pages(si);
        }
        val->freeswap = atomic_long_read(&nr_swap_pages) + nr_to_be_unused;
        val->totalswap = total_swap_pages + nr_to_be_unused;
-       spin_unlock(&swap_lock);
+       percpu_up_read(&swapon_rwsem);
 }

 /*
--
2.55.0



==== Attachment 6 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0006-mm-swap-remove-swapon-mutex-and-update-proc-reader.patch) ====
From 3191606c8f9ec9fed20ff8e7449811d74785ac47 Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:44 +0800
Subject: [RFC PATCH v2 06/13] mm/swap: remove swapon mutex and update proc
 reader

Now that swapon_rwsem protects all swapon/swapoff-sensitive data,
the old swapon_mutex is redundant.  Remove it and convert the
/proc/swaps seq_file reader to use the rwsem instead.  Also convert
the swap_start iterator to use for_each_swap() for consistency.

Also adapt swap_next, it needs an explicit SWP_USED check. swap_file is
not reliably protected by swapon_rwsem, so checking SWP_USED is more
accurate and stable for filtering in-use devices.

Signed-off-by: Kairui Song <kasong@tencent.com>
---
 mm/swapfile.c | 19 +++++--------------
 1 file changed, 5 insertions(+), 14 deletions(-)

diff --git a/mm/swapfile.c b/mm/swapfile.c
index 37bcce77b2cd..b7b8d6c5d49d 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -140,9 +140,6 @@ static inline struct swap_info_struct *__swap_iter(int *i, unsigned
long flag)

 static struct kmem_cache *swap_table_cachep;

-/* Protects si->swap_file for /proc/swaps usage */
-static DEFINE_MUTEX(swapon_mutex);
-
 static DECLARE_WAIT_QUEUE_HEAD(proc_poll_wait);
 /* Activity counter to indicate that a swapon or swapoff has occurred */
 static atomic_t proc_poll_event = ATOMIC_INIT(0);
@@ -3167,7 +3164,6 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
        if (!(p->flags & SWP_SOLIDSTATE))
                atomic_dec(&nr_rotate_swap);

-       mutex_lock(&swapon_mutex);
        percpu_down_write(&swapon_rwsem);
        spin_lock(&p->lock);
        drain_mmlist();
@@ -3182,7 +3178,6 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
        percpu_up_write(&swapon_rwsem);
        arch_swap_invalidate_area(p->type);
        zswap_swapoff(p->type);
-       mutex_unlock(&swapon_mutex);
```

```
         kfree(p->global_cluster);
         p->global_cluster = NULL;
         free_swap_cluster_info(cluster_info, maxpages);
 @@ -3228,20 +3223,18 @@ static __poll_t swaps_poll(struct file *file, poll_table *wait)
         return EPOLLIN | EPOLLRDNORM;
  }

 -/* iterator */
  static void *swap_start(struct seq_file *swap, loff_t *pos)
  {
         struct swap_info_struct *si;
 -       int type;
         loff_t l = *pos;

 -       mutex_lock(&swapon_mutex);
 +       percpu_down_read(&swapon_rwsem);

         if (!l)
                 return SEQ_START_TOKEN;

 -       for (type = 0; (si = swap_type_to_info(type)); type++) {
 -               if (!(si->swap_file))
 +       for_each_swap(si) {
 +               if (!si->swap_file)
                         continue;
                 if (!--l)
                         return si;
 @@ -3262,7 +3255,7 @@ static void *swap_next(struct seq_file *swap, void *v, loff_t
 *pos)

         ++(*pos);
         for (; (si = swap_type_to_info(type)); type++) {
 -               if (!(si->swap_file))
 +               if (!(si->flags & SWP_USED) || !(si->swap_file))
                         continue;
                 return si;
         }
 @@ -3272,7 +3265,7 @@ static void *swap_next(struct seq_file *swap, void *v, loff_t
 *pos)

  static void swap_stop(struct seq_file *swap, void *v)
  {
 -       mutex_unlock(&swapon_mutex);
 +       percpu_up_read(&swapon_rwsem);
  }

  static int swap_show(struct seq_file *swap, void *v)
 @@ -3775,7 +3768,6 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
 swap_flags)
                 goto free_swap_zswap;
         }

 -       mutex_lock(&swapon_mutex);
         prio = DEF_SWAP_PRIO;
         if (swap_flags & SWAP_FLAG_PREFER)
                 prio = swap_flags & SWAP_FLAG_PRIO_MASK;
 @@ -3801,7 +3793,6 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
 swap_flags)
                 (si->flags & SWP_AREA_DISCARD) ? "s" : "",
                 (si->flags & SWP_PAGE_DISCARD) ? "c" : "");

 -       mutex_unlock(&swapon_mutex);
         atomic_inc(&proc_poll_event);
         wake_up_interruptible(&proc_poll_wait);

 --
 2.55.0
```

```
==== Attachment 7 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0007-mm-swap-consolidate-swap-inuse-accounting-helpers.patch) ====
From 74787f6dee14c5db90958027b2b7644a7daa261d Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:45 +0800
Subject: [RFC PATCH v2 07/13] mm/swap: consolidate swap inuse accounting
 helpers

Inline the swap_usage_add/sub helpers into their only callers and
rename the callers to better reflect what they actually do: track
per-device inuse page counts.

No functional change.

Signed-off-by: Kairui Song <kasong@tencent.com>
---
 mm/swapfile.c | 64 +++++++++++++++++++++++++----------------------------
 1 file changed, 27 insertions(+), 37 deletions(-)

diff --git a/mm/swapfile.c b/mm/swapfile.c
index b7b8d6c5d49d..65deea0f4c47 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -49,8 +49,8 @@
 #include "internal.h"
 #include "swap.h"

-static void swap_range_alloc(struct swap_info_struct *si,
-                             unsigned int nr_entries);
+static void swap_device_inuse_add(struct swap_info_struct *si,
+                                  unsigned int nr_entries);
 static bool folio_swapcache_freeable(struct folio *folio);
 static void move_cluster(struct swap_info_struct *si,
                          struct swap_cluster_info *ci, struct list_head *list,
@@ -984,7 +984,7 @@ static bool __swap_cluster_alloc_entries(struct swap_info_struct
*si,
        if (cluster_is_empty(ci))
                ci->order = order;
        ci->count += nr_pages;
-       swap_range_alloc(si, nr_pages);
+       swap_device_inuse_add(si, nr_pages);

        return true;
 }
@@ -1292,53 +1292,36 @@ static void add_to_avail_list(struct swap_info_struct *si)
 }

 /*
- * swap_usage_add / swap_usage_sub of each slot are serialized by ci->lock
- * within each cluster, so the total contribution to the global counter should
- * always be positive and cannot exceed the total number of usable slots.
+ * swap_device_inuse_add / swap_device_inuse_sub are called within cluster
+ * update critical sections and serialized by ci->lock within each cluster,
+ * so the total contribution to the global counter should always be positive
+ * and cannot exceed the total number of usable slots.
  */
-static bool swap_usage_add(struct swap_info_struct *si, unsigned int nr_entries)
+static void swap_device_inuse_add(struct swap_info_struct *si,
+                                  unsigned int nr_entries)
 {
-       long val = atomic_long_add_return_relaxed(nr_entries, &si->inuse_pages);
+       long inuse_pages;

        /*
         * If device is full, and SWAP_USAGE_OFFLIST_BIT is not set,
         * remove it from the plist.
         */
-       if (unlikely(val == si->pages)) {
```

```
-               del_from_avail_list(si, false);
-               return true;
-       }
-
-       return false;
-}
-
-static void swap_usage_sub(struct swap_info_struct *si, unsigned int nr_entries)
-{
-       long val = atomic_long_sub_return_relaxed(nr_entries, &si->inuse_pages);
-
-       /*
-        * If device is not full, and SWAP_USAGE_OFFLIST_BIT is set,
-        * add it to the plist.
-        */
-       if (unlikely(val & SWAP_USAGE_OFFLIST_BIT))
-               add_to_avail_list(si);
-}
-
-static void swap_range_alloc(struct swap_info_struct *si,
-                            unsigned int nr_entries)
-{
-       if (swap_usage_add(si, nr_entries)) {
+       inuse_pages = atomic_long_add_return_relaxed(nr_entries, &si->inuse_pages);
+       if (unlikely(inuse_pages == si->pages)) {
                if (vm_swap_full())
                        schedule_work(&si->reclaim_work);
+               del_from_avail_list(si, false);
        }
+
        atomic_long_sub(nr_entries, &nr_swap_pages);
 }

-static void swap_range_free(struct swap_info_struct *si, unsigned long offset,
-                           unsigned int nr_entries)
+static void swap_device_inuse_sub(struct swap_info_struct *si, unsigned long offset,
+                                 unsigned int nr_entries)
 {
        unsigned long end = offset + nr_entries - 1;
        void (*swap_slot_free_notify)(struct block_device *, unsigned long);
+       long inuse_pages;
        unsigned int i;

        for (i = 0; i < nr_entries; i++)
@@ -1362,7 +1345,14 @@ static void swap_range_free(struct swap_info_struct *si, unsigned
long offset,
         */
        smp_wmb();
        atomic_long_add(nr_entries, &nr_swap_pages);
-       swap_usage_sub(si, nr_entries);
+
+       /*
+        * If device is not full, and SWAP_USAGE_OFFLIST_BIT is set,
+        * add it back to the plist.
+        */
+       inuse_pages = atomic_long_sub_return_relaxed(nr_entries, &si->inuse_pages);
+       if (unlikely(inuse_pages & SWAP_USAGE_OFFLIST_BIT))
+               add_to_avail_list(si);
 }

 static bool get_swap_device_info(struct swap_info_struct *si)
@@ -1975,7 +1965,7 @@ void __swap_cluster_free_entries(struct swap_info_struct *si,
        if (batch_id)
                mem_cgroup_uncharge_swap(batch_id, ci_off - batch_off);

-       swap_range_free(si, ci_head + ci_start, nr_pages);
+       swap_device_inuse_sub(si, ci_head + ci_start, nr_pages);
        swap_cluster_assert_empty(ci, ci_start, nr_pages, false);
```

```
        if (!ci->count)
@@ -2827,7 +2817,7 @@ static int try_to_unuse(unsigned int type)
 success:
        /*
         * Make sure that further cleanups after try_to_unuse() returns happen
-        * after swap_range_free() reduces si->inuse_pages to 0.
+        * after swap_device_inuse_sub() reduces si->inuse_pages to 0.
         */
        smp_mb();
        return 0;
--
2.55.0



==== Attachment 8 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0008-mm-swap-change-back-to-use-each-swap-device-s-percpu.patch) ====
From 13e69c595fe6960283e2dd4caea3301fe25445d8 Mon Sep 17 00:00:00 2001
From: Youngjun Park <youngjun.park@lge.com>
Date: Tue, 14 Jul 2026 01:25:46 +0800
Subject: [RFC PATCH v2 08/13] mm/swap: change back to use each swap device's
 percpu cluster

This reverts commit 1b7e90020eb7 ("mm, swap: use percpu cluster as
allocation fast path").

The global percpu cluster causes several issues:

1) It prevents efficient swap allocation for users that do not want
   the default swap device policy.  Because the cluster cache sits
   above device selection, all allocation users must stick with the
   default allocation policy.  This fundamentally conflicts with
   incoming ideas like swap tiering.

2) It can cause priority inversion.  Consider two memcgs: memcg1 can
   access devices A and B, where A has higher priority; memcg2 can
   only access B.  Memcg2 can write the global percpu cluster with
   device B, then memcg1 takes B in the fast path even though
   higher-priority device A is not exhausted.

3) The fast-path / slow-path design bundled with the global per-CPU
   cache is problematic: only the fast path uses the global cache,
   and the slow path is forced to rotate the allocation list.  This
   complicates consumers like discard.

Revert to per-device percpu clusters first.  A later patch will
introduce new infrastructure for a high-performance global
allocation path.

Suggested-by: Kairui Song <kasong@tencent.com>
Co-developed-by: Baoquan He <bhe@redhat.com>
Signed-off-by: Baoquan He <bhe@redhat.com>
Signed-off-by: Youngjun Park <youngjun.park@lge.com>
Signed-off-by: Kairui Song <kasong@tencent.com>
---
 include/linux/swap.h |  13 ++-
 mm/swapfile.c        | 184 +++++++++++++++++++++++++++----------------
 2 files changed, 72 insertions(+), 125 deletions(-)

diff --git a/include/linux/swap.h b/include/linux/swap.h
index 13f732bd40bf..a005575b7097 100644
--- a/include/linux/swap.h
+++ b/include/linux/swap.h
@@ -245,10 +245,17 @@ enum {
 #endif

 /*
- * We keep using same cluster for rotational device so IO will be sequential.
- * The purpose is to optimize SWAP throughput on these device.
```

```
+ * We assign a cluster to each CPU, so each CPU can allocate swap entry from
+ * its own cluster and swapout sequentially. The purpose is to optimize swapout
+ * throughput.
  */
+struct percpu_cluster {
+       local_lock_t lock; /* Protect the percpu_cluster above */
+       unsigned int next[SWAP_NR_ORDERS]; /* Likely next allocation offset */
+};
+
 struct swap_sequential_cluster {
+       spinlock_t lock; /* Serialize usage of global cluster */
        unsigned int next[SWAP_NR_ORDERS]; /* Likely next allocation offset */
 };

@@ -271,8 +278,8 @@ struct swap_info_struct {
                                        /* list of cluster that are fragmented or
contented */
        unsigned int pages;             /* total of usable pages of swap */
        atomic_long_t inuse_pages;      /* number of those currently in use */
+       struct percpu_cluster   __percpu *percpu_cluster; /* per cpu's swap location */
        struct swap_sequential_cluster *global_cluster; /* Use one global cluster for
rotating device */
-       spinlock_t global_cluster_lock; /* Serialize usage of global cluster */
        struct rb_root swap_extent_root;/* root of the swap extent rbtree */
        struct block_device *bdev;      /* swap device or bdev of swap file */
        struct file *swap_file;         /* seldom referenced */
diff --git a/mm/swapfile.c b/mm/swapfile.c
index 65deea0f4c47..a574459bc6c8 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -146,18 +146,6 @@ static atomic_t proc_poll_event = ATOMIC_INIT(0);

 atomic_t nr_rotate_swap = ATOMIC_INIT(0);

-struct percpu_swap_cluster {
-       struct swap_info_struct *si[SWAP_NR_ORDERS];
-       unsigned long offset[SWAP_NR_ORDERS];
-       local_lock_t lock;
-};
-
-static DEFINE_PER_CPU(struct percpu_swap_cluster, percpu_swap_cluster) = {
-       .si = { NULL },
-       .offset = { SWAP_ENTRY_INVALID },
-       .lock = INIT_LOCAL_LOCK(),
-};
-
 /* May return NULL on invalid type, caller must check for NULL return */
 static struct swap_info_struct *swap_type_to_info(int type)
 {
@@ -562,9 +550,10 @@ swap_cluster_populate(struct swap_info_struct *si,
         * Only cluster isolation from the allocator does table allocation.
         * Swap allocator uses percpu clusters and holds the local lock.
         */
-       lockdep_assert_held(&this_cpu_ptr(&percpu_swap_cluster)->lock);
-       if (!(si->flags & SWP_SOLIDSTATE))
-               lockdep_assert_held(&si->global_cluster_lock);
+       if (si->flags & SWP_SOLIDSTATE)
+               lockdep_assert_held(this_cpu_ptr(&si->percpu_cluster->lock));
+       else
+               lockdep_assert_held(&si->global_cluster->lock);
        lockdep_assert_held(&ci->lock);

        if (!swap_cluster_alloc_table(ci, __GFP_HIGH | __GFP_NOMEMALLOC |
@@ -577,9 +566,10 @@ swap_cluster_populate(struct swap_info_struct *si,
         * the potential recursive allocation is limited.
         */
        spin_unlock(&ci->lock);
-       if (!(si->flags & SWP_SOLIDSTATE))
-               spin_unlock(&si->global_cluster_lock);
```

```
-       local_unlock(&percpu_swap_cluster.lock);
+       if (si->flags & SWP_SOLIDSTATE)
+               local_unlock(&si->percpu_cluster->lock);
+       else
+               spin_unlock(&si->global_cluster->lock);

        ret = swap_cluster_alloc_table(ci, __GFP_HIGH | __GFP_NOMEMALLOC |
                                           GFP_KERNEL);
@@ -592,9 +582,10 @@ swap_cluster_populate(struct swap_info_struct *si,
         * could happen with ignoring the percpu cluster is fragmentation,
         * which is acceptable since this fallback and race is rare.
         */
-       local_lock(&percpu_swap_cluster.lock);
-       if (!(si->flags & SWP_SOLIDSTATE))
-               spin_lock(&si->global_cluster_lock);
+       if (si->flags & SWP_SOLIDSTATE)
+               local_lock(&si->percpu_cluster->lock);
+       else
+               spin_lock(&si->global_cluster->lock);
        spin_lock(&ci->lock);

        if (ret) {
@@ -700,7 +691,7 @@ static bool swap_do_scheduled_discard(struct swap_info_struct *si)
                ci = list_first_entry(&si->discard_clusters, struct swap_cluster_info,
list);
                /*
                 * Delete the cluster from list to prepare for discard, but keep
-                * the CLUSTER_FLAG_DISCARD flag, percpu_swap_cluster could be
+                * the CLUSTER_FLAG_DISCARD flag, there could be percpu_cluster
                 * pointing to it, or ran into by relocate_cluster.
                 */
                list_del(&ci->list);
@@ -1032,12 +1023,10 @@ static unsigned int alloc_swap_scan_cluster(struct
swap_info_struct *si,
 out:
        relocate_cluster(si, ci);
        swap_cluster_unlock(ci);
-       if (si->flags & SWP_SOLIDSTATE) {
-               this_cpu_write(percpu_swap_cluster.offset[order], next);
-               this_cpu_write(percpu_swap_cluster.si[order], si);
-       } else {
+       if (si->flags & SWP_SOLIDSTATE)
+               this_cpu_write(si->percpu_cluster->next[order], next);
+       else
                si->global_cluster->next[order] = next;
-       }
        return found;
 }

@@ -1138,13 +1127,17 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
        if (order && !(si->flags & SWP_BLKDEV))
                return 0;

-       if (!(si->flags & SWP_SOLIDSTATE)) {
+       if (si->flags & SWP_SOLIDSTATE) {
+               /* Fast path using per CPU cluster */
+               local_lock(&si->percpu_cluster->lock);
+               offset = __this_cpu_read(si->percpu_cluster->next[order]);
+       } else {
                /* Serialize HDD SWAP allocation for each device. */
-               spin_lock(&si->global_cluster_lock);
+               spin_lock(&si->global_cluster->lock);
                offset = si->global_cluster->next[order];
-               if (offset == SWAP_ENTRY_INVALID)
-                       goto new_cluster;
        }

+       if (offset != SWAP_ENTRY_INVALID) {
```

```
                ci = swap_cluster_lock(si, offset);
                /* Cluster could have been used by another order */
                if (cluster_is_usable(ci, order)) {
@@ -1158,7 +1151,6 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
                        goto done;
        }

-new_cluster:
        /*
         * If the device need discard, prefer new cluster over nonfull
         * to spread out the writes.
@@ -1215,8 +1207,10 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
                        goto done;
        }
 done:
-       if (!(si->flags & SWP_SOLIDSTATE))
-               spin_unlock(&si->global_cluster_lock);
+       if (si->flags & SWP_SOLIDSTATE)
+               local_unlock(&si->percpu_cluster->lock);
+       else
+               spin_unlock(&si->global_cluster->lock);

        return found;
 }
@@ -1371,41 +1365,8 @@ static bool get_swap_device_info(struct swap_info_struct *si)
        return true;
 }

-/*
- * Fast path try to get swap entries with specified order from current
- * CPU's swap entry pool (a cluster).
- */
-static bool swap_alloc_fast(struct folio *folio)
-{
-       unsigned int order = folio_order(folio);
-       struct swap_cluster_info *ci;
-       struct swap_info_struct *si;
-       unsigned int offset;
-
-       /*
-        * Once allocated, swap_info_struct will never be completely freed,
-        * so checking it's liveness by get_swap_device_info is enough.
-        */
-       si = this_cpu_read(percpu_swap_cluster.si[order]);
-       offset = this_cpu_read(percpu_swap_cluster.offset[order]);
-       if (!si || !offset || !get_swap_device_info(si))
-               return false;
-
-       ci = swap_cluster_lock(si, offset);
-       if (cluster_is_usable(ci, order)) {
-               if (cluster_is_empty(ci))
-                       offset = cluster_offset(si, ci);
-               alloc_swap_scan_cluster(si, ci, folio, offset);
-       } else {
-               swap_cluster_unlock(ci);
-       }
-
-       put_swap_device(si);
-       return folio_test_swapcache(folio);
-}
-
 /* Rotate the device and switch to a new cluster */
-static void swap_alloc_slow(struct folio *folio)
+static void swap_alloc_entry(struct folio *folio)
 {
        struct swap_info_struct *si, *next;
```

```
@@ -1775,10 +1736,7 @@ int folio_alloc_swap(struct folio *folio)
        }

 again:
-       local_lock(&percpu_swap_cluster.lock);
-       if (!swap_alloc_fast(folio))
-               swap_alloc_slow(folio);
-       local_unlock(&percpu_swap_cluster.lock);
+       swap_alloc_entry(folio);

        if (!order && unlikely(!folio_test_swapcache(folio))) {
                if (swap_sync_discard())
@@ -2167,31 +2125,14 @@ void swap_put_entries_direct(swp_entry_t entry, int nr)
  */
 swp_entry_t swap_alloc_hibernation_slot(int type)
 {
-       struct swap_info_struct *pcp_si, *si = swap_type_to_info(type);
-       unsigned long pcp_offset, offset = SWAP_ENTRY_INVALID;
-       struct swap_cluster_info *ci;
+       struct swap_info_struct *si = swap_type_to_info(type);
+       unsigned long offset = SWAP_ENTRY_INVALID;
        swp_entry_t entry = {0};

        if (!si)
                goto fail;

-       /*
-        * Try the local cluster first if it matches the device. If
-        * not, try grab a new cluster and override local cluster.
-        */
-       local_lock(&percpu_swap_cluster.lock);
-       pcp_si = this_cpu_read(percpu_swap_cluster.si[0]);
-       pcp_offset = this_cpu_read(percpu_swap_cluster.offset[0]);
-       if (pcp_si == si && pcp_offset) {
-               ci = swap_cluster_lock(si, pcp_offset);
-               if (cluster_is_usable(ci, 0))
-                       offset = alloc_swap_scan_cluster(si, ci, NULL, pcp_offset);
-               else
-                       swap_cluster_unlock(ci);
-       }
-       if (!offset)
-               offset = cluster_alloc_swap_entry(si, NULL);
-       local_unlock(&percpu_swap_cluster.lock);
+       offset = cluster_alloc_swap_entry(si, NULL);
        if (offset)
                entry = swp_entry(si->type, offset);

@@ -3075,27 +3016,6 @@ static void free_swap_cluster_info(struct swap_cluster_info
*cluster_info,
        kvfree(cluster_info);
 }

-/*
- * Called after swap device's reference count is dead, so
- * neither scan nor allocation will use it.
- */
-static void flush_percpu_swap_cluster(struct swap_info_struct *si)
-{
-       int cpu, i;
-       struct swap_info_struct **pcp_si;
-
-       for_each_possible_cpu(cpu) {
-               pcp_si = per_cpu_ptr(percpu_swap_cluster.si, cpu);
-               /*
-                * Invalidate the percpu swap cluster cache, si->users
-                * is dead, so no new user will point to it, just flush
-                * any existing user.
-                */
-               for (i = 0; i < SWAP_NR_ORDERS; i++)
```

```
-                       cmpxchg(&pcp_si[i], si, NULL);
-       }
-}
-
 SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
 {
        struct swap_info_struct *p = NULL;
@@ -3147,7 +3067,6 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)

        flush_work(&p->discard_work);
        flush_work(&p->reclaim_work);
-       flush_percpu_swap_cluster(p);

        destroy_swap_extents(p, p->swap_file);

@@ -3168,6 +3087,8 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
        percpu_up_write(&swapon_rwsem);
        arch_swap_invalidate_area(p->type);
        zswap_swapoff(p->type);
+       free_percpu(p->percpu_cluster);
+       p->percpu_cluster = NULL;
        kfree(p->global_cluster);
        p->global_cluster = NULL;
        free_swap_cluster_info(cluster_info, maxpages);
@@ -3516,7 +3437,7 @@ static int setup_swap_clusters_info(struct swap_info_struct *si,
 {
        unsigned long nr_clusters = DIV_ROUND_UP(maxpages, SWAPFILE_CLUSTER);
        struct swap_cluster_info *cluster_info;
-       int err = -ENOMEM;
+       int cpu, err = -ENOMEM;
        unsigned long i;

        cluster_info = kvzalloc_objs(*cluster_info, nr_clusters);
@@ -3526,13 +3447,26 @@ static int setup_swap_clusters_info(struct swap_info_struct *si,
        for (i = 0; i < nr_clusters; i++)
                spin_lock_init(&cluster_info[i].lock);

-       if (!(si->flags & SWP_SOLIDSTATE)) {
+       if (si->flags & SWP_SOLIDSTATE) {
+               si->percpu_cluster = alloc_percpu(struct percpu_cluster);
+               if (!si->percpu_cluster)
+                       goto err;
+
+               for_each_possible_cpu(cpu) {
+                       struct percpu_cluster *cluster;
+
+                       cluster = per_cpu_ptr(si->percpu_cluster, cpu);
+                       for (i = 0; i < SWAP_NR_ORDERS; i++)
+                               cluster->next[i] = SWAP_ENTRY_INVALID;
+                       local_lock_init(&cluster->lock);
+               }
+       } else {
                si->global_cluster = kmalloc_obj(*si->global_cluster);
                if (!si->global_cluster)
                        goto err;
                for (i = 0; i < SWAP_NR_ORDERS; i++)
                        si->global_cluster->next[i] = SWAP_ENTRY_INVALID;
-               spin_lock_init(&si->global_cluster_lock);
+               spin_lock_init(&si->global_cluster->lock);
        }

        /*
@@ -3694,11 +3628,6 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)

        maxpages = si->max;

-       /* Set up the swap cluster info */
-       error = setup_swap_clusters_info(si, swap_header, maxpages);
```

```
-       if (error)
-               goto bad_swap_unlock_inode;
-
        if (si->bdev && bdev_stable_writes(si->bdev))
                si->flags |= SWP_STABLE_WRITES;

@@ -3712,6 +3641,15 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
                inced_nr_rotate_swap = true;
        }

+       /*
+        * Set up the swap cluster info. This must run after SWP_SOLIDSTATE
+        * is determined above, as it decides whether to allocate the per-CPU
+        * cluster (solid state) or the global cluster (rotational).
+        */
+       error = setup_swap_clusters_info(si, swap_header, maxpages);
+       if (error)
+               goto bad_swap_unlock_inode;
+
        if ((swap_flags & SWAP_FLAG_DISCARD) &&
            si->bdev && bdev_max_discard_sectors(si->bdev)) {
                /*
@@ -3793,6 +3731,8 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
 bad_swap_unlock_inode:
        inode_unlock(inode);
 bad_swap:
+       free_percpu(si->percpu_cluster);
+       si->percpu_cluster = NULL;
        kfree(si->global_cluster);
        si->global_cluster = NULL;
        inode = NULL;
--
2.55.0
```

```
==== Attachment 9 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0009-mm-swap-add-priority-queue-for-swap-device-allocatio.patch) ====
From a05e05b8d1c1474b8ca8e7b0d48168b64905f068 Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:47 +0800
Subject: [RFC PATCH v2 09/13] mm/swap: add priority queue for swap device
 allocation

The swap allocator uses swap_avail_head, a plist ordered by priority,
to select devices. Devices at the same priority are rotated with
plist_requeue(), so every cluster transition serializes allocators on
swap_avail_lock and repeatedly drops and reacquires that global lock.

Replace device selection with a priority queue made of immutable rings.
Each ring contains devices at one priority, and each CPU has a local
reader that rotates through a ring after a fixed allocation quota. A
task-local cursor keeps a retry walk stable even if another task updates
the shared reader between attempts, so each peer is visited exactly once
before the allocator considers a lower priority.

Disable task migration across the retry walk so queue selection, quota
accounting and per-CPU cluster allocation stay on the same CPU. This
preserves per-CPU pacing without making the sleepable allocation loop an
atomic context.

Keep the queue structure stable under swapon_rwsem. Full or disabled
devices remain in their ring with a tag in the low bit of the stored
pointer. Serialize tag writers with swap_queue_update_lock and pair the
lockless full-pointer reads and writes with READ_ONCE() and WRITE_ONCE().
The in-use counter carries a separate off-list bit so full-to-available
transitions update the counter and pointer tag consistently.
```

```
Publish swap_file, the live percpu reference, queue membership and
SWP_WRITEOK in one swapon writer section. Preserve the writer-serialized
swapoff lookup and disable invariant established earlier in the series.

For large folios, try every device in the selected priority ring before
returning -E2BIG to request a split. Do not fall back to a lower priority
device merely because the first same-priority device is fragmented.

The old available plist is still maintained in parallel in this commit
so the transition remains bisectable. It is removed by the next patch.

Link: https://lore.kernel.org/20260714-swap-pcp-prig-v1-9-de9b164ed419@tencent.com
Link: https://lore.kernel.org/alZ7UBXweuuOX4qz@yjaykim-PowerEdge-T330
Link: https://lore.kernel.org/77d6da3d-10af-49a1-a356-72aa8b462e85@gmail.com
Signed-off-by: Kairui Song <kasong@tencent.com>
Co-developed-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 include/linux/swap.h |   5 +-
 mm/swapfile.c        | 526 ++++++++++++++++++++++++++++++++++++++------
 2 files changed, 464 insertions(+), 67 deletions(-)

diff --git a/include/linux/swap.h b/include/linux/swap.h
index a005575b7097..94173e59dc0b 100644
--- a/include/linux/swap.h
+++ b/include/linux/swap.h
@@ -201,8 +201,9 @@ struct swap_extent {
  * - SWP_USED: Protected by swapon_rwsem. Indicates the device is inuse. Once
  *   set, won't be cleared unless all reference to this device is freed and
  *   swapoff finished.
- * - SWP_WRITEOK: Protected by both swapon_rwsem and swap_avail_lock, clearing
- *   this flag also waits for all current cluster lock users to exit so
+ * - SWP_WRITEOK: Protected by both swapon_rwsem and swap_queue_update_lock.
+ *   Clearing this flag is followed by waiting for all current cluster lock
+ *   users to exit, so
  *   checking this flag while holding any of these locks ensures the device
  *   is safe to use at the moment. Note: clearing this flag doesn't affect
  *   pending IO or async requests, it only prevents further entry allocation
diff --git a/mm/swapfile.c b/mm/swapfile.c
index a574459bc6c8..2c6765506970 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -55,6 +55,7 @@ static bool folio_swapcache_freeable(struct folio *folio);
 static void move_cluster(struct swap_info_struct *si,
                          struct swap_cluster_info *ci, struct list_head *list,
                          enum swap_cluster_flags new_flags);
+static bool get_swap_device_info(struct swap_info_struct *si);

 /*
  * Serializes swapon/swapoff (writers) and protects the swap_info
@@ -160,20 +161,382 @@ static struct swap_info_struct *swap_entry_to_info(swp_entry_t
entry)
        return swap_type_to_info(swp_type(entry));
 }

+/*
+ * All available swap_info_structs are grouped by priority rings, the rings
+ * are ordered in a queue by priority (higher prio value = higher priority).
+ * The allocator iterates and rotates devices within each priority ring.
+ * When all devices in a ring are iterated, it goes to the next lower
+ * priority ring.
+ */
+struct swap_prio_ring {
+       int prio;
+       unsigned int size;
+       struct swap_info_struct *dev[] __counted_by(size);
+};
+
```

```
+/*
+ * The ring is protected by swapon_rwsem so updating it is costly. To make
+ * the allocator and other users skip full devices faster, the lowest bit of
+ * a device pointer is used to mark it disabled (temporarily unavailable).
+ * This relies on the natural alignment of struct swap_info_struct.
+ */
+#define SWAP_DEVICE_MASKED_SHIFT       0
+#define SWAP_DEVICE_MASKED_BIT         BIT(SWAP_DEVICE_MASKED_SHIFT)
+static_assert(__alignof__(struct swap_info_struct) >= 2);
+
+/*
+ * Serializes queue content mutations and keeps SWAP_USAGE_OFFLIST_BIT
+ * consistent with the masked state of each device pointer.
+ */
+static DEFINE_SPINLOCK(swap_queue_update_lock);
+
+/*
+ * Swap queue is protected by both swap_queue_update_lock and swapon_rwsem.
+ * Only swapon/swapoff will take the write lock, and modify the queue length
+ * or any ring's length. swap_queue_update_lock protects the content so
+ * devices can be masked easily without taking the writelock, which is heavy.
+ */
+static struct swap_prio_ring **swap_queue;
+static unsigned int swap_queue_len;
+
+/*
+ * Each CPU has its read iterator, so the queue itself will remain read
+ * only and the CPU side reader rotates by iterating the devices
+ * periodically using the counter.
+ */
+#define SWAP_ROUND_ROBIN_QUOTA SWAPFILE_CLUSTER
+struct swap_ring_iterator {
+       int offset;
+       long rr_counter;
+};
+
+struct swap_queue_reader {
+       local_lock_t lock;
+       struct swap_ring_iterator ri[];
+};
+
+struct swap_queue_cursor {
+       bool valid;
+       bool ring_only;
+       unsigned int ring_idx;
+       unsigned int offset;
+};
+
+static __percpu struct swap_queue_reader *swap_queue_readers;
+
+static inline bool swap_device_masked(struct swap_info_struct *si)
+{
+       return (unsigned long)si & SWAP_DEVICE_MASKED_BIT;
+}
+
+static inline struct swap_info_struct *swap_device_mask_ptr(struct swap_info_struct
*si)
+{
+       return (struct swap_info_struct *)((unsigned long)si |
+                                          SWAP_DEVICE_MASKED_BIT);
+}
+
+static inline struct swap_info_struct *swap_device_unmask_ptr(struct swap_info_struct
*si)
+{
+       return (struct swap_info_struct *)((unsigned long)si & ~SWAP_DEVICE_MASKED_BIT);
+}
+
+static struct swap_queue_reader __percpu *swap_queue_prealloc_readers(int nr_rings,
```

```
 gfp_t gfp)
 +{
 +       struct swap_queue_reader __percpu *readers;
 +
 +       if (!nr_rings)
 +               return NULL;
 +
 +       readers = __alloc_percpu_gfp(struct_size(readers, ri, nr_rings),
 +                                    __alignof__(*readers), gfp);
 +       return readers;
 +}
 +
 +static void swap_queue_install_readers(struct swap_queue_reader __percpu *readers)
 +{
 +       int ring_idx, cpu;
 +       struct swap_prio_ring *ring;
 +
 +       free_percpu(swap_queue_readers);
 +       swap_queue_readers = readers;
 +       if (!readers)
 +               return;
 +
 +       /* Distribute each CPU's swap IO fairly across devices. */
 +       for_each_possible_cpu(cpu) {
 +               local_lock_init(&per_cpu_ptr(readers, cpu)->lock);
 +
 +               for (ring_idx = 0; ring_idx < swap_queue_len; ring_idx++) {
 +                       ring = swap_queue[ring_idx];
 +                       per_cpu_ptr(readers, cpu)->ri[ring_idx].offset =
 +                                   cpu % ring->size;
 +                       per_cpu_ptr(readers, cpu)->ri[ring_idx].rr_counter =
 +                                   SWAP_ROUND_ROBIN_QUOTA;
 +               }
 +       }
 +}
 +
 +static struct swap_info_struct *swap_queue_get_device(long nr_alloc, int nr_iter,
 +                                                      struct swap_queue_cursor *cursor)
 +{
 +       bool rotate = false;
 +       struct swap_info_struct *si;
 +       struct swap_ring_iterator *ri;
 +       struct swap_prio_ring *ring;
 +       unsigned int dev_idx, queue_idx;
 +
 +       if (!swap_queue_len)
 +               return ERR_PTR(-ENOENT);
 +
 +       queue_idx = 0;
 +       while (nr_iter >= swap_queue[queue_idx]->size) {
 +               nr_iter -= swap_queue[queue_idx]->size;
 +               if (++queue_idx >= swap_queue_len)
 +                       return ERR_PTR(cursor->ring_only ? -E2BIG : -ENOENT);
 +       }
 +       if (cursor->ring_only && cursor->ring_idx != queue_idx)
 +               return ERR_PTR(-E2BIG);
 +
 +       ring = swap_queue[queue_idx];
 +       local_lock(&swap_queue_readers->lock);
 +       ri = this_cpu_ptr(&swap_queue_readers->ri[queue_idx]);
 +       /*
 +        * Snapshot the starting offset for this allocation's walk. The shared
 +        * iterator can move between retries, but the cursor must visit every
 +        * device in the ring exactly once before falling through.
 +        */
 +       if (!cursor->valid || cursor->ring_idx != queue_idx) {
 +               cursor->valid = true;
 +               cursor->ring_idx = queue_idx;
 +               if (ri->rr_counter < nr_alloc)
```

```
+                       rotate = true;
+               else if (ri->offset >= ring->size)
+                       rotate = true;
+               if (rotate) {
+                       ri->offset++;
+                       ri->offset %= ring->size;
+                       ri->rr_counter = SWAP_ROUND_ROBIN_QUOTA;
+               }
+               cursor->offset = ri->offset;
+       }
+
+       dev_idx = (cursor->offset + nr_iter) % ring->size;
+       if (nr_iter) {
+               ri->offset = dev_idx;
+               ri->rr_counter = SWAP_ROUND_ROBIN_QUOTA;
+       }
+       ri->rr_counter -= nr_alloc;
+       si = READ_ONCE(ring->dev[dev_idx]);
+       local_unlock(&swap_queue_readers->lock);
+
+       if (swap_device_masked(si))
+               return ERR_PTR(-EBUSY);
+
+       si = swap_device_unmask_ptr(si);
+       return si;
+}
+
+static bool swap_queue_find(struct swap_info_struct *si,
+                           unsigned int *ring_idx, unsigned int *dev_idx)
+{
+       unsigned int i, j;
+       struct swap_prio_ring *ring;
+
+       lockdep_assert(lockdep_is_held(&swapon_rwsem) ||
+                      lockdep_is_held(&swap_queue_update_lock));
+
+       for (i = 0; i < swap_queue_len; i++) {
+               ring = swap_queue[i];
+               if (ring->prio != si->prio)
+                       continue;
+               for (j = 0; j < ring->size; j++) {
+                       if (swap_device_unmask_ptr(READ_ONCE(ring->dev[j])) != si)
+                               continue;
+                       *ring_idx = i;
+                       *dev_idx = j;
+                       return true;
+               }
+       }
+       return false;
+}
+
+static void swap_queue_mask(struct swap_info_struct *si)
+{
+       unsigned int ring_idx, dev_idx;
+
+       lockdep_assert_held(&swap_queue_update_lock);
+       if (swap_queue_find(si, &ring_idx, &dev_idx))
+               WRITE_ONCE(swap_queue[ring_idx]->dev[dev_idx],
+                          swap_device_mask_ptr(si));
+}
+
+static void swap_queue_unmask(struct swap_info_struct *si)
+{
+       unsigned int ring_idx, dev_idx;
+
+       lockdep_assert_held(&swap_queue_update_lock);
+       if (swap_queue_find(si, &ring_idx, &dev_idx))
+               WRITE_ONCE(swap_queue[ring_idx]->dev[dev_idx], si);
+}
```

```
+
+static int swap_queue_add(struct swap_info_struct *si)
+{
+	struct swap_prio_ring **new_queue = NULL, **old_queue = NULL;
+	struct swap_queue_reader __percpu *new_readers = NULL;
+	struct swap_prio_ring *ring, *new_ring = NULL, *old_ring = NULL;
+	int prio = si->prio;
+	int i, pos, err = -ENOMEM;
+	gfp_t gfp;
+
+	/* Swap not usable here because this is swap, just reclaim cache. */
+	gfp = GFP_NOIO | __GFP_HIGH;
+	lockdep_assert_held_write(&swapon_rwsem);
+
+	for (pos = 0; pos < swap_queue_len; pos++) {
+		if (swap_queue[pos]->prio == prio)
+			goto add_to_ring;
+		if (swap_queue[pos]->prio < prio)
+			break;
+	}
+
+	/* No ring at this priority: insert a new one at pos. */
+	new_readers = swap_queue_prealloc_readers(swap_queue_len + 1, gfp);
+	if (!new_readers)
+		goto failed;
+	new_queue = kmalloc_array(swap_queue_len + 1, sizeof(*swap_queue), gfp);
+	if (!new_queue)
+		goto failed;
+	new_ring = kmalloc(struct_size(new_ring, dev, 1), gfp);
+	if (!new_ring)
+		goto failed;
+	if (!get_swap_device_info(si))
+		goto failed;
+
+	new_ring->prio = prio;
+	new_ring->size = 1;
+	new_ring->dev[0] = si;
+	for (i = 0; i < pos; i++)
+		new_queue[i] = swap_queue[i];
+	new_queue[pos] = new_ring;
+	for (i = pos; i < swap_queue_len; i++)
+		new_queue[i + 1] = swap_queue[i];
+
+	spin_lock(&swap_queue_update_lock);
+	old_queue = swap_queue;
+	swap_queue = new_queue;
+	swap_queue_len++;
+	spin_unlock(&swap_queue_update_lock);
+	kfree(old_queue);
+
+	swap_queue_install_readers(new_readers);
+	return 0;
+
+add_to_ring:
+	ring = swap_queue[pos];
+	new_ring = kmalloc(struct_size(ring, dev, ring->size + 1), gfp);
+	if (!new_ring)
+		goto failed;
+	if (!get_swap_device_info(si))
+		goto failed;
+	spin_lock(&swap_queue_update_lock);
+	memcpy(new_ring, ring, struct_size(ring, dev, ring->size));
+	new_ring->size++;
+	new_ring->dev[new_ring->size - 1] = si;
+	old_ring = swap_queue[pos];
+	swap_queue[pos] = new_ring;
+	spin_unlock(&swap_queue_update_lock);
+	kfree(old_ring);
+	return 0;
```

```
+
+failed:
+	free_percpu(new_readers);
+	kfree(new_queue);
+	kfree(new_ring);
+	return err;
+}
+
+static void swap_queue_del(struct swap_info_struct *si)
+{
+	gfp_t gfp;
+	unsigned int ring_idx, dev_idx;
+	struct swap_queue_reader __percpu *new_readers = NULL;
+	struct swap_prio_ring *ring, *new_ring = NULL, *old_ring = NULL;
+	struct swap_prio_ring **new_queue = NULL, **old_queue = NULL;
+
+	lockdep_assert_held_write(&swapon_rwsem);
+	if (!swap_queue_find(si, &ring_idx, &dev_idx)) {
+		WARN_ON(1);
+		return;
+	}
+
+	/*
+	 * To shrink memory usage, pre-allocate new smaller data before
+	 * locking. Failure is fine, swapoff will release them anyway.
+	 */
+	gfp = GFP_NOIO | __GFP_HIGH;
+	ring = swap_queue[ring_idx];
+	if (ring->size > 1)
+		new_ring = kmalloc(struct_size(ring, dev, ring->size - 1), gfp);
+	if (ring->size == 1 && swap_queue_len > 1) {
+		new_readers = swap_queue_prealloc_readers(swap_queue_len - 1,
+							  gfp);
+		new_queue = kmalloc(sizeof(*swap_queue) *
+				    (swap_queue_len - 1), gfp);
+	}
+
+	spin_lock(&swap_queue_update_lock);
+	if (ring->size > 1) {
+		/* Shift trailing devices left to fill the gap. */
+		while (++dev_idx < ring->size)
+			ring->dev[dev_idx - 1] =
+				ring->dev[dev_idx];
+		ring->size--;
+		if (new_ring) {
+			memcpy(new_ring, ring,
+			       struct_size(ring, dev, ring->size));
+			old_ring = ring;
+			swap_queue[ring_idx] = new_ring;
+		}
+	} else {
+		/* Last device in this ring: remove the ring. */
+		old_ring = ring;
+		swap_queue_len--;
+		while (++ring_idx <= swap_queue_len)
+			swap_queue[ring_idx - 1] =
+				swap_queue[ring_idx];
+		if (new_queue) {
+			memcpy(new_queue, swap_queue,
+			       sizeof(*swap_queue) * swap_queue_len);
+			old_queue = swap_queue;
+			swap_queue = new_queue;
+		} else if (!swap_queue_len) {
+			old_queue = swap_queue;
+			swap_queue = NULL;
+		}
+		if (new_readers || !swap_queue_len)
+			swap_queue_install_readers(new_readers);
+	}
```

```
+       spin_unlock(&swap_queue_update_lock);
+
+       kfree(old_ring);
+       kfree(old_queue);
+       put_swap_device(si);
+}
+
 /*
  * Use the second highest bit of inuse_pages counter as the indicator
- * if one swap device is on the available plist, so the atomic can
+ * if one swap device is unavailable for allocation, so the atomic can
  * still be updated arithmetically while having special data embedded.
  *
  * inuse_pages counter is the only thing indicating if a device should
- * be on avail_lists or not (except swapon / swapoff). By embedding the
- * off-list bit in the atomic counter, updates no longer need any lock
- * to check the list status.
+ * be in the available queue or not (except swapon / swapoff). By
+ * embedding the off-list bit in the atomic counter, updates no longer
+ * need any lock to check the list status.
  *
- * This bit will be set if the device is not on the plist and not
- * usable, will be cleared if the device is on the plist.
+ * This bit will be set if the device is not in the available queue
+ * and not usable, will be cleared if the device is in the queue.
  */
-#define SWAP_USAGE_OFFLIST_BIT (1UL << (BITS_PER_TYPE(atomic_t) - 2))
+#define SWAP_USAGE_OFFLIST_BIT BIT(BITS_PER_LONG - 2)
 #define SWAP_USAGE_COUNTER_MASK (~SWAP_USAGE_OFFLIST_BIT)
 static long swap_usage_in_pages(struct swap_info_struct *si)
 {
@@ -1221,6 +1584,7 @@ static void del_from_avail_list(struct swap_info_struct *si, bool
swapoff)
        unsigned long pages;

        spin_lock(&swap_avail_lock);
+       spin_lock(&swap_queue_update_lock);

        /*
         * Force remove it only for swapoff. Else, take it off-list only if
@@ -1238,9 +1602,10 @@ static void del_from_avail_list(struct swap_info_struct *si, bool
swapoff)
                atomic_long_or(SWAP_USAGE_OFFLIST_BIT, &si->inuse_pages);
        }

+       swap_queue_mask(si);
        plist_del(&si->avail_list, &swap_avail_head);
-
 skip:
+       spin_unlock(&swap_queue_update_lock);
        spin_unlock(&swap_avail_lock);
 }

@@ -1251,12 +1616,12 @@ static void add_to_avail_list(struct swap_info_struct *si)
        unsigned long pages;

        spin_lock(&swap_avail_lock);
+       spin_lock(&swap_queue_update_lock);

        /*
-        * Add the device to the avail list if SWP_WRITEOK is set and
-        * SWAP_USAGE_OFFLIST_BIT is still set. Swapoff clears
-        * SWP_WRITEOK first, so the device won't be re-added after
-        * swapoff starts unless swap_device_enable resurrects it.
+        * Mark the device as avail if SWP_WRITEOK is set. Swapoff clears
+        * SWP_WRITEOK first, so check that first so the device won't be
+        * re-added after swapoff started.
         */
        if (!(si->flags & SWP_WRITEOK))
```

```
                goto skip;
@@ -1267,21 +1632,23 @@ static void add_to_avail_list(struct swap_info_struct *si)
        val = atomic_long_fetch_and_relaxed(~SWAP_USAGE_OFFLIST_BIT, &si->inuse_pages);

        /*
-        * When device is full and device is on the plist, only one updater will
-        * see (inuse_pages == si->pages) and will call del_from_avail_list. If
-        * that updater happen to be here, just skip adding.
+        * When device is full and marked as available, one reader will see
+        * (inuse_pages == si->pages) and should mark it as unavailable and
+        * set SWAP_USAGE_OFFLIST_BIT. If that updater happens to be here, just
+        * skip the rest.
         */
        pages = si->pages;
-       if (val == pages) {
+       if ((val & SWAP_USAGE_COUNTER_MASK) == pages) {
                /* Just like the cmpxchg in del_from_avail_list */
                if (atomic_long_try_cmpxchg(&si->inuse_pages, &pages,
                                            pages | SWAP_USAGE_OFFLIST_BIT))
                        goto skip;
        }

+       swap_queue_unmask(si);
        plist_add(&si->avail_list, &swap_avail_head);
-
 skip:
+       spin_unlock(&swap_queue_update_lock);
        spin_unlock(&swap_avail_lock);
 }

@@ -1298,7 +1665,7 @@ static void swap_device_inuse_add(struct swap_info_struct *si,

        /*
         * If device is full, and SWAP_USAGE_OFFLIST_BIT is not set,
-        * remove it from the plist.
+        * mark it unavailable.
         */
        inuse_pages = atomic_long_add_return_relaxed(nr_entries, &si->inuse_pages);
        if (unlikely(inuse_pages == si->pages)) {
@@ -1342,7 +1709,7 @@ static void swap_device_inuse_sub(struct swap_info_struct *si,
unsigned long off

        /*
         * If device is not full, and SWAP_USAGE_OFFLIST_BIT is set,
-        * add it back to the plist.
+        * add it back to the available queue.
         */
        inuse_pages = atomic_long_sub_return_relaxed(nr_entries, &si->inuse_pages);
        if (unlikely(inuse_pages & SWAP_USAGE_OFFLIST_BIT))
@@ -1365,41 +1732,44 @@ static bool get_swap_device_info(struct swap_info_struct *si)
        return true;
 }

-/* Rotate the device and switch to a new cluster */
-static void swap_alloc_entry(struct folio *folio)
+static int swap_alloc_entry(struct folio *folio)
 {
-       struct swap_info_struct *si, *next;
+       struct swap_queue_cursor cursor = {};
+       long nr_pages = folio_nr_pages(folio);
+       struct swap_info_struct *si;
+       int nr_iter, ret;

-       spin_lock(&swap_avail_lock);
-start_over:
-       plist_for_each_entry_safe(si, next, &swap_avail_head, avail_list) {
-               /* Rotate the device and switch to a new cluster */
-               plist_requeue(&si->avail_list, &swap_avail_head);
-               spin_unlock(&swap_avail_lock);
```

```
-               if (get_swap_device_info(si)) {
-                       cluster_alloc_swap_entry(si, folio);
-                       put_swap_device(si);
-                       if (folio_test_swapcache(folio))
-                               return;
-                       if (folio_test_large(folio))
-                               return;
+       percpu_down_read(&swapon_rwsem);
+       migrate_disable();
+       for (nr_iter = 0;; nr_iter++) {
+               si = swap_queue_get_device(nr_pages, nr_iter, &cursor);
+               if (IS_ERR(si)) {
+                       ret = PTR_ERR(si);
+                       if (ret == -EBUSY)
+                               continue;
+                       break;
+               }
+               cluster_alloc_swap_entry(si, folio);
+
+               if (folio_test_swapcache(folio)) {
+                       ret = 0;
+                       break;
                }

-               spin_lock(&swap_avail_lock);
                /*
-                * if we got here, it's likely that si was almost full before,
-                * multiple callers probably all tried to get a page from the
-                * same si and it filled up before we could get one; or, the si
-                * filled up between us dropping swap_avail_lock.
-                * Since we dropped the swap_avail_lock, the swap_avail_list
-                * may have been modified; so if next is still in the
-                * swap_avail_head list then try it, otherwise start over if we
-                * have not gotten any slots.
+                * For large allocations, try every device at the same priority,
+                * but ask the caller to split instead of falling back to a lower
+                * priority ring.
                 */
-               if (plist_node_empty(&next->avail_list))
-                       goto start_over;
+               if (folio_test_large(folio)) {
+                       cursor.ring_only = true;
+                       continue;
+               }
        }
-       spin_unlock(&swap_avail_lock);
+
+       migrate_enable();
+       percpu_up_read(&swapon_rwsem);
+       return ret;
 }

 /*
@@ -1713,6 +2083,7 @@ int folio_alloc_swap(struct folio *folio)
 {
        unsigned int order = folio_order(folio);
        unsigned int size = 1 << order;
+       int ret;

        VM_BUG_ON_FOLIO(!folio_test_locked(folio), folio);
        VM_BUG_ON_FOLIO(!folio_test_uptodate(folio), folio);
@@ -1736,7 +2107,7 @@ int folio_alloc_swap(struct folio *folio)
        }

 again:
-       swap_alloc_entry(folio);
+       ret = swap_alloc_entry(folio);

        if (!order && unlikely(!folio_test_swapcache(folio))) {
```

```
                if (swap_sync_discard())
@@ -1748,7 +2119,7 @@ int folio_alloc_swap(struct folio *folio)
                swap_cache_del_folio(folio);

        if (unlikely(!folio_test_swapcache(folio)))
-               return -ENOMEM;
+               return ret ? ret : -ENOMEM;

        return 0;
 }
@@ -2914,24 +3285,29 @@ static int setup_swap_extents(struct swap_info_struct *sis,
 }

 /*
- * Mark a fully initialized swap device writable and expose it to the
- * allocator. The caller must have resurrected its percpu ref first.
+ * Mark a fully initialized swap device writable and expose it to the allocator.
+ * The caller must have resurrected its percpu ref before entering this helper.
  */
-static void swap_device_enable(struct swap_info_struct *si)
+static void __swap_device_enable(struct swap_info_struct *si)
 {
-       percpu_down_write(&swapon_rwsem);
-       spin_lock(&swap_avail_lock);
-       si->flags |= SWP_WRITEOK;
-       spin_unlock(&swap_avail_lock);
+       lockdep_assert_held_write(&swapon_rwsem);

+       spin_lock(&swap_queue_update_lock);
+       si->flags |= SWP_WRITEOK;
+       spin_unlock(&swap_queue_update_lock);
        atomic_long_add(si->pages, &nr_swap_pages);
        total_swap_pages += si->pages;
        plist_add(&si->list, &swap_active_head);
-       percpu_up_write(&swapon_rwsem);
-
        add_to_avail_list(si);
 }

+static void swap_device_enable(struct swap_info_struct *si)
+{
+       percpu_down_write(&swapon_rwsem);
+       __swap_device_enable(si);
+       percpu_up_write(&swapon_rwsem);
+}
+
 static int swap_device_disable(struct address_space *mapping,
                               struct swap_info_struct **swap_info)
 {
@@ -2965,10 +3341,9 @@ static int swap_device_disable(struct address_space *mapping,
        }
        vm_unacct_memory(si->pages);

-       spin_lock(&swap_avail_lock);
+       spin_lock(&swap_queue_update_lock);
        si->flags &= ~SWP_WRITEOK;
-       spin_unlock(&swap_avail_lock);
-
+       spin_unlock(&swap_queue_update_lock);
        plist_del(&si->list, &swap_active_head);
        total_swap_pages -= si->pages;
        atomic_long_sub(si->pages, &nr_swap_pages);
@@ -3053,6 +3428,11 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
                return err;
        }

+       percpu_down_write(&swapon_rwsem);
+       swap_queue_del(p);
+       percpu_ref_kill(&p->users);
```

```
+       percpu_up_write(&swapon_rwsem);
+
        /*
         * Wait for swap operations protected by get/put_swap_device()
         * to complete.  Because of synchronize_rcu() here, all swap
@@ -3061,7 +3441,6 @@ SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
         * prevent folio_test_swapcache() and the following swap cache
         * operations from racing with swapoff.
         */
-       percpu_ref_kill(&p->users);
        synchronize_rcu();
        wait_for_completion(&p->comp);

@@ -3707,11 +4086,28 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
        si->prio = prio;
        si->list.prio = -si->prio;
        si->avail_list.prio = -si->prio;
-       si->swap_file = swap_file;

-       /* Sets SWP_WRITEOK, resurrect the percpu ref, expose the swap device */
+       /*
+        * Publish swap_file before making the percpu ref live, then add the device
+        * to the queue and make it writable under the same write-side lock. This
+        * keeps lockless ref users and /proc/swaps from observing partial state.
+        */
+       percpu_down_write(&swapon_rwsem);
+       si->swap_file = swap_file;
        percpu_ref_resurrect(&si->users);
-       swap_device_enable(si);
+       error = swap_queue_add(si);
+       if (error) {
+               si->swap_file = NULL;
+               percpu_ref_kill(&si->users);
+       } else {
+               __swap_device_enable(si);
+       }
+       percpu_up_write(&swapon_rwsem);
+       if (error) {
+               wait_for_completion(&si->comp);
+               inode->i_flags &= ~S_SWAPFILE;
+               goto free_swap_zswap;
+       }

        pr_info("Adding %uk swap on %s.  Priority:%d extents:%d across:%lluk
%s%s%s%s\n",
                K(si->pages), name->name, si->prio, nr_extents,
--
2.55.0



==== Attachment 10 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0010-mm-swap-remove-available-list.patch) ====
From f205c502011b787773fd2732ea002eda5b24cc3e Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:48 +0800
Subject: [RFC PATCH v2 10/13] mm/swap: remove available list

Pure cleanup, no functional change.

After the priority queue replaced the allocator's device selection,
swap_avail_head and swap_avail_lock are no longer used. Remove them.

The throttle path now walks swap_active_head under swapon_rwsem.
Check SWP_WRITEOK and the existing off-list state. This preserves the old
priority ordering and does not select a full swap device.

Signed-off-by: Kairui Song <kasong@tencent.com>
```

```
Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 include/linux/swap.h |  1 -
 mm/swapfile.c        | 33 ++++++--------------------------
 2 files changed, 6 insertions(+), 28 deletions(-)

diff --git a/include/linux/swap.h b/include/linux/swap.h
index 94173e59dc0b..bf6e1a491ae1 100644
--- a/include/linux/swap.h
+++ b/include/linux/swap.h
@@ -293,7 +293,6 @@ struct swap_info_struct {
        struct work_struct discard_work; /* discard worker */
        struct work_struct reclaim_work; /* reclaim worker */
        struct list_head discard_clusters; /* discard clusters list */
-       struct plist_node avail_list;   /* entry in swap_avail_head */
        const struct swap_ops *ops;
 };

diff --git a/mm/swapfile.c b/mm/swapfile.c
index 2c6765506970..5b32a33341d5 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -96,21 +96,6 @@ static const char Bad_offset[] = "Bad swap offset entry ";
  */
 static PLIST_HEAD(swap_active_head);

-/*
- * all available (active, not full) swap_info_structs
- * protected with swap_avail_lock, ordered by priority.
- * This is used by folio_alloc_swap() instead of swap_active_head
- * because swap_active_head includes all swap_info_structs,
- * but folio_alloc_swap() doesn't need to look at full ones.
- * This uses its own lock instead of swapon_rwsem because when a
- * swap_info_struct changes between not-full/full, it needs to
- * add/remove itself to/from this list, but the swap_info_struct->lock
- * is held and the locking order requires swapon_rwsem to be taken
- * before any swap_info_struct->lock.
- */
-static PLIST_HEAD(swap_avail_head);
-static DEFINE_SPINLOCK(swap_avail_lock);
-
 static inline struct swap_info_struct *__swap_iter(int *i, unsigned long flag)
 {
        lockdep_assert_held(&swapon_rwsem);
@@ -1583,7 +1568,6 @@ static void del_from_avail_list(struct swap_info_struct *si, bool
swapoff)
 {
        unsigned long pages;

-       spin_lock(&swap_avail_lock);
        spin_lock(&swap_queue_update_lock);

        /*
@@ -1603,10 +1587,8 @@ static void del_from_avail_list(struct swap_info_struct *si, bool
swapoff)
        }

        swap_queue_mask(si);
-       plist_del(&si->avail_list, &swap_avail_head);
 skip:
        spin_unlock(&swap_queue_update_lock);
-       spin_unlock(&swap_avail_lock);
 }

 /* SWAP_USAGE_OFFLIST_BIT can only be cleared by this helper. */
@@ -1615,7 +1597,6 @@ static void add_to_avail_list(struct swap_info_struct *si)
        long val;
        unsigned long pages;
```

```
-       spin_lock(&swap_avail_lock);
        spin_lock(&swap_queue_update_lock);

        /*
@@ -1646,10 +1627,8 @@ static void add_to_avail_list(struct swap_info_struct *si)
        }

        swap_queue_unmask(si);
-       plist_add(&si->avail_list, &swap_avail_head);
 skip:
        spin_unlock(&swap_queue_update_lock);
-       spin_unlock(&swap_avail_lock);
 }

 /*
@@ -3677,7 +3656,6 @@ static struct swap_info_struct *alloc_swap_info(void)
        }
        p->swap_extent_root = RB_ROOT;
        plist_node_init(&p->list, 0);
-       plist_node_init(&p->avail_list, 0);
        p->flags = SWP_USED;
        percpu_up_write(&swapon_rwsem);
        if (defer) {
@@ -4085,7 +4063,6 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
         */
        si->prio = prio;
        si->list.prio = -si->prio;
-       si->avail_list.prio = -si->prio;

        /*
         * Publish swap_file before making the percpu ref live, then add the device
@@ -4229,14 +4206,16 @@ void __folio_throttle_swaprate(struct folio *folio, gfp_t gfp)
        if (current->throttle_disk)
                return;

-       spin_lock(&swap_avail_lock);
-       plist_for_each_entry(si, &swap_avail_head, avail_list) {
-               if (si->bdev) {
+       percpu_down_read(&swapon_rwsem);
+       plist_for_each_entry(si, &swap_active_head, list) {
+               if ((si->flags & SWP_WRITEOK) &&
+                   !(atomic_long_read(&si->inuse_pages) &
+                     SWAP_USAGE_OFFLIST_BIT) && si->bdev) {
                        blkcg_schedule_throttle(si->bdev->bd_disk, true);
                        break;
                }
        }
-       spin_unlock(&swap_avail_lock);
+       percpu_up_read(&swapon_rwsem);
 }
 #endif

--
2.55.0



==== Attachment 11 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0011-mm-swap-bound-synchronous-discard-during-allocation.patch) ====
From 05b43f03ab25bb8f57d78453582e62339f93ac61 Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:49 +0800
Subject: [RFC PATCH v2 11/13] mm/swap: bound synchronous discard during
 allocation

The old global per-CPU cluster cache made it difficult to select a
specific device before handling its pending discards.  Now that cluster
state is per-device, let an allocator issue discard for that device when
```

```
its free-cluster list is drained instead of waiting for a global fallback
scan.

Do not drain the entire discard list from one allocation.  The list is
also updated by freeing paths, so an unlocked non-empty check races with
those writers, and a full synchronous drain has no fixed latency bound
when other CPUs continue to enqueue clusters.

Factor out swap_discard_one_cluster().  Isolate one cluster under
si->lock, retain CLUSTER_FLAG_DISCARD while I/O is in flight, and return
it to the free list after the discard completes.  The background worker
continues looping until the list is empty, while an allocator handles at
most one cluster before retrying allocation.

This keeps the proactive per-device policy while giving each allocation
a fixed synchronous discard budget.

Link: https://lore.kernel.org/all/CAMgjq7CsYhEjvtN85XGkrONYAJxve7gG593TFeOGV-oax++kWA@mail.gmail.com/
Signed-off-by: Kairui Song <kasong@tencent.com>
Co-developed-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 mm/swapfile.c | 149 ++++++++++++++++++++++---------------------------
 1 file changed, 64 insertions(+), 85 deletions(-)

diff --git a/mm/swapfile.c b/mm/swapfile.c
index 5b32a33341d5..b1c322239b01 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -528,6 +528,26 @@ static long swap_usage_in_pages(struct swap_info_struct *si)
        return atomic_long_read(&si->inuse_pages) & SWAP_USAGE_COUNTER_MASK;
 }

+/*
+ * Serialize the allocation on single CPU or globally to avoid
+ * fragmentation and make the workflow easier to follow.
+ */
+static void swap_alloc_lock_device(struct swap_info_struct *si)
+{
+       if (si->flags & SWP_SOLIDSTATE)
+               local_lock(&si->percpu_cluster->lock);
+       else
+               spin_lock(&si->global_cluster->lock);
+}
+
+static void swap_alloc_unlock_device(struct swap_info_struct *si)
+{
+       if (si->flags & SWP_SOLIDSTATE)
+               local_unlock(&si->percpu_cluster->lock);
+       else
+               spin_unlock(&si->global_cluster->lock);
+}
+
 /* Reclaim the swap entry anyway if possible */
 #define TTRS_ANYWAY            0x1
 /*
@@ -914,10 +934,7 @@ swap_cluster_populate(struct swap_info_struct *si,
         * the potential recursive allocation is limited.
         */
        spin_unlock(&ci->lock);
-       if (si->flags & SWP_SOLIDSTATE)
-               local_unlock(&si->percpu_cluster->lock);
-       else
-               spin_unlock(&si->global_cluster->lock);
+       swap_alloc_unlock_device(si);

        ret = swap_cluster_alloc_table(ci, __GFP_HIGH | __GFP_NOMEMALLOC |
                                           GFP_KERNEL);
```

```
@@ -930,10 +947,7 @@ swap_cluster_populate(struct swap_info_struct *si,
         * could happen with ignoring the percpu cluster is fragmentation,
         * which is acceptable since this fallback and race is rare.
         */
-       if (si->flags & SWP_SOLIDSTATE)
-               local_lock(&si->percpu_cluster->lock);
-       else
-               spin_lock(&si->global_cluster->lock);
+       swap_alloc_lock_device(si);
        spin_lock(&ci->lock);

        if (ret) {
@@ -1023,44 +1037,40 @@ static struct swap_cluster_info *isolate_lock_cluster(
 }

 /*
- * Doing discard actually. After a cluster discard is finished, the cluster
- * will be added to free cluster list. Discard cluster is a bit special as
- * they don't participate in allocation or reclaim, so clusters marked as
- * CLUSTER_FLAG_DISCARD must remain off-list or on discard list.
+ * Discard one cluster. After the discard is finished, the cluster will be
+ * added to the free cluster list. Discard clusters are special because they
+ * don't participate in allocation or reclaim, so CLUSTER_FLAG_DISCARD must
+ * remain set while a cluster is either queued or being discarded.
  */
-static bool swap_do_scheduled_discard(struct swap_info_struct *si)
+static bool swap_discard_one_cluster(struct swap_info_struct *si)
 {
        struct swap_cluster_info *ci;
-       bool ret = false;
        unsigned int idx;

        spin_lock(&si->lock);
-       while (!list_empty(&si->discard_clusters)) {
-               ci = list_first_entry(&si->discard_clusters, struct swap_cluster_info,
list);
-               /*
-                * Delete the cluster from list to prepare for discard, but keep
-                * the CLUSTER_FLAG_DISCARD flag, there could be percpu_cluster
-                * pointing to it, or ran into by relocate_cluster.
-                */
-               list_del(&ci->list);
-               idx = cluster_index(si, ci);
+       if (list_empty(&si->discard_clusters)) {
                spin_unlock(&si->lock);
-               discard_swap_cluster(si, idx * SWAPFILE_CLUSTER,
-                                    SWAPFILE_CLUSTER);
-
-               spin_lock(&ci->lock);
-               /*
-                * Discard is done, clear its flags as it's off-list, then
-                * return the cluster to allocation list.
-                */
-               ci->flags = CLUSTER_FLAG_NONE;
-               __free_cluster(si, ci);
-               spin_unlock(&ci->lock);
-               ret = true;
-               spin_lock(&si->lock);
+               return false;
        }
+
+       ci = list_first_entry(&si->discard_clusters,
+                             struct swap_cluster_info, list);
+       /*
+        * Delete the cluster from the list, but keep CLUSTER_FLAG_DISCARD
+        * set while the discard is in flight. A percpu cluster may still
+        * point to it, or relocate_cluster() may encounter it.
+        */
+       list_del(&ci->list);
```

```
+       idx = cluster_index(si, ci);
        spin_unlock(&si->lock);
-       return ret;
+
+       discard_swap_cluster(si, idx * SWAPFILE_CLUSTER, SWAPFILE_CLUSTER);
+
+       spin_lock(&ci->lock);
+       ci->flags = CLUSTER_FLAG_NONE;
+       __free_cluster(si, ci);
+       spin_unlock(&ci->lock);
+       return true;
 }

 static void swap_discard_work(struct work_struct *work)
@@ -1069,7 +1079,8 @@ static void swap_discard_work(struct work_struct *work)

        si = container_of(work, struct swap_info_struct, discard_work);

-       swap_do_scheduled_discard(si);
+       while (swap_discard_one_cluster(si))
+               cond_resched();
 }

 static void swap_users_ref_free(struct percpu_ref *ref)
@@ -1467,6 +1478,7 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
        struct swap_cluster_info *ci;
        unsigned int order = likely(folio) ? folio_order(folio) : 0;
        unsigned int offset = SWAP_ENTRY_INVALID, found = SWAP_ENTRY_INVALID;
+       bool discarded = false;

        /*
         * Swapfile is not block device so unable
@@ -1475,15 +1487,12 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
        if (order && !(si->flags & SWP_BLKDEV))
                return 0;

-       if (si->flags & SWP_SOLIDSTATE) {
-               /* Fast path using per CPU cluster */
-               local_lock(&si->percpu_cluster->lock);
+restart:
+       swap_alloc_lock_device(si);
+       if (si->flags & SWP_SOLIDSTATE)
                offset = __this_cpu_read(si->percpu_cluster->next[order]);
-       } else {
-               /* Serialize HDD SWAP allocation for each device. */
-               spin_lock(&si->global_cluster->lock);
+       else
                offset = si->global_cluster->next[order];
-       }

        if (offset != SWAP_ENTRY_INVALID) {
                ci = swap_cluster_lock(si, offset);
@@ -1507,6 +1516,15 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
                found = alloc_swap_scan_list(si, &si->free_clusters, folio, false);
                if (found)
                        goto done;
+
+               if (!discarded) {
+                       swap_alloc_unlock_device(si);
+                       if (swap_discard_one_cluster(si)) {
+                               discarded = true;
+                               goto restart;
+                       }
+                       swap_alloc_lock_device(si);
+               }
        }
```

```
        if (order < PMD_ORDER) {
@@ -1555,10 +1573,7 @@ static unsigned long cluster_alloc_swap_entry(struct
swap_info_struct *si,
                        goto done;
        }
 done:
-       if (si->flags & SWP_SOLIDSTATE)
-               local_unlock(&si->percpu_cluster->lock);
-       else
-               spin_unlock(&si->global_cluster->lock);
+       swap_alloc_unlock_device(si);

        return found;
 }
@@ -1751,36 +1766,6 @@ static int swap_alloc_entry(struct folio *folio)
        return ret;
 }

-/*
- * Discard pending clusters in a synchronized way when under high pressure.
- * Return: true if any cluster is discarded.
- */
-static bool swap_sync_discard(void)
-{
-       bool ret = false;
-       struct swap_info_struct *si, *next;
-
-       percpu_down_read(&swapon_rwsem);
-start_over:
-       plist_for_each_entry_safe(si, next, &swap_active_head, list) {
-               percpu_up_read(&swapon_rwsem);
-               if (get_swap_device_info(si)) {
-                       if (si->flags & SWP_PAGE_DISCARD)
-                               ret = swap_do_scheduled_discard(si);
-                       put_swap_device(si);
-               }
-               if (ret)
-                       return true;
-
-               percpu_down_read(&swapon_rwsem);
-               if (plist_node_empty(&next->list))
-                       goto start_over;
-       }
-       percpu_up_read(&swapon_rwsem);
-
-       return false;
-}
-
 static int swap_extend_table_alloc(struct swap_info_struct *si,
                                   struct swap_cluster_info *ci,
                                   unsigned int ci_off, gfp_t gfp)
@@ -2085,14 +2070,8 @@ int folio_alloc_swap(struct folio *folio)
                }
        }

-again:
        ret = swap_alloc_entry(folio);

-       if (!order && unlikely(!folio_test_swapcache(folio))) {
-               if (swap_sync_discard())
-                       goto again;
-       }
-
        /* Need to call this even if allocation failed, for MEMCG_SWAP_FAIL. */
        if (unlikely(mem_cgroup_try_charge_swap(folio)))
                swap_cache_del_folio(folio);
--
2.55.0
```

```
==== Attachment 12 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0012-mm-swap-drop-swap-active-plist.patch) ====
From 510984375538cd4d2e00bba161ca29490696289d Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:50 +0800
Subject: [RFC PATCH v2 12/13] mm/swap: drop swap active plist

The swap_active_head plist tracked all writable swap devices, ordered
by priority.  Now that every consumer that needed priority ordering
has moved to the priority queue, the remaining users only need to
find any writable device.

Remove swap_active_head, the associated plist_node from
swap_info_struct, and migrate the remaining plist operations.

Signed-off-by: Kairui Song <kasong@tencent.com>
Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 include/linux/swap.h |  1 -
 mm/swapfile.c        | 27 +++++----------------------
 2 files changed, 5 insertions(+), 23 deletions(-)

diff --git a/include/linux/swap.h b/include/linux/swap.h
index bf6e1a491ae1..d5bde7c9d14d 100644
--- a/include/linux/swap.h
+++ b/include/linux/swap.h
@@ -267,7 +267,6 @@ struct swap_info_struct {
        struct percpu_ref users;        /* indicate and keep swap device valid. */
        unsigned long   flags;          /* SWP_USED etc: see above */
        signed short    prio;           /* swap priority of this type */
-       struct plist_node list;         /* entry in swap_active_head */
        signed char     type;           /* strange name for an index */
        unsigned int    max;            /* size of this swap device */
        struct swap_cluster_info *cluster_info; /* cluster info. Only for SSD */
diff --git a/mm/swapfile.c b/mm/swapfile.c
index b1c322239b01..c0a0da49e4e3 100644
--- a/mm/swapfile.c
+++ b/mm/swapfile.c
@@ -41,7 +41,6 @@
 #include <linux/completion.h>
 #include <linux/suspend.h>
 #include <linux/zswap.h>
-#include <linux/plist.h>

 #include <asm/tlbflush.h>
 #include <linux/leafops.h>
@@ -90,12 +89,6 @@ bool swap_migration_ad_supported;
 static const char Bad_file[] = "Bad swap file entry ";
 static const char Bad_offset[] = "Bad swap offset entry ";

-/*
- * all active swap_info_structs
- * protected with swapon_rwsem, and ordered by priority.
- */
-static PLIST_HEAD(swap_active_head);
-
 static inline struct swap_info_struct *__swap_iter(int *i, unsigned long flag)
 {
        lockdep_assert_held(&swapon_rwsem);
@@ -3255,7 +3248,6 @@ static void __swap_device_enable(struct swap_info_struct *si)
        spin_unlock(&swap_queue_update_lock);
        atomic_long_add(si->pages, &nr_swap_pages);
        total_swap_pages += si->pages;
-       plist_add(&si->list, &swap_active_head);
        add_to_avail_list(si);
 }
```

```
@@ -3275,9 +3267,8 @@ static int swap_device_disable(struct address_space *mapping,
        int err = -EINVAL;

        percpu_down_write(&swapon_rwsem);
-       plist_for_each_entry(si, &swap_active_head, list) {
-               if ((si->flags & SWP_WRITEOK) &&
-                   si->swap_file->f_mapping == mapping) {
+       for_each_avail_swap(si) {
+               if (si->swap_file->f_mapping == mapping) {
                        err = 0;
                        break;
                }
@@ -3302,7 +3293,6 @@ static int swap_device_disable(struct address_space *mapping,
        spin_lock(&swap_queue_update_lock);
        si->flags &= ~SWP_WRITEOK;
        spin_unlock(&swap_queue_update_lock);
-       plist_del(&si->list, &swap_active_head);
        total_swap_pages -= si->pages;
        atomic_long_sub(si->pages, &nr_swap_pages);

@@ -3634,7 +3624,6 @@ static struct swap_info_struct *alloc_swap_info(void)
                 */
        }
        p->swap_extent_root = RB_ROOT;
-       plist_node_init(&p->list, 0);
        p->flags = SWP_USED;
        percpu_up_write(&swapon_rwsem);
        if (defer) {
@@ -4036,12 +4025,7 @@ SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
        if (swap_flags & SWAP_FLAG_PREFER)
                prio = swap_flags & SWAP_FLAG_PRIO_MASK;

-       /*
-        * The plist prio is negated because plist ordering is
-        * low-to-high, while swap ordering is high-to-low
-        */
        si->prio = prio;
-       si->list.prio = -si->prio;

        /*
         * Publish swap_file before making the percpu ref live, then add the device
@@ -4162,7 +4146,7 @@ int swap_dup_entry_direct(swp_entry_t entry)
 #if defined(CONFIG_MEMCG) && defined(CONFIG_BLK_CGROUP)
 static bool __has_usable_swap(void)
 {
-       return !plist_head_empty(&swap_active_head);
+       return READ_ONCE(total_swap_pages) > 0;
 }

 void __folio_throttle_swaprate(struct folio *folio, gfp_t gfp)
@@ -4186,9 +4170,8 @@ void __folio_throttle_swaprate(struct folio *folio, gfp_t gfp)
                return;

        percpu_down_read(&swapon_rwsem);
-       plist_for_each_entry(si, &swap_active_head, list) {
-               if ((si->flags & SWP_WRITEOK) &&
-                   !(atomic_long_read(&si->inuse_pages) &
+       for_each_avail_swap(si) {
+               if (!(atomic_long_read(&si->inuse_pages) &
                      SWAP_USAGE_OFFLIST_BIT) && si->bdev) {
                        blkcg_schedule_throttle(si->bdev->bd_disk, true);
                        break;
--
2.55.0
```

```
==== Attachment 13 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/01-
kernel-v2-patches/0013-lib-plist.c-remove-requeue-function.patch) ====
From d5c8964cf19fcffd7eea75dc9f28c928b46503bf Mon Sep 17 00:00:00 2001
From: Kairui Song <kasong@tencent.com>
Date: Tue, 14 Jul 2026 01:25:51 +0800
Subject: [RFC PATCH v2 13/13] lib/plist.c: remove requeue function

Now the last user of plist requeue is gone, this function can be
removed.

Signed-off-by: Kairui Song <kasong@tencent.com>
---
 include/linux/plist.h |  2 --
 lib/plist.c           | 64 -------------------------------------------
 2 files changed, 66 deletions(-)

diff --git a/include/linux/plist.h b/include/linux/plist.h
index 16cf4355b5c1..73d359c1a053 100644
--- a/include/linux/plist.h
+++ b/include/linux/plist.h
@@ -132,8 +132,6 @@ static inline void plist_node_init(struct plist_node *node, int
prio)
 extern void plist_add(struct plist_node *node, struct plist_head *head);
 extern void plist_del(struct plist_node *node, struct plist_head *head);

-extern void plist_requeue(struct plist_node *node, struct plist_head *head);
-
 /**
  * plist_for_each - iterate over the plist
  * @pos:       the type * to use as a loop counter
diff --git a/lib/plist.c b/lib/plist.c
index a5bef38add43..b05ae7ffea87 100644
--- a/lib/plist.c
+++ b/lib/plist.c
@@ -142,58 +142,6 @@ void plist_del(struct plist_node *node, struct plist_head *head)
        plist_check_head(head);
 }

-/**
- * plist_requeue - Requeue @node at end of same-prio entries.
- *
- * This is essentially an optimized plist_del() followed by
- * plist_add().  It moves an entry already in the plist to
- * after any other same-priority entries.
- *
- * @node:      &struct plist_node pointer - entry to be moved
- * @head:      &struct plist_head pointer - list head
- */
-void plist_requeue(struct plist_node *node, struct plist_head *head)
-{
-       struct plist_node *iter;
-       struct list_head *node_next = &head->node_list;
-
-       plist_check_head(head);
-       BUG_ON(plist_head_empty(head));
-       BUG_ON(plist_node_empty(node));
-
-       if (node == plist_last(head))
-               return;
-
-       iter = plist_next(node);
-
-       if (node->prio != iter->prio)
-               return;
-
-       plist_del(node, head);
-
-       /*
-        * After plist_del(), iter is the replacement of the node.  If the node
```

```
-        * was on prio_list, take shortcut to find node_next instead of looping.
-        */
-       if (!list_empty(&iter->prio_list)) {
-               iter = list_entry(iter->prio_list.next, struct plist_node,
-                                 prio_list);
-               node_next = &iter->node_list;
-               goto queue;
-       }
-
-       plist_for_each_continue(iter, head) {
-               if (node->prio != iter->prio) {
-                       node_next = &iter->node_list;
-                       break;
-               }
-       }
-queue:
-       list_add_tail(&node->node_list, node_next);
-
-       plist_check_head(head);
-}
-
 #ifdef CONFIG_DEBUG_PLIST
 #include <linux/sched.h>
 #include <linux/sched/clock.h>
@@ -231,14 +179,6 @@ static void __init plist_test_check(int nr_expect)
        BUG_ON(prio_pos->prio_list.next != &first->prio_list);
 }

-static void __init plist_test_requeue(struct plist_node *node)
-{
-       plist_requeue(node, &test_head);
-
-       if (node != plist_last(&test_head))
-               BUG_ON(node->prio == plist_next(node)->prio);
-}
-
 static int  __init plist_test(void)
 {
        int nr_expect = 0, i, loop;
@@ -262,10 +202,6 @@ static int  __init plist_test(void)
                        nr_expect--;
                }
                plist_test_check(nr_expect);
-               if (!plist_node_empty(test_node + i)) {
-                       plist_test_requeue(test_node + i);
-                       plist_test_check(nr_expect);
-               }
        }

        for (i = 0; i < ARRAY_SIZE(test_node); i++) {
--
2.55.0



==== Attachment 14 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-
review-and-testing/14-v1-to-v2-range-diff.txt) ====
 1:  c7c284e4f9bd =  1:  22ec384c63f5 mm/swap: remove unused parameter for reading swap
header
 2:  f81cc30a1218 =  2:  a940e5611be0 mm/swap: slightly cleanup the code for hibernation
error handling
 3:  cd923abbdf6e !  3:  f7ad0bf80490 mm/swap: cleanup and document swap device
availability flag usage
    @@ Metadata
      ## Commit message ##
         mm/swap: cleanup and document swap device availability flag usage

    -    Rework swap device flag and metedata handling and swapon/swapoff to
    +    Rework swap device flag and metadata handling and swapon/swapoff to
```

```
      be cleaner and better documented, in preparation for locking cleanup.

-     Consolidate existing routines and introduce swap_device_enable and
-     swap_device_disable as the clean boundary of exposing or isolating
-     a swap device. swap_device_enable sets the proper flags and exposes
-     the device as an allocation candidate, while swap_device_disable
-     clears related flags and ensures no more allocations will happen.
+     Consolidate existing routines and introduce swap_device_enable() and
+     swap_device_disable() as the clean boundary of exposing or isolating a
+     swap device. swap_device_enable() sets the proper flags and exposes the
+     device as an allocation candidate, while swap_device_disable() clears
+     related flags and ensures no more allocations will happen.

-     And add comment blocks documenting the lifetime and locking rules for
-     swap device flags and their locking conventions.
+     Keep the mapping lookup and disable transition in the same swap_lock
+     critical section. Otherwise, a concurrent swapoff can release the
+     selected slot and swapon can reuse it for a different device before the
+     first syscall disables the swap_info_struct it found. Drain the cluster
+     allocators in the same helper after dropping swap_lock, so the identity
+     transition remains atomic without holding the lock over all cluster
+     locks.

-     No feature change, only code rearrangement and documentation.
+     Add comment blocks documenting the lifetime and locking rules for swap
+     device flags and their locking conventions.
+
+     Apart from closing that lifecycle race, the remaining changes are code
+     rearrangement and documentation.

      Signed-off-by: Kairui Song <kasong@tencent.com>
+     Co-developed-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
+     Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>

   ## include/linux/swap.h ##
  @@ include/linux/swap.h: struct swap_extent {
 @@ mm/swapfile.c: static int setup_swap_extents(struct swap_info_struct *sis,
   }

  -static void _enable_swap_info(struct swap_info_struct *si)
 --{
 ++/*
 ++ * Mark a fully initialized swap device writable and expose it to the
 ++ * allocator. The caller must have resurrected its percpu ref first.
 ++ */
 ++static void swap_device_enable(struct swap_info_struct *si)
 + {
  -  atomic_long_add(si->pages, &nr_swap_pages);
  -  total_swap_pages += si->pages;
 --
 ++  spin_lock(&swap_lock);
 +
  -  assert_spin_locked(&swap_lock);
 --
 --  plist_add(&si->list, &swap_active_head);
 --
 ++  spin_lock(&swap_avail_lock);
 ++  si->flags |= SWP_WRITEOK;
 ++  spin_unlock(&swap_avail_lock);
 +
 ++  atomic_long_add(si->pages, &nr_swap_pages);
 ++  total_swap_pages += si->pages;
 +   plist_add(&si->list, &swap_active_head);
 ++  spin_unlock(&swap_lock);
 +
  -  /* Add back to available list */
  -  add_to_avail_list(si, true);
 --}
 --
```

```
- /*
++  add_to_avail_list(si);
+ }
+
+-/*
 - * Called after the swap device is ready, resurrect its percpu ref, it's now
 - * safe to reference it. Add it to the list to expose it to the allocator.
-+ * Called after the swap device is ready to be used. Marking it writable and
-+ * exposing it to the allocator. Resurrect its percpu ref if it was dead before.
-  */
+- */
 -static void enable_swap_info(struct swap_info_struct *si)
-+static void swap_device_enable(struct swap_info_struct *si)
++static int swap_device_disable(struct address_space *mapping,
++                           struct swap_info_struct **swap_info)
  {
 -  percpu_ref_resurrect(&si->users);
-   spin_lock(&swap_lock);
+-  spin_lock(&swap_lock);
 -  spin_lock(&si->lock);
 -  _enable_swap_info(si);
 -  spin_unlock(&si->lock);
 -  spin_unlock(&swap_lock);
 -}
++  struct swap_info_struct *si;
++  struct swap_cluster_info *ci;
++  unsigned long offset, end;
++  int err = -EINVAL;

 -static void reinsert_swap_info(struct swap_info_struct *si)
 -{
--  spin_lock(&swap_lock);
+   spin_lock(&swap_lock);
 -  spin_lock(&si->lock);
 -  _enable_swap_info(si);
 -  spin_unlock(&si->lock);
-+  spin_lock(&swap_avail_lock);
-+  si->flags |= SWP_WRITEOK;
-+  spin_unlock(&swap_avail_lock);
-+
-+  atomic_long_add(si->pages, &nr_swap_pages);
-+  total_swap_pages += si->pages;
-+  plist_add(&si->list, &swap_active_head);
-   spin_unlock(&swap_lock);
-+
-+  add_to_avail_list(si);
- }
+-  spin_unlock(&swap_lock);
+-}
++  plist_for_each_entry(si, &swap_active_head, list) {
++          if ((si->flags & SWP_WRITEOK) &&
++              si->swap_file->f_mapping == mapping) {
++                  err = 0;
++                  break;
++          }
++  }
++  if (err)
++          goto unlock;

 -/*
 - * Called after clearing SWP_WRITEOK, ensures cluster_alloc_range
 - * see the updated flags, so there will be no more allocations.
 - */
 -static void wait_for_allocation(struct swap_info_struct *si)
-+static int swap_device_disable(struct swap_info_struct *si)
- {
-   unsigned long offset;
-   unsigned long end = ALIGN(si->max, SWAPFILE_CLUSTER);
-   struct swap_cluster_info *ci;
```

```
-
--  BUG_ON(si->flags & SWP_WRITEOK);
+-{
+-  unsigned long offset;
+-  unsigned long end = ALIGN(si->max, SWAPFILE_CLUSTER);
+-  struct swap_cluster_info *ci;
 +  /*
-+   * If SWP_WRITEOK is not set: another process already disabling it.
-+   * If SWP_HIBERNATION is set: the device is pinned for hibernation.
++   * Refuse swapoff while the device is pinned for hibernation.
 +   */
-+  spin_lock(&swap_lock);
-+  if (!(si->flags & SWP_WRITEOK) ||
-+      si->flags & SWP_HIBERNATION) {
-+          spin_unlock(&swap_lock);
-+          return -EBUSY;
++  if (si->flags & SWP_HIBERNATION) {
++          err = -EBUSY;
++          goto unlock;
 +  }

+-  BUG_ON(si->flags & SWP_WRITEOK);
 +  if (security_vm_enough_memory_mm(current->mm, si->pages)) {
-+          spin_unlock(&swap_lock);
-+          return -ENOMEM;
++          err = -ENOMEM;
++          goto unlock;
 +  }
 +  vm_unacct_memory(si->pages);
-+
+
 +  spin_lock(&swap_avail_lock);
 +  si->flags &= ~SWP_WRITEOK;
 +  spin_unlock(&swap_avail_lock);
@@ mm/swapfile.c: static int setup_swap_extents(struct swap_info_struct *sis,
 +  plist_del(&si->list, &swap_active_head);
 +  total_swap_pages -= si->pages;
 +  atomic_long_sub(si->pages, &nr_swap_pages);
++
++  end = ALIGN(si->max, SWAPFILE_CLUSTER);
++unlock:
 +  spin_unlock(&swap_lock);
++  if (err)
++          return err;
 +
 +  del_from_avail_list(si, true);
 +
 +  /*
-+   * Swap allocator doesn't touch si lock, so looping through all
-+   * ci locks ensures __swap_cluster_alloc_entries sees the
-+   * updated flags, and no more allocations will occur.
++   * The swap allocator doesn't take swap_lock. Looping through every
++   * cluster lock after clearing SWP_WRITEOK ensures that allocators see
++   * the updated flag and that no allocation remains in flight.
 +   */
    for (offset = 0; offset < end; offset += SWAPFILE_CLUSTER) {
            ci = swap_cluster_lock(si, offset);
            swap_cluster_unlock(ci);
    }
 +
++  *swap_info = si;
 +  return 0;
  }

@@ mm/swapfile.c: static void flush_percpu_swap_cluster(struct swap_info_struct *si
  {
    struct swap_info_struct *p = NULL;
 @@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
+   struct address_space *mapping;
```

```
      +   struct inode *inode;
      +   unsigned int maxpages;
      +-  int err, found = 0;
      ++  int err;
      +
      +   if (!capable(CAP_SYS_ADMIN))
      +           return -EPERM;
      +@@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
      +           return PTR_ERR(victim);
      +
          mapping = victim->f_mapping;
      -   spin_lock(&swap_lock);
      -   plist_for_each_entry(p, &swap_active_head, list) {
      +-  spin_lock(&swap_lock);
      +-  plist_for_each_entry(p, &swap_active_head, list) {
       -          if (p->flags & SWP_WRITEOK) {
       -                  if (p->swap_file->f_mapping == mapping) {
       -                          found = 1;
       -                          break;
       -                  }
      -+          if (p->flags & SWP_WRITEOK &&
      -+              p->swap_file->f_mapping == mapping) {
      -+                  found = 1;
      -+                  break;
      -           }
      -   }
      +-          }
      +-  }
       -  if (!found) {
       -          err = -EINVAL;
       -          spin_unlock(&swap_lock);
      @@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
       -  atomic_long_sub(p->pages, &nr_swap_pages);
       -  total_swap_pages -= p->pages;
       -  spin_unlock(&p->lock);
      -   spin_unlock(&swap_lock);
      +-  spin_unlock(&swap_lock);
      ++  err = swap_device_disable(mapping, &p);
       +  filp_close(victim, NULL);
      -+
      -+  if (!found)
      -+          return -EINVAL;

       -  wait_for_allocation(p);
      -+  err = swap_device_disable(p);
       +  if (err)
       +          return err;

 4:  9a5a2718aeee =  4:  da61e75e4748 mm/swap: introduce swap device iteration helper
 5:  abffa8fbc848 !  5:  893fb7195cbe mm/swap: change the swapon lock into a percpu
rwsem
    @@ Commit message
         name change.

         Signed-off-by: Kairui Song <kasong@tencent.com>
    +    Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>

      ## include/linux/swap.h ##
     @@ include/linux/swap.h: struct swap_extent {
    @@ mm/swapfile.c: static void swap_device_enable(struct swap_info_struct *si)

         add_to_avail_list(si);
      }
    -@@ mm/swapfile.c: static int swap_device_disable(struct swap_info_struct *si)
    -    * If SWP_WRITEOK is not set: another process already disabling it.
    -    * If SWP_HIBERNATION is set: the device is pinned for hibernation.
    -    */
    +@@ mm/swapfile.c: static int swap_device_disable(struct address_space *mapping,
    +   unsigned long offset, end;
```

```
      +   int err = -EINVAL;
      +
       -  spin_lock(&swap_lock);
       +  percpu_down_write(&swapon_rwsem);
      -   if (!(si->flags & SWP_WRITEOK) ||
      -       si->flags & SWP_HIBERNATION) {
      --          spin_unlock(&swap_lock);
      -+          percpu_up_write(&swapon_rwsem);
      -           return -EBUSY;
      -   }
      +   plist_for_each_entry(si, &swap_active_head, list) {
      +           if ((si->flags & SWP_WRITEOK) &&
      +               si->swap_file->f_mapping == mapping) {
      +@@ mm/swapfile.c: static int swap_device_disable(struct address_space *mapping,

      -   if (security_vm_enough_memory_mm(current->mm, si->pages)) {
      --          spin_unlock(&swap_lock);
      -+          percpu_up_write(&swapon_rwsem);
      -           return -ENOMEM;
      -   }
      -   vm_unacct_memory(si->pages);
      -@@ mm/swapfile.c: static int swap_device_disable(struct swap_info_struct *si)
      -   plist_del(&si->list, &swap_active_head);
      -   total_swap_pages -= si->pages;
      -   atomic_long_sub(si->pages, &nr_swap_pages);
      +   end = ALIGN(si->max, SWAPFILE_CLUSTER);
      + unlock:
       -  spin_unlock(&swap_lock);
       +  percpu_up_write(&swapon_rwsem);
      +   if (err)
      +           return err;

          del_from_avail_list(si, true);

      -@@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
      -           return PTR_ERR(victim);
      -
      -   mapping = victim->f_mapping;
      --  spin_lock(&swap_lock);
      -+  percpu_down_read(&swapon_rwsem);
      -   plist_for_each_entry(p, &swap_active_head, list) {
      -           if (p->flags & SWP_WRITEOK &&
      -               p->swap_file->f_mapping == mapping) {
      -@@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
      -                   break;
      -           }
      -   }
      --  spin_unlock(&swap_lock);
      -+  percpu_up_read(&swapon_rwsem);
      -   filp_close(victim, NULL);
      -
      -   if (!found)
      +   /*
      +-   * The swap allocator doesn't take swap_lock. Looping through every
      ++   * The swap allocator doesn't take swapon_rwsem. Looping through every
      +    * cluster lock after clearing SWP_WRITEOK ensures that allocators see
      +    * the updated flag and that no allocation remains in flight.
      +    */
       @@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
                  atomic_dec(&nr_rotate_swap);

   6:  41eeb785c15b =  6:  3191606c8f9e mm/swap: remove swapon mutex and update proc
  reader
   7:  f2a88b90727d =  7:  74787f6dee14 mm/swap: consolidate swap inuse accounting helpers
   8:  4a7d8bd1b664 =  8:  13e69c595fe6 mm/swap: change back to use each swap device's
  percpu cluster
   9:  d9b536582361 !  9:  a05e05b8d1c1 mm/swap: add priority queue for swap device
  allocation
      @@ Metadata
```

```
     ## Commit message ##
        mm/swap: add priority queue for swap device allocation

    -    The swap allocator uses a plist (swap_avail_head) ordered by
    -    priority to pick the next device.  To spread IO across devices at
    -    the same priority it rotates the current device to the tail via
    -    plist_requeue(), which requires holding swap_avail_lock.  It then
    -    drops the lock for the allocation attempt and reacquires to check
    -    whether the next device is still on the list.  This per-allocation
    -    lock cycling and the single global rotation point mean all CPUs
    -    contend for the same list head.
    +    The swap allocator uses swap_avail_head, a plist ordered by priority,
    +    to select devices. Devices at the same priority are rotated with
    +    plist_requeue(), so every cluster transition serializes allocators on
    +    swap_avail_lock and repeatedly drops and reacquires that global lock.

    -    Add a custom priority queue that groups devices into rings, ordered
    -    by priority to form a static array as the queue.  Devices in the
    -    same priority ring also form a static array.
    +    Replace device selection with a priority queue made of immutable rings.
    +    Each ring contains devices at one priority, and each CPU has a local
    +    reader that rotates through a ring after a fixed allocation quota. A
    +    task-local cursor keeps a retry walk stable even if another task updates
    +    the shared reader between attempts, so each peer is visited exactly once
    +    before the allocator considers a lower priority.

    -    To eliminate CPU contention on a global lock, swapon_rwsem is reused
    -    to protect the queue.  The queue stays read-only, only swapon and
    -    swapoff can adjust the length of the queue or rings in it.  The
    -    writer lock is costly but swapon and swapoff are rare so this is
    -    acceptable.
    +    Disable task migration across the retry walk so queue selection, quota
    +    accounting and per-CPU cluster allocation stay on the same CPU. This
    +    preserves per-CPU pacing without making the sleepable allocation loop an
    +    atomic context.

    -    Since the queue is read-only, rotation is done from the reader side.
    -    Each CPU maintains its own iterator using a counter that counts down
    -    to advance to the next device in the same ring, so allocation is
    -    round-robin while retaining locality for clustering.  After a failed
    -    attempt the iterator also advances so the caller does not retry the
    -    same full or fragmented device.
    +    Keep the queue structure stable under swapon_rwsem. Full or disabled
    +    devices remain in their ring with a tag in the low bit of the stored
    +    pointer. Serialize tag writers with swap_queue_update_lock and pair the
    +    lockless full-pointer reads and writes with READ_ONCE() and WRITE_ONCE().
    +    The in-use counter carries a separate off-list bit so full-to-available
    +    transitions update the counter and pointer tag consistently.

    -    If a whole ring is attempted and allocation could not be made, the
    -    iterator falls through to the next ring.  This almost never happens
    -    because we always want to allocate from the device with the highest
    -    priority, unless that device is full.
    +    Publish swap_file, the live percpu reference, queue membership and
    +    SWP_WRITEOK in one swapon writer section. Preserve the writer-serialized
    +    swapoff lookup and disable invariant established earlier in the series.

    -    To speed up iteration when there are multiple full devices, a device
    -    pointer stored in the ring can be marked as unavailable so the
    -    iterator skips past it.  Fully used devices can mark their pointer
    -    as unavailable without taking the rwsem lock.  The marking is done using
    -    a spin lock to sync with potential writer because percpu rwsem is too
    -    heavy for writers, and full device transitions are more frequent and
    -    performance-sensitive compared to swapon and swapoff.
    +    For large folios, try every device in the selected priority ring before
    +    returning -E2BIG to request a split. Do not fall back to a lower priority
    +    device merely because the first same-priority device is fragmented.

    -    With this patch, swap allocation should be faster and cleaner.  The
```

```
-    percpu rwsem has very low overhead for the allocator, and it can now
-    hold swapon_rwsem for the entire allocation loop, no longer needing to
-    keep dropping and reacquiring a lock to rotate list entries.
-
-    The old swap_avail_head plist is still maintained in parallel for
-    now, updated alongside the queue in del_from_avail_list() and
-    add_to_avail_list().  It will be removed once the remaining consumer
-    is converted in a follow-up patch.
+    The old available plist is still maintained in parallel in this commit
+    so the transition remains bisectable. It is removed by the next patch.

+    Link: https://lore.kernel.org/20260714-swap-pcp-priq-v1-9-
de9b164ed419@tencent.com
+    Link: https://lore.kernel.org/alZ7UBXweuuOX4qz@yjaykim-PowerEdge-T330
+    Link: https://lore.kernel.org/77d6da3d-10af-49a1-a356-72aa8b462e85@gmail.com
     Signed-off-by: Kairui Song <kasong@tencent.com>
+    Co-developed-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
+    Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
+
+ ## include/linux/swap.h ##
+@@ include/linux/swap.h: struct swap_extent {
+  * - SWP_USED: Protected by swapon_rwsem. Indicates the device is inuse. Once
+  *   set, won't be cleared unless all reference to this device is freed and
+  *   swapoff finished.
+- * - SWP_WRITEOK: Protected by both swapon_rwsem and swap_avail_lock, clearing
+-  *   this flag also waits for all current cluster lock users to exit so
++ * - SWP_WRITEOK: Protected by both swapon_rwsem and swap_queue_update_lock.
++  *   Clearing this flag is followed by waiting for all current cluster lock
++  *   users to exit, so
+  *   checking this flag while holding any of these locks ensures the device
+  *   is safe to use at the moment. Note: clearing this flag doesn't affect
+  *   pending IO or async requests, it only prevents further entry allocation

  ## mm/swapfile.c ##
 @@ mm/swapfile.c: static bool folio_swapcache_freeable(struct folio *folio);
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en

 +/*
 + * All available swap_info_structs are grouped by priority rings, the rings
-+ * are ordered in a queue by priority (lower prio value = higher priority).
++ * are ordered in a queue by priority (higher prio value = higher priority).
 + * The allocator iterates and rotates devices within each priority ring.
 + * When all devices in a ring are iterated, it goes to the next lower
 + * priority ring.
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 + * The ring is protected by swapon_rwsem so updating it is costly. To make
 + * the allocator and other users skip full devices faster, the lowest bit of
 + * a device pointer is used to mark it disabled (temporarily unavailable).
-+ * This relies on struct swap_info_struct being sufficiently
-+ * aligned (guaranteed by kmalloc).
++ * This relies on the natural alignment of struct swap_info_struct.
 + */
 +#define SWAP_DEVICE_MASKED_SHIFT  0
 +#define SWAP_DEVICE_MASKED_BIT            BIT(SWAP_DEVICE_MASKED_SHIFT)
++static_assert(__alignof__(struct swap_info_struct) >= 2);
 +
 +/*
 + * Serializes queue content mutations and keeps SWAP_USAGE_OFFLIST_BIT
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +  struct swap_ring_iterator ri[];
 +};
 +
++struct swap_queue_cursor {
++  bool valid;
++  bool ring_only;
++  unsigned int ring_idx;
++  unsigned int offset;
++};
++
```

```
 +static __percpu struct swap_queue_reader *swap_queue_readers;
 +
 +static inline bool swap_device_masked(struct swap_info_struct *si)
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +  return (unsigned long)si & SWAP_DEVICE_MASKED_BIT;
 +}
 +
++static inline struct swap_info_struct *swap_device_mask_ptr(struct
swap_info_struct *si)
++{
++  return (struct swap_info_struct *)((unsigned long)si |
++                                     SWAP_DEVICE_MASKED_BIT);
++}
++
 +static inline struct swap_info_struct *swap_device_unmask_ptr(struct
swap_info_struct *si)
 +{
 +  return (struct swap_info_struct *)((unsigned long)si & ~SWAP_DEVICE_MASKED_BIT);
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +                  ring = swap_queue[ring_idx];
 +                  per_cpu_ptr(readers, cpu)->ri[ring_idx].offset =
 +                              cpu % ring->size;
++                  per_cpu_ptr(readers, cpu)->ri[ring_idx].rr_counter =
++                              SWAP_ROUND_ROBIN_QUOTA;
 +          }
 +  }
 +}
 +
-+static struct swap_info_struct *swap_queue_get_device(long nr_alloc, int nr_iter)
++static struct swap_info_struct *swap_queue_get_device(long nr_alloc, int nr_iter,
++                                                      struct swap_queue_cursor *cursor)
 +{
 +  bool rotate = false;
 +  struct swap_info_struct *si;
 +  struct swap_ring_iterator *ri;
 +  struct swap_prio_ring *ring;
-+  unsigned int queue_idx;
++  unsigned int dev_idx, queue_idx;
 +
 +  if (!swap_queue_len)
 +          return ERR_PTR(-ENOENT);
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +  while (nr_iter >= swap_queue[queue_idx]->size) {
 +          nr_iter -= swap_queue[queue_idx]->size;
 +          if (++queue_idx >= swap_queue_len)
-+                  return ERR_PTR(-ENOENT);
++                  return ERR_PTR(cursor->ring_only ? -E2BIG : -ENOENT);
 +  }
++  if (cursor->ring_only && cursor->ring_idx != queue_idx)
++          return ERR_PTR(-E2BIG);
 +
 +  ring = swap_queue[queue_idx];
 +  local_lock(&swap_queue_readers->lock);
 +  ri = this_cpu_ptr(&swap_queue_readers->ri[queue_idx]);
-+  /* Rotate while iterating the ring, just not on the first try */
-+  if (nr_iter)
-+          rotate = true;
-+  else if (ri->rr_counter < nr_alloc)
-+          rotate = true;
-+  else if (ri->offset >= ring->size)
-+          rotate = true;
-+  if (rotate) {
-+          ri->offset++;
-+          ri->offset %= ring->size;
++  /*
++   * Snapshot the starting offset for this allocation's walk. The shared
++   * iterator can move between retries, but the cursor must visit every
++   * device in the ring exactly once before falling through.
++   */
```

```
++  if (!cursor->valid || cursor->ring_idx != queue_idx) {
++          cursor->valid = true;
++          cursor->ring_idx = queue_idx;
++          if (ri->rr_counter < nr_alloc)
++                  rotate = true;
++          else if (ri->offset >= ring->size)
++                  rotate = true;
++          if (rotate) {
++                  ri->offset++;
++                  ri->offset %= ring->size;
++                  ri->rr_counter = SWAP_ROUND_ROBIN_QUOTA;
++          }
++          cursor->offset = ri->offset;
++  }
++
++  dev_idx = (cursor->offset + nr_iter) % ring->size;
++  if (nr_iter) {
++          ri->offset = dev_idx;
 +          ri->rr_counter = SWAP_ROUND_ROBIN_QUOTA;
 +  }
 +  ri->rr_counter -= nr_alloc;
-+  si = READ_ONCE(ring->dev[ri->offset]);
++  si = READ_ONCE(ring->dev[dev_idx]);
 +  local_unlock(&swap_queue_readers->lock);
 +
 +  if (swap_device_masked(si))
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +
 +  lockdep_assert_held(&swap_queue_update_lock);
 +  if (swap_queue_find(si, &ring_idx, &dev_idx))
-+          __set_bit(SWAP_DEVICE_MASKED_SHIFT,
-+                    (unsigned long *)&swap_queue[ring_idx]->dev[dev_idx]);
++          WRITE_ONCE(swap_queue[ring_idx]->dev[dev_idx],
++                     swap_device_mask_ptr(si));
 +}
 +
 +static void swap_queue_unmask(struct swap_info_struct *si)
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +
 +  lockdep_assert_held(&swap_queue_update_lock);
 +  if (swap_queue_find(si, &ring_idx, &dev_idx))
-+          __clear_bit(SWAP_DEVICE_MASKED_SHIFT,
-+                      (unsigned long *)&swap_queue[ring_idx]->dev[dev_idx]);
++          WRITE_ONCE(swap_queue[ring_idx]->dev[dev_idx], si);
 +}
 +
 +static int swap_queue_add(struct swap_info_struct *si)
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 +  if (ring->size > 1)
 +          new_ring = kmalloc(struct_size(ring, dev, ring->size - 1), gfp);
 +  if (ring->size == 1 && swap_queue_len > 1) {
-+          new_readers = swap_queue_prealloc_readers(
-+                                    swap_queue_len - 1, gfp);
++          new_readers = swap_queue_prealloc_readers(swap_queue_len - 1,
++                                                    gfp);
 +          new_queue = kmalloc(sizeof(*swap_queue) *
 +                              (swap_queue_len - 1), gfp);
 +  }
@@ mm/swapfile.c: static struct swap_info_struct *swap_entry_to_info(swp_entry_t en
 + * This bit will be set if the device is not in the available queue
 + * and not usable, will be cleared if the device is in the queue.
   */
- #define SWAP_USAGE_OFFLIST_BIT (1UL << (BITS_PER_TYPE(atomic_t) - 2))
+-#define SWAP_USAGE_OFFLIST_BIT (1UL << (BITS_PER_TYPE(atomic_t) - 2))
++#define SWAP_USAGE_OFFLIST_BIT BIT(BITS_PER_LONG - 2)
  #define SWAP_USAGE_COUNTER_MASK (~SWAP_USAGE_OFFLIST_BIT)
+ static long swap_usage_in_pages(struct swap_info_struct *si)
+ {
 @@ mm/swapfile.c: static void del_from_avail_list(struct swap_info_struct *si, bool
```

```
  swapoff)
          unsigned long pages;

      @@ mm/swapfile.c: static void add_to_avail_list(struct swap_info_struct *si)
       +   * skip the rest.
           */
          pages = si->pages;
      -   if (val == pages) {
      -@@ mm/swapfile.c: static void add_to_avail_list(struct swap_info_struct *si)
      +-  if (val == pages) {
      ++  if ((val & SWAP_USAGE_COUNTER_MASK) == pages) {
      +           /* Just like the cmpxchg in del_from_avail_list */
      +           if (atomic_long_try_cmpxchg(&si->inuse_pages, &pages,
      +                                       pages | SWAP_USAGE_OFFLIST_BIT))
                          goto skip;
          }

      @@ mm/swapfile.c: static void add_to_avail_list(struct swap_info_struct *si)
          plist_add(&si->avail_list, &swap_avail_head);
       -
        skip:
      --  spin_unlock(&swap_avail_lock);
       +  spin_unlock(&swap_queue_update_lock);
      +   spin_unlock(&swap_avail_lock);
        }

      - /*
       @@ mm/swapfile.c: static void swap_device_inuse_add(struct swap_info_struct *si,

          /*
      @@ mm/swapfile.c: static bool get_swap_device_info(struct swap_info_struct *si)
       +static int swap_alloc_entry(struct folio *folio)
        {
       -  struct swap_info_struct *si, *next;
      ++  struct swap_queue_cursor cursor = {};
       +  long nr_pages = folio_nr_pages(folio);
       +  struct swap_info_struct *si;
       +  int nr_iter, ret;
      @@ mm/swapfile.c: static bool get_swap_device_info(struct swap_info_struct *si)
       -                  if (folio_test_large(folio))
       -                          return;
       +  percpu_down_read(&swapon_rwsem);
      ++  migrate_disable();
       +  for (nr_iter = 0;; nr_iter++) {
      -+          si = swap_queue_get_device(nr_pages, nr_iter);
      ++          si = swap_queue_get_device(nr_pages, nr_iter, &cursor);
       +          if (IS_ERR(si)) {
       +                  ret = PTR_ERR(si);
       +                  if (ret == -EBUSY)
      @@ mm/swapfile.c: static bool get_swap_device_info(struct swap_info_struct *si)
       -           * may have been modified; so if next is still in the
       -           * swap_avail_head list then try it, otherwise start over if we
       -           * have not gotten any slots.
      -+           * For large allocation, return error directly to inform the
      -+           * caller to split it instead of fallback to other devices.
      ++           * For large allocations, try every device at the same priority,
      ++           * but ask the caller to split instead of falling back to a lower
      ++           * priority ring.
                   */
       -          if (plist_node_empty(&next->avail_list))
       -                  goto start_over;
       +          if (folio_test_large(folio)) {
      -+                  ret = -E2BIG;
      -+                  break;
      ++                  cursor.ring_only = true;
      ++                  continue;
       +          }
          }
       -  spin_unlock(&swap_avail_lock);
```

```
 +
++  migrate_enable();
 +  percpu_up_read(&swapon_rwsem);
 +  return ret;
  }
@@ mm/swapfile.c: int folio_alloc_swap(struct folio *folio)
    return 0;
  }
 @@ mm/swapfile.c: static int setup_swap_extents(struct swap_info_struct *sis,
- static void swap_device_enable(struct swap_info_struct *si)
+ }
+
+ /*
+- * Mark a fully initialized swap device writable and expose it to the
+- * allocator. The caller must have resurrected its percpu ref first.
++ * Mark a fully initialized swap device writable and expose it to the allocator.
++ * The caller must have resurrected its percpu ref before entering this helper.
+  */
+-static void swap_device_enable(struct swap_info_struct *si)
++static void __swap_device_enable(struct swap_info_struct *si)
  {
-   percpu_down_write(&swapon_rwsem);
+-  percpu_down_write(&swapon_rwsem);
 -  spin_lock(&swap_avail_lock);
-+  spin_lock(&swap_queue_update_lock);
-   si->flags |= SWP_WRITEOK;
+-  si->flags |= SWP_WRITEOK;
 -  spin_unlock(&swap_avail_lock);
--
++  lockdep_assert_held_write(&swapon_rwsem);
+
++  spin_lock(&swap_queue_update_lock);
++  si->flags |= SWP_WRITEOK;
 +  spin_unlock(&swap_queue_update_lock);
    atomic_long_add(si->pages, &nr_swap_pages);
    total_swap_pages += si->pages;
    plist_add(&si->list, &swap_active_head);
-@@ mm/swapfile.c: static int swap_device_disable(struct swap_info_struct *si)
+-  percpu_up_write(&swapon_rwsem);
+-
+-  add_to_avail_list(si);
+ }
+
++static void swap_device_enable(struct swap_info_struct *si)
++{
++  percpu_down_write(&swapon_rwsem);
++  __swap_device_enable(si);
++  percpu_up_write(&swapon_rwsem);
++}
++
+ static int swap_device_disable(struct address_space *mapping,
+                                struct swap_info_struct **swap_info)
+ {
+@@ mm/swapfile.c: static int swap_device_disable(struct address_space *mapping,
    }
    vm_unacct_memory(si->pages);

@@ mm/swapfile.c: static int swap_device_disable(struct swap_info_struct *si)
    plist_del(&si->list, &swap_active_head);
    total_swap_pages -= si->pages;
    atomic_long_sub(si->pages, &nr_swap_pages);
--  percpu_up_write(&swapon_rwsem);
--
-   del_from_avail_list(si, true);
-+  percpu_up_write(&swapon_rwsem);
-
-   /*
-    * Swap allocator doesn't touch si lock, so looping through all
 @@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
```

```
                return err;
        }

     +  percpu_down_write(&swapon_rwsem);
     +  swap_queue_del(p);
    ++  percpu_ref_kill(&p->users);
    ++  percpu_up_write(&swapon_rwsem);
     +
        /*
         * Wait for swap operations protected by get/put_swap_device()
         * to complete.  Because of synchronize_rcu() here, all swap
     @@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)
    -   if (!(p->flags & SWP_SOLIDSTATE))
    -           atomic_dec(&nr_rotate_swap);
    -
    --  percpu_down_write(&swapon_rwsem);
    -   spin_lock(&p->lock);
    -   drain_mmlist();
    +    * prevent folio_test_swapcache() and the following swap cache
    +    * operations from racing with swapoff.
    +    */
    +-  percpu_ref_kill(&p->users);
    +   synchronize_rcu();
    +   wait_for_completion(&p->comp);

     @@ mm/swapfile.c: SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int,
swap_flags)
    +   si->prio = prio;
    +   si->list.prio = -si->prio;
    +   si->avail_list.prio = -si->prio;
    +-  si->swap_file = swap_file;

    -   /* Sets SWP_WRITEOK, resurrect the percpu ref, expose the swap device */
    -   percpu_ref_resurrect(&si->users);
    +-  /* Sets SWP_WRITEOK, resurrect the percpu ref, expose the swap device */
    ++  /*
    ++   * Publish swap_file before making the percpu ref live, then add the device
    ++   * to the queue and make it writable under the same write-side lock. This
    ++   * keeps lockless ref users and /proc/swaps from observing partial state.
    ++   */
     +  percpu_down_write(&swapon_rwsem);
    ++  si->swap_file = swap_file;
    +   percpu_ref_resurrect(&si->users);
    +-  swap_device_enable(si);
     +  error = swap_queue_add(si);
    ++  if (error) {
    ++          si->swap_file = NULL;
    ++          percpu_ref_kill(&si->users);
    ++  } else {
    ++          __swap_device_enable(si);
    ++  }
     +  percpu_up_write(&swapon_rwsem);
    -+  if (error)
    ++  if (error) {
    ++          wait_for_completion(&si->comp);
    ++          inode->i_flags &= ~S_SWAPFILE;
     +          goto free_swap_zswap;
    -+
    -   swap_device_enable(si);
    ++  }

        pr_info("Adding %uk swap on %s.  Priority:%d extents:%d across:%lluk
%s%s%s%s\n",
    +           K(si->pages), name->name, si->prio, nr_extents,
10:  0f008a0a607b ! 10:  f205c502011b mm/swap: remove available list
    @@ Commit message
         Pure cleanup, no functional change.

         After the priority queue replaced the allocator's device selection,
```

```
      -    swap_avail_head and swap_avail_lock are useless now.  Remove them.
      +    swap_avail_head and swap_avail_lock are no longer used. Remove them.

      -    __folio_throttle_swaprate() now iterates swap_active_head under
      -    swapon_rwsem instead, the SWP_WRITEOK filtering makes it equivalent to
      -    before.
      +    The throttle path now walks swap_active_head under swapon_rwsem.
      +    Check SWP_WRITEOK and the existing off-list state. This preserves the old
      +    priority ordering and does not select a full swap device.

           Signed-off-by: Kairui Song <kasong@tencent.com>
      +    Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>

        ## include/linux/swap.h ##
       @@ include/linux/swap.h: struct swap_info_struct {
      @@ include/linux/swap.h: struct swap_info_struct {
           struct work_struct reclaim_work; /* reclaim worker */
           struct list_head discard_clusters; /* discard clusters list */
       -   struct plist_node avail_list;   /* entry in swap_avail_head */
      +    const struct swap_ops *ops;
        };

      - static inline swp_entry_t page_swap_entry(struct page *page)

        ## mm/swapfile.c ##
       @@ mm/swapfile.c: static const char Bad_offset[] = "Bad swap offset entry ";
      @@ mm/swapfile.c: static void add_to_avail_list(struct swap_info_struct *si)
       -   plist_add(&si->avail_list, &swap_avail_head);
        skip:
           spin_unlock(&swap_queue_update_lock);
      +-   spin_unlock(&swap_avail_lock);
        }
      +
      + /*
       @@ mm/swapfile.c: static struct swap_info_struct *alloc_swap_info(void)
           }
           p->swap_extent_root = RB_ROOT;
      @@ mm/swapfile.c: SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int, sw
           si->prio = prio;
           si->list.prio = -si->prio;
       -   si->avail_list.prio = -si->prio;
      -    si->swap_file = swap_file;

      -    /* Sets SWP_WRITEOK, resurrect the percpu ref, expose the swap device */
      +    /*
      +     * Publish swap_file before making the percpu ref live, then add the device
       @@ mm/swapfile.c: void __folio_throttle_swaprate(struct folio *folio, gfp_t gfp)
           if (current->throttle_disk)
                   return;
      @@ mm/swapfile.c: void __folio_throttle_swaprate(struct folio *folio, gfp_t gfp)
       -           if (si->bdev) {
       +   percpu_down_read(&swapon_rwsem);
       +   plist_for_each_entry(si, &swap_active_head, list) {
      -+           if ((si->flags & SWP_WRITEOK) && si->bdev) {
      ++           if ((si->flags & SWP_WRITEOK) &&
      ++               !(atomic_long_read(&si->inuse_pages) &
      ++                 SWAP_USAGE_OFFLIST_BIT) && si->bdev) {
                         blkcg_schedule_throttle(si->bdev->bd_disk, true);
                         break;
                   }
  11:  5779f452ad52 ! 11:  05b43f03ab25 mm/swap: perform sync discard on single device
  more proactively
      @@ Metadata
       Author: Kairui Song <kasong@tencent.com>

        ## Commit message ##
      -    mm/swap: perform sync discard on single device more proactively
      +    mm/swap: bound synchronous discard during allocation
```

```
-    Previous commit 9fb749cd15078 ("mm, swap: do not perform synchronous
-    discard during allocation") added a workaround-style global sync discard
-    when all devices are drained to prevent OOM. It wasn't an optimal
-    solution and in the discussion [1], it's preferred to discard more
-    proactively, and in the scope of a single device instead of globally.
-    That wasn't achievable due to the limitations of the previous swap
-    allocation design, or would have ended up too complex.
+    The old global per-CPU cluster cache made it difficult to select a
+    specific device before handling its pending discards.  Now that cluster
+    state is per-device, let an allocator issue discard for that device when
+    its free-cluster list is drained instead of waiting for a global fallback
+    scan.

-    Now with the new workflow and swap cluster cache moved inside the device
-    scope, we can easily achieve that. So drop the old workaround and
-    implement a more proactive and simpler solution. Doing discard when the
-    free list is drained will prevent fragmentation better and avoid a
-    global discard scan.
+    Do not drain the entire discard list from one allocation.  The list is
+    also updated by freeing paths, so an unlocked non-empty check races with
+    those writers, and a full synchronous drain has no fixed latency bound
+    when other CPUs continue to enqueue clusters.

-    Link: https://lore.kernel.org/all/CAMgjq7CsYhEjvtN85XGkrONYAJxve7gG593TFeOGV-oax++kWA@mail.gmail.com/ [1]
+    Factor out swap_discard_one_cluster().  Isolate one cluster under
+    si->lock, retain CLUSTER_FLAG_DISCARD while I/O is in flight, and return
+    it to the free list after the discard completes.  The background worker
+    continues looping until the list is empty, while an allocator handles at
+    most one cluster before retrying allocation.
+
+    This keeps the proactive per-device policy while giving each allocation
+    a fixed synchronous discard budget.
+
+    Link: https://lore.kernel.org/all/CAMgjq7CsYhEjvtN85XGkrONYAJxve7gG593TFeOGV-oax++kWA@mail.gmail.com/
     Signed-off-by: Kairui Song <kasong@tencent.com>
+    Co-developed-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
+    Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>

  ## mm/swapfile.c ##
 @@ mm/swapfile.c: static long swap_usage_in_pages(struct swap_info_struct *si)
@@ mm/swapfile.c: swap_cluster_populate(struct swap_info_struct *si,
     spin_lock(&ci->lock);

     if (ret) {
+@@ mm/swapfile.c: static struct swap_cluster_info *isolate_lock_cluster(
+ }
+
+ /*
+- * Doing discard actually. After a cluster discard is finished, the cluster
+- * will be added to free cluster list. Discard cluster is a bit special as
+- * they don't participate in allocation or reclaim, so clusters marked as
+- * CLUSTER_FLAG_DISCARD must remain off-list or on discard list.
++ * Discard one cluster. After the discard is finished, the cluster will be
++ * added to the free cluster list. Discard clusters are special because they
++ * don't participate in allocation or reclaim, so CLUSTER_FLAG_DISCARD must
++ * remain set while a cluster is either queued or being discarded.
+  */
+-static bool swap_do_scheduled_discard(struct swap_info_struct *si)
++static bool swap_discard_one_cluster(struct swap_info_struct *si)
+ {
+-   struct swap_cluster_info *ci;
+-   bool ret = false;
+-   unsigned int idx;
+
+    spin_lock(&si->lock);
+-   while (!list_empty(&si->discard_clusters)) {
+-           ci = list_first_entry(&si->discard_clusters, struct swap_cluster_info,
```

```
  list);
      +-          /*
      +-           * Delete the cluster from list to prepare for discard, but keep
      +-           * the CLUSTER_FLAG_DISCARD flag, there could be percpu_cluster
      +-           * pointing to it, or ran into by relocate_cluster.
      +-           */
      +-          list_del(&ci->list);
      +-          idx = cluster_index(si, ci);
      ++  if (list_empty(&si->discard_clusters)) {
      +           spin_unlock(&si->lock);
      +-          discard_swap_cluster(si, idx * SWAPFILE_CLUSTER,
      +-                               SWAPFILE_CLUSTER);
      +-
      +-          spin_lock(&ci->lock);
      +-          /*
      +-           * Discard is done, clear its flags as it's off-list, then
      +-           * return the cluster to allocation list.
      +-           */
      +-          ci->flags = CLUSTER_FLAG_NONE;
      +-          __free_cluster(si, ci);
      +-          spin_unlock(&ci->lock);
      +-          ret = true;
      +-          spin_lock(&si->lock);
      ++          return false;
      +   }
      ++
      ++  ci = list_first_entry(&si->discard_clusters,
      ++                         struct swap_cluster_info, list);
      ++  /*
      ++   * Delete the cluster from the list, but keep CLUSTER_FLAG_DISCARD
      ++   * set while the discard is in flight. A percpu cluster may still
      ++   * point to it, or relocate_cluster() may encounter it.
      ++   */
      ++  list_del(&ci->list);
      ++  idx = cluster_index(si, ci);
      +   spin_unlock(&si->lock);
      +-  return ret;
      ++
      ++  discard_swap_cluster(si, idx * SWAPFILE_CLUSTER, SWAPFILE_CLUSTER);
      ++
      ++  spin_lock(&ci->lock);
      ++  ci->flags = CLUSTER_FLAG_NONE;
      ++  __free_cluster(si, ci);
      ++  spin_unlock(&ci->lock);
      ++  return true;
      + }
      +
      + static void swap_discard_work(struct work_struct *work)
      +@@ mm/swapfile.c: static void swap_discard_work(struct work_struct *work)
      +
      +   si = container_of(work, struct swap_info_struct, discard_work);
      +
      +-  swap_do_scheduled_discard(si);
      ++  while (swap_discard_one_cluster(si))
      ++          cond_resched();
      + }
      +
      + static void swap_users_ref_free(struct percpu_ref *ref)
      +@@ mm/swapfile.c: static unsigned long cluster_alloc_swap_entry(struct
  swap_info_struct *si,
      +   struct swap_cluster_info *ci;
      +   unsigned int order = likely(folio) ? folio_order(folio) : 0;
      +   unsigned int offset = SWAP_ENTRY_INVALID, found = SWAP_ENTRY_INVALID;
      ++  bool discarded = false;
      +
      +   /*
      +    * Swapfile is not block device so unable
       @@ mm/swapfile.c: static unsigned long cluster_alloc_swap_entry(struct
  swap_info_struct *si,
```

```
          if (order && !(si->flags & SWP_BLKDEV))
                  return 0;
      @@ mm/swapfile.c: static unsigned long cluster_alloc_swap_entry(struct swap_info_st
                  if (found)
                          goto done;
       +
      -+          if (!list_empty(&si->discard_clusters)) {
      ++          if (!discarded) {
       +                  swap_alloc_unlock_device(si);
      -+                  swap_do_scheduled_discard(si);
      -+                  goto restart;
      ++                  if (swap_discard_one_cluster(si)) {
      ++                          discarded = true;
      ++                          goto restart;
      ++                  }
      ++                  swap_alloc_lock_device(si);
       +          }
          }

   12:  d7ce531b224d ! 12:  510984375538 mm/swap: drop swap active plist
      @@ Commit message
           swap_info_struct, and migrate the remaining plist operations.

           Signed-off-by: Kairui Song <kasong@tencent.com>
      +    Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>

        ## include/linux/swap.h ##
       @@ include/linux/swap.h: struct swap_info_struct {
      @@ mm/swapfile.c: bool swap_migration_ad_supported;
        static inline struct swap_info_struct *__swap_iter(int *i, unsigned long flag)
        {
          lockdep_assert_held(&swapon_rwsem);
      -@@ mm/swapfile.c: static void swap_device_enable(struct swap_info_struct *si)
      +@@ mm/swapfile.c: static void __swap_device_enable(struct swap_info_struct *si)
          spin_unlock(&swap_queue_update_lock);
          atomic_long_add(si->pages, &nr_swap_pages);
          total_swap_pages += si->pages;
       -  plist_add(&si->list, &swap_active_head);
      -   percpu_up_write(&swapon_rwsem);
      -
          add_to_avail_list(si);
      -@@ mm/swapfile.c: static int swap_device_disable(struct swap_info_struct *si)
      + }
      +
      +@@ mm/swapfile.c: static int swap_device_disable(struct address_space *mapping,
      +   int err = -EINVAL;
      +
      +   percpu_down_write(&swapon_rwsem);
      +-  plist_for_each_entry(si, &swap_active_head, list) {
      +-          if ((si->flags & SWP_WRITEOK) &&
      +-              si->swap_file->f_mapping == mapping) {
      ++  for_each_avail_swap(si) {
      ++          if (si->swap_file->f_mapping == mapping) {
      +                   err = 0;
      +                   break;
      +           }
      +@@ mm/swapfile.c: static int swap_device_disable(struct address_space *mapping,
          spin_lock(&swap_queue_update_lock);
          si->flags &= ~SWP_WRITEOK;
          spin_unlock(&swap_queue_update_lock);
       -  plist_del(&si->list, &swap_active_head);
          total_swap_pages -= si->pages;
          atomic_long_sub(si->pages, &nr_swap_pages);
      -   del_from_avail_list(si, true);
      -@@ mm/swapfile.c: SYSCALL_DEFINE1(swapoff, const char __user *, specialfile)

      -   mapping = victim->f_mapping;
      -   percpu_down_read(&swapon_rwsem);
      --  plist_for_each_entry(p, &swap_active_head, list) {
```

```
      --          if (p->flags & SWP_WRITEOK &&
      --              p->swap_file->f_mapping == mapping) {
      -+  for_each_avail_swap(p) {
      -+          if (p->swap_file->f_mapping == mapping) {
      -                   found = 1;
      -                   break;
      -           }
       @@ mm/swapfile.c: static struct swap_info_struct *alloc_swap_info(void)
                   */
          }
      @@ mm/swapfile.c: SYSCALL_DEFINE2(swapon, const char __user *, specialfile, int, sw
       -   */
          si->prio = prio;
       -  si->list.prio = -si->prio;
      -   si->swap_file = swap_file;

      -   /* Sets SWP_WRITEOK, resurrect the percpu ref, expose the swap device */
      +   /*
      +    * Publish swap_file before making the percpu ref live, then add the device
       @@ mm/swapfile.c: int swap_dup_entry_direct(swp_entry_t entry)
        #if defined(CONFIG_MEMCG) && defined(CONFIG_BLK_CGROUP)
        static bool __has_usable_swap(void)
      @@ mm/swapfile.c: void __folio_throttle_swaprate(struct folio *folio, gfp_t gfp)

          percpu_down_read(&swapon_rwsem);
       -  plist_for_each_entry(si, &swap_active_head, list) {
      --          if ((si->flags & SWP_WRITEOK) && si->bdev) {
      +-          if ((si->flags & SWP_WRITEOK) &&
      +-              !(atomic_long_read(&si->inuse_pages) &
       +  for_each_avail_swap(si) {
      -+          if (si->bdev) {
      ++          if (!(atomic_long_read(&si->inuse_pages) &
      +                 SWAP_USAGE_OFFLIST_BIT) && si->bdev) {
                     blkcg_schedule_throttle(si->bdev->bd_disk, true);
                     break;
      -           }
  13:  a438694aa41a = 13:  d5c8964cf19f lib/plist.c: remove requeue function


  ==== Attachment 15 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-
  review-and-testing/15-kmb-support.patch) ====
  From 91817c66ab07fc39400cf426b36ac4678cd048d9 Mon Sep 17 00:00:00 2001
  From: "Lian Wang (Processmission)" <lianux.mm@gmail.com>
  Date: Fri, 7 Aug 2026 11:03:13 +0800
  Subject: [KMB PATCH 1/4] tests: add swap queue v1/v2 reproduction matrix

  Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
  ---
   .../.build-repro-common.sh                    | 161 ++++++++++++++++++
   .../.prepare.sh                               |  44 +++++
   .../10-kernel-build-2g-j96.sh                 |  12 ++
   .../20-kernel-build-3g-j96.sh                 |  12 ++
   .../30-usemem-brd-scaled-12dev.sh             |  53 ++++++
   5 files changed, 282 insertions(+)
   create mode 100644 testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
   create mode 100755 testsuites/kairui-swapq-v1-v2-repro-20260807/.prepare.sh
   create mode 100755 testsuites/kairui-swapq-v1-v2-repro-20260807/10-kernel-build-2g-
  j96.sh
   create mode 100755 testsuites/kairui-swapq-v1-v2-repro-20260807/20-kernel-build-3g-
  j96.sh
   create mode 100755 testsuites/kairui-swapq-v1-v2-repro-20260807/30-usemem-brd-scaled-
  12dev.sh

  diff --git a/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
  b/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
  new file mode 100644
  index 0000000..10d5e6f
  --- /dev/null
  +++ b/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
```

```
@@ -0,0 +1,161 @@
+#!/bin/bash
+# SPDX-License-Identifier: GPL-2.0
+
+# This file is sourced by the ordered test steps. Dot-prefixing keeps KMB from
+# treating it as an independent benchmark case.
+
+BUILD_REPETITIONS=${KMB_BUILD_REPETITIONS:-12}
+BUILD_JOBS=${KMB_BUILD_JOBS:-96}
+ZRAM_TOTAL_SIZE=${KMB_ZRAM_TOTAL_SIZE:-64G}
+ZRAM_DEVICES=${KMB_ZRAM_DEVICES:-8}
+build_pid=
+sampler_pid=
+tmp_dir=$(mktemp -d /tmp/kmb-swapq-build-repro.XXXXXX)
+root_swap_was_active=0
+dmesg_before="$tmp_dir/dmesg.before"
+dmesg > "$dmesg_before"
+
+if awk '$1 == "/swap.img" { found = 1 } END { exit !found }' /proc/swaps; then
+       root_swap_was_active=1
+fi
+
+cleanup_build_repro()
+{
+       local rc=${1:-0}
+       local dev
+
+       trap - EXIT INT TERM
+       set +e
+       [ -z "$build_pid" ] || kill -TERM "$build_pid" 2>/dev/null
+       [ -z "$sampler_pid" ] || kill -TERM "$sampler_pid" 2>/dev/null
+       wait 2>/dev/null
+
+       for dev in /dev/zram*; do
+               [ -b "$dev" ] || continue
+               swapoff "$dev" 2>/dev/null
+       done
+       for dev in /sys/block/zram*; do
+               [ -w "$dev/reset" ] || continue
+               printf '1\n' > "$dev/reset" 2>/dev/null
+       done
+
+       if [ "$root_swap_was_active" -eq 1 ] &&
+          ! awk '$1 == "/swap.img" { found = 1 } END { exit !found }' /proc/swaps; then
+               swapon -p -1 /swap.img || rc=1
+       fi
+
+       dmesg | tail -n "+$(( $(wc -l < "$dmesg_before") + 1 ))" >
"$tmp_dir/dmesg.delta"
+       if grep -Ei 'BUG:|WARNING:|Oops:|kernel panic|KASAN:|KCSAN:|UBSAN:|soft
lockup|hard LOCKUP|rcu.*stall' \
+               "$tmp_dir/dmesg.delta"; then
+               rc=1
+       fi
+
+       echo "=== final swap state ==="
+       cat /proc/swaps
+       rm -rf -- "$tmp_dir"
+       exit "$rc"
+}
+
+trap 'cleanup_build_repro $?' EXIT
+trap 'exit 130' INT
+trap 'exit 143' TERM
+
+sample_swap_peaks()
+{
+       local watched_pid=$1
+       local output=$2
```

```
+	local -a peaks=(0 0 0 0 0 0 0 0)
+	local i used
+
+	while kill -0 "$watched_pid" 2>/dev/null; do
+		for ((i = 0; i < ZRAM_DEVICES; i++)); do
+			used=$(awk -v dev="/dev/zram$i" '$1 == dev { print $4; found = 1
} END { if (!found) print 0 }' /proc/swaps)
+			[ "$used" -le "${peaks[$i]}" ] || peaks[$i]=$used
+		done
+		sleep 1
+	done
+
+	{
+		printf 'peak_used_kib'
+		for ((i = 0; i < ZRAM_DEVICES; i++)); do
+			printf ' zram%d=%d' "$i" "${peaks[$i]}"
+		done
+		printf '\n'
+	} > "$output"
+}
+
+setup_zram_for_sample()
+{
+	local count priorities
+
+	kmb-util-setup-zram-swap.sh --algo lzo-rle --size "$ZRAM_TOTAL_SIZE" --split
"$ZRAM_DEVICES"
+	count=$(awk '$1 ~ /^\/dev\/zram[0-9]+$/ { count++ } END { print count + 0 }'
/proc/swaps)
+	priorities=$(awk '$1 ~ /^\/dev\/zram[0-9]+$/ { seen[$5] = 1 } END { for (p in
seen) count++; print count + 0 }' /proc/swaps)
+	[ "$count" -eq "$ZRAM_DEVICES" ] || {
+		echo "expected $ZRAM_DEVICES active zram swaps, found $count" >&2
+		return 1
+	}
+	[ "$priorities" -eq 1 ] || {
+		echo "zram swap priorities differ" >&2
+		return 1
+	}
+	cat /proc/swaps
+}
+
+run_build_sample()
+{
+	local memory_bytes=$1
+	local memory_label=$2
+	local sample_label=$3
+	local measured=$4
+	local kernel_tag peak_file build_rc
+
+	setup_zram_for_sample || return 1
+	kernel_tag=$(uname -r | tr -c 'A-Za-z0-9._-' '-')
+	peak_file="$tmp_dir/peak-${memory_label}-${sample_label}.txt"
+
+	echo "=== KMB_SWAPQ_SAMPLE_BEGIN memory=$memory_label sample=$sample_label
measured=$measured ==="
+	kmb-test-build-linux-kernel.sh \
+		-j "$BUILD_JOBS" \
+		--memory "$memory_bytes" \
+		--config defconfig \
+		--name "swapq-${kernel_tag}-${memory_label}-${sample_label}" &
+	build_pid=$!
+	sample_swap_peaks "$build_pid" "$peak_file" &
+	sampler_pid=$!
+
+	if wait "$build_pid"; then
+		build_rc=0
+	else
+		build_rc=$?
```

```
+	fi
+	build_pid=
+	wait "$sampler_pid" || build_rc=1
+	sampler_pid=
+	cat "$peak_file"
+	echo "build_exit=$build_rc"
+	echo "=== KMB_SWAPQ_SAMPLE_END memory=$memory_label sample=$sample_label
measured=$measured ==="
+	[ "$build_rc" -eq 0 ]
+}
+
+run_build_matrix()
+{
+	local memory_bytes=$1
+	local memory_label=$2
+	local sample
+
+	echo "=== Kairui v1 kernel-build reproduction ==="
+	echo "kernel=$(uname -r)"
+	echo "memory_limit=$memory_label ($memory_bytes bytes)"
+	echo "jobs=$BUILD_JOBS"
+	echo "zram_devices=$ZRAM_DEVICES"
+	echo "zram_total_size=$ZRAM_TOTAL_SIZE"
+	echo "warmups=1"
+	echo "measured_repetitions=$BUILD_REPETITIONS"
+
+	swapoff /swap.img 2>/dev/null || true
+	run_build_sample "$memory_bytes" "$memory_label" warmup 0 || return 1
+	for sample in $(seq -w 1 "$BUILD_REPETITIONS"); do
+		run_build_sample "$memory_bytes" "$memory_label" "$sample" 1 || return 1
+	done
+}
diff --git a/testsuites/kairui-swapq-v1-v2-repro-20260807/.prepare.sh
b/testsuites/kairui-swapq-v1-v2-repro-20260807/.prepare.sh
new file mode 100755
index 0000000..1d3330a
--- /dev/null
+++ b/testsuites/kairui-swapq-v1-v2-repro-20260807/.prepare.sh
@@ -0,0 +1,44 @@
+#!/bin/bash
+# SPDX-License-Identifier: GPL-2.0
+
+set -eu
+
+for command in awk sha256sum tar time; do
+	command -v "$command" >/dev/null || {
+		echo "missing required command: $command" >&2
+		exit 1
+	}
+done
+
+for helper in kmb-test-build-linux-kernel.sh kmb-util-setup-zram-swap.sh; do
+	command -v "$helper" >/dev/null || {
+		echo "missing KMB helper: $helper" >&2
+		exit 1
+	}
+done
+
+if ! command -v usemem >/dev/null; then
+	[ -n "${KMB_LIB_DIR:-}" ] || {
+		echo "KMB_LIB_DIR is not set; cannot build usemem" >&2
+		exit 1
+	}
+	make -C "$KMB_LIB_DIR" usemem
+fi
+
+archive=
+for base in "${HOME}/linux" "${KMB_ASSETS:-}/linux" /root/linux linux; do
+	for suffix in .tar .tgz .tar.gz .tar.xz; do
```

```
+               [ ! -f "${base}${suffix}" ] || archive="${base}${suffix}"
+       done
+       [ -z "$archive" ] || break
+done
+[ -n "$archive" ] || {
+       echo "fixed Linux source archive not found" >&2
+       exit 1
+}
+
+echo "Kairui swap queue v1/v2 reproduction preflight"
+echo "source_archive=$archive"
+sha256sum "$archive"
+echo "cpu_count=$(nproc)"
+awk '/MemTotal/ { print "mem_total_kib=" $2 }' /proc/meminfo
diff --git a/testsuites/kairui-swapq-v1-v2-repro-20260807/10-kernel-build-2g-j96.sh
b/testsuites/kairui-swapq-v1-v2-repro-20260807/10-kernel-build-2g-j96.sh
new file mode 100755
index 0000000..6b9255b
--- /dev/null
+++ b/testsuites/kairui-swapq-v1-v2-repro-20260807/10-kernel-build-2g-j96.sh
@@ -0,0 +1,12 @@
+#!/bin/bash
+# SPDX-License-Identifier: GPL-2.0
+
+set -u
+. "$(dirname "$(realpath "$0")")/../lib-bench.sh"
+. "$(dirname "$(realpath "$0")")/.build-repro-common.sh"
+
+info "Kairui v1 reproduction: kernel build, 2 GiB memcg, make -j96"
+stabilize_performance
+disable_zswap
+run_build_matrix 2147483648 2g
+dump_sys_info
diff --git a/testsuites/kairui-swapq-v1-v2-repro-20260807/20-kernel-build-3g-j96.sh
b/testsuites/kairui-swapq-v1-v2-repro-20260807/20-kernel-build-3g-j96.sh
new file mode 100755
index 0000000..01b605a
--- /dev/null
+++ b/testsuites/kairui-swapq-v1-v2-repro-20260807/20-kernel-build-3g-j96.sh
@@ -0,0 +1,12 @@
+#!/bin/bash
+# SPDX-License-Identifier: GPL-2.0
+
+set -u
+. "$(dirname "$(realpath "$0")")/../lib-bench.sh"
+. "$(dirname "$(realpath "$0")")/.build-repro-common.sh"
+
+info "Kairui v1 reproduction: kernel build, 3 GiB memcg, make -j96"
+stabilize_performance
+disable_zswap
+run_build_matrix 3221225472 3g
+dump_sys_info
diff --git a/testsuites/kairui-swapq-v1-v2-repro-20260807/30-usemem-brd-scaled-12dev.sh
b/testsuites/kairui-swapq-v1-v2-repro-20260807/30-usemem-brd-scaled-12dev.sh
new file mode 100755
index 0000000..e745bfc
--- /dev/null
+++ b/testsuites/kairui-swapq-v1-v2-repro-20260807/30-usemem-brd-scaled-12dev.sh
@@ -0,0 +1,53 @@
+#!/bin/bash
+# SPDX-License-Identifier: GPL-2.0
+
+set -u
+. "$(dirname "$(realpath "$0")")/../lib-bench.sh"
+
+root_swap_was_active=0
+if awk '$1 == "/swap.img" { found = 1 } END { exit !found }' /proc/swaps; then
+       root_swap_was_active=1
+fi
```

```
+
+cleanup_brd()
+{
+       local rc=${1:-0}
+       local dev
+
+       trap - EXIT INT TERM
+       set +e
+       for dev in /dev/ram*; do
+               [ -b "$dev" ] || continue
+               swapoff "$dev" 2>/dev/null
+       done
+       rmmod brd 2>/dev/null
+       if [ "$root_swap_was_active" -eq 1 ] &&
+          ! awk '$1 == "/swap.img" { found = 1 } END { exit !found }' /proc/swaps; then
+               swapon -p -1 /swap.img || rc=1
+       fi
+       echo "=== final swap state ==="
+       cat /proc/swaps
+       exit "$rc"
+}
+
+trap 'cleanup_brd $?' EXIT
+trap 'exit 130' INT
+trap 'exit 143' TERM
+
+info "Kairui v1 BRD trend test: scaled VM workload, not the original 48 GiB workload"
+echo "kernel=$(uname -r)"
+echo "original_workload=12 brd, usemem -n 32 1536M (48 GiB; requires >=64 GiB host)"
+echo "scaled_vm_workload=12 brd totaling 2040M, usemem -n 32 160M (5 GiB)"
+
+command -v usemem >/dev/null || {
+       echo "usemem is not installed" >&2
+       exit 1
+}
+kmb-util-setup-brd-swap.sh --preflight
+stabilize_performance
+set_thp_never
+swapoff -a
+kmb-util-setup-brd-swap.sh --size 2040M --numa-node 0 --split 12
+cat /proc/swaps
+/usr/bin/time usemem --init-time -O -y -x -n 32 160M
+dump_sys_info
--
2.55.0


From 2cf80eca955d4b5d99c1af4075008f7947e4c2cc Mon Sep 17 00:00:00 2001
From: "Lian Wang (Processmission)" <lianux.mm@gmail.com>
Date: Fri, 7 Aug 2026 11:16:35 +0800
Subject: [KMB PATCH 2/4] lib: handle unset KMB_ROOT with nounset

Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 libexec/lib.sh | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

diff --git a/libexec/lib.sh b/libexec/lib.sh
index 2a051ca..fae6d47 100755
--- a/libexec/lib.sh
+++ b/libexec/lib.sh
@@ -1,7 +1,7 @@
 #!/bin/sh
 # SPDX-License-Identifier: GPL-2.0

-if [ -z "$KMB_ROOT" ]; then
+if [ -z "${KMB_ROOT:-}" ]; then
        KMB_ROOT="$(dirname "$(realpath "$0")")"
        while [ "$KMB_ROOT" != "/" ]; do
```

```
                [ -f "$KMB_ROOT/.kmb-root" ] && break
--
2.55.0


From dd7f0f45683163a05c0294eed9f404fdddc2a1a2 Mon Sep 17 00:00:00 2001
From: "Lian Wang (Processmission)" <lianux.mm@gmail.com>
Date: Fri, 7 Aug 2026 11:33:23 +0800
Subject: [KMB PATCH 3/4] tests: reduce swap peak sampling overhead

Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 .../.build-repro-common.sh                    | 24 +++++++++++++-----
 1 file changed, 18 insertions(+), 6 deletions(-)

diff --git a/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
b/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
index 10d5e6f..81134ca 100644
--- a/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
+++ b/testsuites/kairui-swapq-v1-v2-repro-20260807/.build-repro-common.sh
@@ -64,14 +64,26 @@ sample_swap_peaks()
 {
        local watched_pid=$1
        local output=$2
-       local -a peaks=(0 0 0 0 0 0 0 0)
-       local i used
+       local -a peaks=()
+       local dev i size type used priority
+
+       for ((i = 0; i < ZRAM_DEVICES; i++)); do
+               peaks[$i]=0
+       done

        while kill -0 "$watched_pid" 2>/dev/null; do
-               for ((i = 0; i < ZRAM_DEVICES; i++)); do
-                       used=$(awk -v dev="/dev/zram$i" '$1 == dev { print $4; found = 1
} END { if (!found) print 0 }' /proc/swaps)
-                       [ "$used" -le "${peaks[$i]}" ] || peaks[$i]=$used
-               done
+               while read -r dev type size used priority; do
+                       case "$dev" in
+                       /dev/zram*)
+                               i=${dev#/dev/zram}
+                               case "$i" in
+                               ''|*[!0-9]*) continue ;;
+                               esac
+                               [ "$i" -lt "$ZRAM_DEVICES" ] || continue
+                               [ "$used" -le "${peaks[$i]}" ] || peaks[$i]=$used
+                               ;;
+                       esac
+               done < /proc/swaps
                sleep 1
        done

--
2.55.0


From ebb5b5d538a6c17c2cbc3c250768a6ac3a5616ef Mon Sep 17 00:00:00 2001
From: "Lian Wang (Processmission)" <lianux.mm@gmail.com>
Date: Fri, 7 Aug 2026 11:48:02 +0800
Subject: [KMB PATCH 4/4] lib: tolerate unavailable NMI watchdog control

Signed-off-by: Lian Wang (Processmission) <lianux.mm@gmail.com>
---
 libexec/lib-bench.sh | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

diff --git a/libexec/lib-bench.sh b/libexec/lib-bench.sh
```

```
index 5d73b61..a4eb5af 100755
--- a/libexec/lib-bench.sh
+++ b/libexec/lib-bench.sh
@@ -497,7 +497,7 @@ stabilize_performance() {
        done

        # Disable NMI watchdog — reduces periodic interrupts
-       echo 0 > /proc/sys/kernel/nmi_watchdog
+       echo 0 > /proc/sys/kernel/nmi_watchdog 2>/dev/null || true

        # Disable deep C-states to avoid wake-up latency
        for f in /sys/devices/system/cpu/cpu*/cpuidle/state*/disable; do
--
2.55.0
```

==== Attachment 16 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-review-and-testing/16-vm-raw-report-zh.md) ====

Kairui Swap Priority Queue v2 — VM Raw Report 2026-08-07

Summary

- **Status**: Build and boot gate PASSED, functional test gate PASSED (5/5, exit 0)
- **Commit**: `d5c8964cf19fcffd7eea75dc9f28c928b46503bf`
- **Kernel**: `7.2.0-rc5-00672-gd5c8964cf19f`
- **VM**: `lianux-VMware20-1` (172.16.213.132)
- **Result dir**: `/var/lib/kmultibench/results/kairui-swapq-v2-d5c8964-final-functional-20260807/`

Audit Gate

Check	Result
hostname	`lianux-VMware20-1`
uptime	22h59m, idle (load 0.03)
CPU	10 cores
RAM	8.2Gi total, 6.5Gi available
Swap	4Gi /swap.img, 6M used
Running kernel (pre)	`7.2.0-rc5-1--00672-g2e767b7c673c`
Source repo	`/root/kairui-swap-queue-kmb-src-20260803`, clean, `codex/kairui-swap-queue-v2-rewrite` at `2e767b7c673c`
KMB repo	`/root/kernel-multi-bench-swapq-20260803`, clean, `codex/kairui-swapq-v2-functional` at `d7a3f5d`
KMB sessions	None running
Base commit `94f9b3980dd4`	Present in repo
Approved commit `d5c8964c`	Not yet in repo (expected)
Config	CONFIG_ZRAM=m, CONFIG_SWAP=y, CONFIG_MEMCG=y, CONFIG_TRANSPARENT_HUGEPAGE=y
No other servers contacted	Confirmed

Build and Boot Gate

Bundle verification

- **Local SHA-256**: `499a162a786cca957ce9d434e746d484cd1310177b6238343a6cd1bedadd7be7`
- **VM SHA-256**: `499a162a786cca957ce9d434e746d484cd1310177b6238343a6cd1bedadd7be7` — MATCH
- **Git bundle verify**: 2 refs, 28 prerequisites, sha1 algorithm — OK

Branch creation

```
git fetch /root/kairui-swapq-v1-v2-20260807.bundle \
  refs/heads/kairui-swap-queue-v2:refs/heads/kairui-swap-queue-v2-final-test-20260807
git checkout kairui-swap-queue-v2-final-test-20260807
```

```

- HEAD: `d5c8964cf19f lib/plist.c: remove requeue function`

### Patch stack verification

13 patches from base `94f9b3980dd446b56acf1dfed649e9b32a9f3813`:

```
d5c8964cf19f lib/plist.c: remove requeue function
510984375538 mm/swap: drop swap active plist
05b43f03ab25 mm/swap: bound synchronous discard during allocation
f205c502011b mm/swap: remove available list
a05e05b8d1c1 mm/swap: add priority queue for swap device allocation
13e69c595fe6 mm/swap: change back to use each swap device's percpu cluster
74787f6dee14 mm/swap: consolidate swap inuse accounting helpers
3191606c8f9e mm/swap: remove swapon mutex and update proc reader
893fb7195cbe mm/swap: change the swapon lock into a percpu rwsem
da61e75e4748 mm/swap: introduce swap device iteration helper
f7ad0bf80490 mm/swap: cleanup and document swap device availability flag usage
a940e5611be0 mm/swap: slightly cleanup the code for hibernation error handling
22ec384c63f5 mm/swap: remove unused parameter for reading swap header
```

- **git diff --check**: clean (no whitespace errors)
- **Authors**: Kairui Song `<kasong@tencent.com>`, Youngjun Park `<youngjun.park@lge.com>`, Baoquan He `<bhe@redhat.com>` (Co-developed-by)
- **No AI attribution**: Confirmed — no Claude, Anthropic, DeepSeek, Codex, GPT, or Copilot in any trailer

### Build

- `make olddefconfig` — no change to .config
- `make -j10` — completed successfully
- `make modules_install` — completed
- `make install` — completed, initrd generated
- Boot image: `/boot/vmlinux-7.2.0-rc5-00672-gd5c8964cf19f` (62749184 bytes)
- Boot image SHA-256: `06cbf42fd7c1c79961f7a12ec0ccfd9ad4823775fcc77848d61f186ffacdaccd`

### Boot

- GRUB default saved to `/root/grubenv-pre-d5c8964c-backup` (saved_entry=gnulinux-barry-latest-...)
- GRUB entry: `gnulinux-advanced-...>gnulinux-7.2.0-rc5-00672-gd5c8964cf19f-advanced-...`
- Initial attempt with `grub-reboot` (next_entry) booted wrong kernel (7.1.0-rc5+) — submenu path not resolved
- Corrected with `grub-set-default` using `submenu>entry` format — booted correctly

### Kernel identity verification

| Check | Value | Match |
|---|---|---|
| `uname -r` | `7.2.0-rc5-00672-gd5c8964cf19f` | YES |
| `/proc/version` | `7.2.0-rc5-00672-gd5c8964cf19f` (#8 SMP PREEMPT_DYNAMIC Fri Aug 7 10:45:57 CST 2026) | YES |
| Source HEAD | `d5c8964cf19f lib/plist.c: remove requeue function` | YES |
| Boot image | `/boot/vmlinux-7.2.0-rc5-00672-gd5c8964cf19f` (62749184 bytes) | YES |
| Build host | `root@lianux-VMware20-1` | YES |
| Compiler | gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0 | YES |

---

## Functional Test Gate

Test suite: `testsuites/kairui-swapq-v2-functional-20260804/`
Result name: `kairui-swapq-v2-d5c8964-final-functional-20260807`
Helper tools: `swapq_pageout` (SHA-256: `2eca8f58...`), `swapq_large` (SHA-256: `f912bd7b...`)
```

00-kernel-identity.sh — exit 0

```
Linux lianux-VMware20-1 7.2.0-rc5-00672-gd5c8964cf19f #8 SMP PREEMPT_DYNAMIC
boot_id: 6d6dbb4e-d5c3-4ecf-95f7-d32c7a486e0d
CONFIG_IKCONFIG=y CONFIG_IKCONFIG_PROC=y CONFIG_MEMCG=y CONFIG_SWAP=y CONFIG_ZRAM=m
Initial swap: /swap.img file 4194300 0 -1
No failure signatures in dmesg
```

20-same-priority-distribution.sh — exit 0

```
3 zram devices: 2 at priority 10, 1 at priority 0
helper: 96 pages swapped (100663296 bytes)
used_kib A=49152 B=49152 C=0
→ Same-priority devices got equal distribution; priority 0 device got 0
Final state: only /swap.img, no zram, no dmesg warnings
```

30-swapon-swapoff-concurrency.sh — exit 0

```
2 zram devices, 5 concurrent workers (4x swapon/swapoff x 200 iters + 1x /proc/swaps
reader x 2000)
No malformed /proc/swaps output, no worker failures
Final state: clean, only /swap.img
No dmesg warnings
```

40-full-unmask-vmstat.sh — exit 0

```
Single zram device (16380 KiB), filled to full with 16 MiB allocation
pswpout counters: before=24480 -> full=28559 (+4079) -> refill=30599 (+2040)
After swapoff second device and refill: used_kib dropped 16380->8192
-> Full device correctly unmasks when space frees; refill succeeds on same device
Final state: clean, only /swap.img
No dmesg warnings
```

50-large-folio-peer.sh — exit 0

```
2 zram devices at same priority 10
Device A: pre-filled to ~8 MiB (fragmented)
Device B: empty
Large folio (2 MiB THP) allocation:
  used_kib A=8188->8188 (unchanged - fragmented, skipped)
  used_kib B=0->2048 (large folio landed here)
  thp_swpout=0->1  thp_swpout_fallback=0->0
-> Large folio correctly retried same-priority peer instead of falling back to split
Final state: clean, only /swap.img
No dmesg warnings
```

Dmesg delta

No BUG, WARNING, Oops, KASAN, KCSAN, UBSAN, soft lockup, hard LOCKUP, RCU stall, or hung task signatures in any test dmesg delta.

Cleanup verification

- `/proc/swaps`: only `/swap.img` file 4194300 0 -1
- No `/dev/zram*` devices
- No residual `swapq_pageout` or `swapq_large` processes
- GRUB default restored to pre-run saved_entry

```
---

## Result Directory

```
/var/lib/kmultibench/results/kairui-swapq-v2-d5c8964-final-functional-20260807/
`-- 1-7.2.0-rc5-00672-gd5c8964cf19f/
    |-- 00-kernel-identity.sh.log          (771 bytes, exit 0)
    |-- 20-same-priority-distribution.sh.log (943 bytes, exit 0)
    |-- 30-swapon-swapoff-concurrency.sh.log (340 bytes, exit 0)
    |-- 40-full-unmask-vmstat.sh.log       (658 bytes, exit 0)
    `-- 50-large-folio-peer.sh.log         (604 bytes, exit 0)
```

State Restoration

- GRUB saved_entry restored to: `gnulinux-barry-latest-4a6fa60b-8ce8-4836-a1fa-827f2ecf59df`
- Pre-run backup: `/root/grubenv-pre-d5c8964c-backup`
- Post-test backup: `/root/grubenv-d5c8964c-post-test-backup`
- Source repo: `kairui-swap-queue-v2-final-test-20260807` branch intact at d5c8964c
- No existing branches altered or deleted
- Bundle file: `/root/kairui-swapq-v1-v2-20260807.bundle`

Blocker / Uncertainty

- Initial GRUB `grub-reboot` (next_entry) did not boot the target kernel — submenu entries require `submenu_id>entry_id` format. Corrected.
- The `barry-latest` saved_entry references a menu entry not present in the current GRUB config (the "latest" entry uses ID `gnulinux-latest-...` not `gnulinux-barry-latest-...`). This was the pre-existing state and has been restored as-is.
- No performance tests were run. This is functional verification only.

Next Safe Action

1. v1 benchmark reproduction gate (A/B/C/D/E with kernel build `make -j96` under 2G/3G memcg, scaled 32-thread/12-brd workload)
2. Do NOT run multi-SSD performance tests
3. Do NOT push, merge, or publish results without explicit authorization

---

v1/v2 Performance Reproduction Staging（进行中）

固定比较点

Arm	Commit	作用
A	`bdc38bfc1262e3d1432afadd2aa2ffd83d139dbb`	v1 Before
B	`4a7d8bd1b6644d139f5aa9074141437aa3b060a3`	v1 patch 8
C	`a438694aa41a39aaaa23926e14d7560c147f6af3`	v1 patch 13
D	`94f9b3980dd446b56acf1dfed649e9b32a9f3813`	v2 base
E	`d5c8964cf19fcffd7eea75dc9f28c928b46503bf`	当前 v2

- 固定 workload 源码：由 VM kernel repo 的精确 `94f9b398...` 通过 `git archive --prefix=linux/` 生成 `/root/linux.tar.xz`；所有 arm 编译同一归档；
- KMB 测试分支：`kairui-swapq-v1-v2-benchmark`；
- 测试套件 commit：`91817c66ab07fc39400cf426b36ac4678cd048d9`；
- KMB nounset 修复 commit：`2cf80eca955d4b5d99c1af4075008f7947e4c2cc`；
- smoke 使用的 KMB bundle SHA-256：
```

  `ba84b4bf07b6674afd69643b2ced522abc76c744088905203c2f4a38b525d32b`;
- 当前 KMB tip: `ebb5b5d538a6c17c2cbc3c250768a6ac3a5616ef`; 总栈为
  `91817c6 -> 2cf80ec -> dd7f0f4 -> ebb5b5d`;
- 当前 KMB bundle SHA-256:
  `66c2c83b6d7d71e4594c87abc35ba9ee5bacea2e579ba889f381b5f905d0f80f`;
- 套件:
  `/root/kernel-multi-bench/testsuites/kairui-swapq-v1-v2-repro-20260807/`。

Harness failure（不计入内核结果）

第一次 2G smoke 在进入 workload 前退出，精确错误为:

```text
/root/kernel-multi-bench/libexec/lib.sh: line 4: KMB_ROOT: unbound variable
```

原因是 suite 启用 `set -u` 后，KMB 使用未保护的 `$KMB_ROOT`。独立修复:

```diff
-if [ -z "$KMB_ROOT" ]; then
+if [ -z "${KMB_ROOT:-}" ]; then
```

修复经过 `sh -n` 和
`env -u KMB_ROOT bash -u ./kmultibench --help` 验证。第一次失败未开始
kernel build，`/swap.img` 保持启用且 Used=0，因此明确标为 harness failure，
不得计入 A/B/C/D/E。

当前 v2 / 2G smoke 最终结果

- VM kernel: E，`7.2.0-rc5-00672-gd5c8964cf19f`;
- workload: 2G memcg、`make -j96`、8 个同 priority ZRAM（64G 逻辑容量);
- 本次只运行 warm-up，`KMB_BUILD_REPETITIONS=0`，不属于正式计量样本;
- `build_exit=0`，service `status=0/SUCCESS`，`exit.status=0`;
- GNU time: user `4107.21s`、system `18107.36s`、elapsed `37:28.02`、
  CPU `988%`、maxresident `616132 KiB`;
- VM counters delta: `workingset_refault_anon +893492659`、
  `pgmajfault +870256664`、`pgpgin +3606832001`;
- ZRAM peak USED（KiB):
  `1740964 1729416 1750484 1744436 1708652 1730320 1742056 1733496`;
  max/min 比约 `1.0245`，8 个设备均被使用;
- cgroup `oom_kill=0`; 11:19:49 至 11:57:27 的测试窗口无 Oops、BUG、
  WARNING、panic、hung task 或 swap error; 11:58:31 的 NVMe timeout 晚于
  测试完成，不得归因于本样本;
- `/proc/swaps` 最终仅 `/swap.img`（Used=0，priority=-1); 无残留 cgroup、
  build process 或 tmpfs workspace; `/dev/zram0..7` 节点仍存在，但
  `disksize=0` 且 `zramctl` 无 active device;
- 完整日志:
  `/var/lib/kmultibench/results/kairui-swapq-v2-2g-smoke-retry1-20260807/smoke.log`;
- `smoke.log` SHA-256:
  `1f3170905197f72a8710fd6002b6fda1175fa98da404b87b4eebafb11d2be262`;
- `exit.status` SHA-256:
  `9a271f2a916b0b6ee6cecb2426f0b3206ef074578be55d9bc94f6f3fe3ab86aa`;
- 状态: **PASS**。

固定 workload: `/root/linux.tar.xz`，size `163446972`，SHA-256
`2d3a3dc70e4a44d607051b3e6929fe050f39e5f276343e9d62d399c77c233a34`;
来源 commit `94f9b3980dd446b56acf1dfed649e9b32a9f3813`。`usemem` size

`133528`，SHA-256
`cf800a59c614c7994a792580ea7ac8eab954a8ec91ba02a4275c228660873f50`。

Harness 噪声说明：ZRAM reset 第一次返回 `EBUSY` 后按 helper 设计重试成功；
`nmi_watchdog` sysctl 在该 VM 返回 error 524，属于不支持的可选调优项。两者
均未导致样本失败。后者已在本地 KMB 独立 commit `ebb5b5d` 中容错，尚未在
本次运行中生效。

下一闸门

Smoke 闸门已经通过。下一步先同步本地 KMB tip `ebb5b5d` 并构建 A/B/C/D/E
五个精确 kernel；VM 只运行少量同机 trend sample。仅 2G 的完整五臂 1+12 次
已经有超过 40 小时的容量下界，完整 12-run 与 48GiB BRD 原口径转入后续
服务器工单。当前阶段仍明确排除服务器和实体 SSD 测试。

A/B/C/D/E 五内核 build-only 最终结果

- Service: `kairui-swapq-v1-v2-five-kernel-build-20260807.service`;
- 结果: `Result=success`, `exit.status=0`;
- Build log:
  `/var/lib/kmultibench/results/kairui-swapq-v1-v2-five-kernel-build-20260807-build/build.log`;
- Inventory:
  `/var/lib/kmultibench/run-kairui-swapq-v1-v2-five-kernel-build-20260807/kernels.list`;
- Config snapshot SHA-256:
  `e7bb64e9f0cb8d24beb079969fd4e2ac26bbaf0389d854d7d9839514147d8570`;

Arm	Commit	Installed kernel
A	`bdc38bfc1262`	`/boot/vmlinux-7.2.0-rc2-1--00390-gbdc38bfc1262`
B	`4a7d8bd1b664`	`/boot/vmlinux-7.2.0-rc2-2--00398-g4a7d8bd1b664`
C	`a438694aa41a`	`/boot/vmlinux-7.2.0-rc2-3--00403-ga438694aa41a`
D	`94f9b3980dd4`	`/boot/vmlinux-7.2.0-rc5-4--00659-g94f9b3980dd4`
E	`d5c8964cf19f`	`/boot/vmlinux-7.2.0-rc5-5--00672-gd5c8964cf19f`

五个 kernel 均完成 config、build、modules_install、kernel install 和 initrd；
build log 未发现真实 `error:`/`fatal:`。根文件系统剩余约 29GiB。
本阶段使用 `--build-only`，没有重启，也没有运行 A/B/C/D/E benchmark。
构建后 `/swap.img` 约使用 79MiB；正式 KMB run 前必须通过重启与 preflight
重新建立干净基线，并核对五个实际 `/boot/config-*` 与 image SHA-256。

==== Attachment 17 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-review-and-testing/17-test-plan-zh.md) ====
Kairui swap priority queue v2 测试方案

日期: 2026-08-07

1. 设计问题与术语边界

当前代码中的 `swap_prio_ring` 是分配器内部结构：每个不同的 swap
priority 对应一个 ring，同一 priority 的设备在该 ring 内轮转；多个
ring 再按 priority 从高到低组成 queue。

Youngjun 当前 swap tier RFC 表达的是策略：一个 tier 可以覆盖一段
priority range，memcg 可以选择允许使用哪些 tier。因此建议暂时保持:

- **priority group / ring**: 精确相同 priority 的设备集合，属于 allocator
  实现;

- **tier**: 一个或多个 priority group 的策略集合，属于用户与 memcg policy;
- **allocation strategy**: ring 内如何选择设备，例如固定 quota round-robin、weighted round-robin 或 fill-first。

不建议在 v2 直接加入“ring 数量”ABI。ring 数量应由当前已启用设备中不同 priority 的数量自然产生。若要比较策略，先在实验分支做编译期变量或仅测试用控制面，取得数据后再决定是否需要稳定 ABI。

2. 必须回答的问题

1. 同 priority 的设备是否公平轮转，且不会提前使用低 priority 设备?
2. 一个设备 full、masked、fragmented 或正在 swapoff 时，retry 是否完整访问同 ring 的其他设备?
3. CPU 迁移期间，task-local cursor、per-CPU iterator、quota accounting 和 per-device percpu cluster 是否保持一致?
4. swapon/swapoff 重建 ring 时是否存在 use-after-free、ABA、漏引用或错误设备归属?
5. large folio 在首个同级设备碎片化时，是否重试同 ring peer，而不是直接 split 或降级到低 priority?
6. 新 queue 相比旧 plist 是否消除全局锁瓶颈，且没有可观测性能退化?

3. VM 最终功能门槛

精确测试对象：本地分支 `kairui-swap-queue-v2`，提交 `d5c8964cf19fcffd7eea75dc9f28c928b46503bf`。

3.1 基础门槛

- clean build;
- `git diff --check`、patch stack 和身份检查通过;
- 启动后 `uname -r`、内核 image、source HEAD 三者匹配;
- dmesg 无新增 WARN/Oops/BUG/panic/lockdep/KASAN/KCSAN/UBSAN/stall;
- 所有测试结束后恢复 `/swap.img`、zram、THP、swappiness、cgroup 和启动项。

3.2 KMB 功能套件

1. kernel identity/config;
2. 三个 zram：两个同 priority、一个低 priority，验证同级分配与低级隔离;
3. 并发 swapon/swapoff 与 `/proc/swaps` reader churn;
4. 设备填满、回收、unmask、再次分配;
5. large-folio：首设备碎片化、同级 peer 可容纳，要求 peer 成功且 `thp_swpout_fallback` 不增加;
6. negative control：在缺少 same-ring retry 修复的内核上确认测试能够失败;
7. CPU migration stress：无固定亲和性的多线程 pageout，同时在允许 CPU 集合内周期性迁移线程，验证无遗漏 peer、无低级泄漏、无 stall;
8. ring mutation stress：allocation 压力下有界地添加/删除同 priority 设备，验证 cleanup、引用和队列重建。

3.3 VM 比较臂

- A：v1 cover 的 Before，`bdc38bfc1262e3d1432afadd2aa2ffd83d139dbb`;
- B：v1 After patch 8，`4a7d8bd1b6644d139f5aa9074141437aa3b060a3`;
- C：v1 After patch 13，`a438694aa41a39aaaa23926e14d7560c147f6af3`;
- D：当前 v2 自身的 mm-unstable base，即 `22ec384c63f5^`;
- E：当前 v2 `d5c8964cf19f`;
- F：可选 negative-control commit，仅证明测试灵敏度，不参与性能结论。

A/B/C 复现 Kairui v1 内部相对关系；D/E 隔离当前 rebase 后的 v2 增量。
由于 v1 与当前 v2 不在同一 base，A 与 E 的绝对横比只能作参考，不能把 base
期间的其他 MM 变化归因于 queue。

每一臂至少重复 5 次功能循环。并发和 migration stress 至少运行 10 分钟，
记录每一步实际退出码、per-device used pages、`pswpout`、THP counters、CPU、
上下文切换和 dmesg delta。

3.4 v1 cover benchmark 复现

原始 Message-ID:
`<20260714-swap-pcp-priq-v1-0-de9b164ed419@tencent.com>`。

原始数据口径:

- 8 个 ZRAM，kernel build `make -j96`，2G memcg，12 次平均 system time:
  Before 6633.09s、patch 8 12294.74s、patch 13 6551.27s;
- 同上，3G memcg: 3312.63s、6450.35s、3311.38s;
- 12 个 brd，`usemem --init-time -O -y -x -n 32 1536M`:
  4347.58、2773.39、4345.04 MB/s;
- 8 ZRAM 的 USED 分布约为每设备 692.4M 至 695.5M。

复现要求:

1. A/B/C 严格使用相同 kernel source workload、config、KMB 脚本、swap 设置、
   cache 状态和运行顺序;
2. 加入 D/E，但单独报告 base drift;
3. kernel build 先做 1 次 warm-up，再做 12 次计量，分别测试 2G 和 3G;
4. 报告每次 user/system/elapsed、退出码、OOM、`pswpout/pswpin` 和每设备 USED;
5. VM 只有 10 CPU，保留 `-j96` 是为了复现并发压力，不把绝对秒数与原机器横比;
6. 原始 usemem 命令产生约 48GiB 工作集，8.6GB VM 无法安全原样运行。VM
   使用 12 brd、32 线程、按可用 RAM 缩放的每线程大小作趋势验证，清楚标注
   scaled; 原始 48GiB 项等待至少 64GB RAM 主机;
7. 任一 arm 出现 OOM、dmesg 异常、cleanup 失败或实际 swap counter 无增量，
   该样本无效并停止后续解释。
8. 容量闸门: 当前 E/2G warm-up 在 35 分钟时仍未完成。仅 2G 的五臂
   `5 × (1 warm-up + 12 measured)` 已超过 38 VM 小时的下界，不含 3G、BRD、
   构建测试内核和重启。因此 VM 只承担功能闸门与少量同机 trend sample;
   A/B/C 的 12 次原口径复现和 D/E 的正式统计转入后续服务器工单。不得把
   缩减后的 VM 样本包装为原始 12-run 复现。

4. 单 SSD 主机上的虚拟多设备方案

4.1 zram: 算法正确性

创建 2、4、8 个独立 zram swap device，用不同 `swapon -p` 形成一个或多个
priority group。它能验证 device selection、fairness、full/unmask、retry、
large folio 和 swapon/swapoff 生命周期，但 CPU 压缩开销会污染 I/O 性能。

4.2 memory-backed null_blk: 可控设备模型

为每个虚拟设备独立配置:

- `memory_backed=1`，保证 swap-in 能读回数据;
- `completion_nsec`，模拟不同延迟;
- `mbps`，模拟带宽上限;
- `submit_queues` 和 `hw_queue_depth`，模拟不同队列能力;

- `irqmode=2`，让 completion latency 生效。

建议拓扑：

- fast0、fast1：同 priority、同带宽与延迟；
- fast2-degraded：同 priority、较小 queue depth 或较低带宽；
- slow0：低 priority、较高延迟和较低带宽。

如需要分别控制读、写、flush 延迟，可在 memory-backed device 上叠加 `dm-delay`。所有虚拟设备仍共享 CPU、内存和主机资源，数据只能称为 **synthetic allocator/block-model evidence**。

4.3 单物理 SSD 的边界

在同一 SSD 上划分 partition、loop file 或 dm-linear device，只能形成多个 `swap_info_struct`，不能形成独立介质、控制器或真实并行带宽。因此不得据此声称“多 SSD 吞吐提升”或“跨设备硬件扩展性”。

真实多 SSD 性能结论至少需要两块独立 SSD，最好位于独立 NVMe namespace/controller 路径，并记录 NUMA、PCIe、IRQ affinity、queue depth、discard 和温度状态。如果当前服务器只有一块 SSD，最终真实硬件数据应由临时增加第二块 SSD，或邀请 Kairui/Youngjun/其他测试者在多设备机器复现获得。

5. 性能矩阵

拓扑

- 1 ring × 2/4/8 devices;
- 2 rings × 2+2 devices;
- 同级等速、同级异速、一个 full、一个 fragmented、一个 masked;
- 1、4、全部 CPU；单线程与多线程 pageout。

策略实验

- 当前 fixed-cluster-quota round-robin;
- quota = 1 的细粒度 round-robin;
- weighted round-robin;
- fill-first，作为对照而非默认候选。

指标

- correctness：低 priority 泄漏、peer 覆盖、swapoff 后访问、异常退出；
- fairness：每设备 pages/bytes、max/min、标准差和 Jain fairness index;
- performance：pageout throughput、IOPS、p50/p95/p99、CPU time、context switch;
- contention：`swap_queue_update_lock`、旧 `swap_avail_lock` 对照、每次 slot allocation 成本;
- reclaim：`pswpout/pswpin`、`pgscan_*`、`pgsteal_*`、THP swap counters;
- stability：dmesg、hung task、lockdep/KASAN/KCSAN（对应调试构建）。

6. 发布门槛

- VM 当前 HEAD 功能套件全部通过；
- baseline/v2 至少 5 次可复现 synthetic comparison，无已知退化；
- negative control 证明关键 retry 测试确实能捕获旧行为；
- 单 SSD/虚拟设备结果明确标注，禁止冒充真实多 SSD 数据；
- 若没有真实多 SSD，cover letter 诚实列为待补，并请求有硬件的维护者协测；
- 邮件、推送、合并、PR 和结果公开均需 Lian 对当前具体产物再次确认。

==== Attachment 18 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-review-and-testing/18-current-status-zh.md) ====

Kairui swap priority queue v2: 当前交接状态

- Series: Kairui/Youngjun swap priority queue v1 复现与当前 v2 推进;
- 当前范围: VM 功能验证、v1 A/B/C 复现、v2 D/E 对照;
- 明确排除: 服务器、实体 SSD、Barry/MGLRU、邮件、push、merge、PR、发布;
- VM: `root@172.16.213.132` (`lianux-VMware20-1`);
- kernel repo: `/root/kairui-swap-queue-kmb-src-20260803`;
- kernel branch/HEAD: `kairui-swap-queue-v2-final-test-20260807` / `d5c8964cf19fcffd7eea75dc9f28c928b46503bf`;
- running kernel: `7.2.0-rc5-00672-gd5c8964cf19f`;
- KMB repo: `/root/kernel-multi-bench`;
- KMB branch/HEAD: `kairui-swapq-v1-v2-benchmark` / `2cf80eca955d4b5d99c1af4075008f7947e4c2cc`;
- 已验证: current v2 build/boot 通过; 功能套件 5/5 exit 0;
- 当前 phase: 2G warm-up smoke 与 A/B/C/D/E 五内核 build-only 均已完成;
- 当前结果: PASS, exit 0, elapsed `37:28.02`, system `18107.36s`, cgroup `oom_kill=0`, 8 个 ZRAM 均有负载;
- 运行日志: `/var/lib/kmultibench/results/kairui-swapq-v2-2g-smoke-retry1-20260807/smoke.log`;
- 已知 harness 事件: 首次 smoke 因 KMB 未保护 `$KMB_ROOT` 在 `set -u` 下退出; 已由独立 commit `2cf80ec` 修复, 首次失败不计入内核结果;
- cleanup: 已验证 `/swap.img` 恢复、无 active ZRAM、无 cgroup/process/tmpfs 残留;
- Build service: `kairui-swapq-v1-v2-five-kernel-build-20260807.service`, `Result=success`、`exit.status=0`;
- Build log: `/var/lib/kmultibench/results/kairui-swapq-v1-v2-five-kernel-build-20260807-build/build.log`;
- Config SHA-256: `e7bb64e9f0cb8d24beb079969fd4e2ac26bbaf0389d854d7d9839514147d8570`;
- Inventory: `/var/lib/kmultibench/run-kairui-swapq-v1-v2-five-kernel-build-20260807/kernels.list`;
- 下一安全动作: 核对五个 image/config/hash/inventory 后启动 VM trend; 完整 12-run 与 48GiB BRD 留给后续服务器; 不得重复启动 build 或 smoke。

==== Attachment 19 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-review-and-testing/19-kmb-usage-and-test-guide.md) ====

Swap priority queue v2: KMB reproduction guide

This guide describes the private validation package prepared for the current
13-patch v2. It is not an upstream performance claim and it must not be mixed
with Barry/MGLRU or other swap-tier branches.

1. Fixed comparison points

Arm	Commit	Meaning
A	`bdc38bfc1262e3d1432afadd2aa2ffd83d139dbb`	v1 before
B	`4a7d8bd1b6644d139f5aa9074141437aa3b060a3`	v1 after patch 8
C	`a438694aa41a39aaaa23926e14d7560c147f6af3`	v1 after patch 13
D	`94f9b3980dd446b56acf1dfed649e9b32a9f3813`	v2 base
E	`d5c8964cf19fcffd7eea75dc9f28c928b46503bf`	current v2

A/B/C reproduce the original v1 relationship. D/E isolate the current v2
delta on its newer base. A and E must not be compared as though all
differences came from this series.

2. KMB and workload identities

- KMB branch: `kairui-swapq-v1-v2-benchmark`
- KMB tip: `ebb5b5d538a6c17c2cbc3c250768a6ac3a5616ef`
- KMB stack:
  - `91817c6` tests: add swap queue v1/v2 reproduction matrix
  - `2cf80ec` lib: handle unset KMB_ROOT with nounset
  - `dd7f0f4` tests: reduce swap peak sampling overhead
  - `ebb5b5d` lib: tolerate unavailable NMI watchdog control
- Test suite:
  `testsuites/kairui-swapq-v1-v2-repro-20260807/`
- Fixed workload archive: `/root/linux.tar.xz`
- Workload source commit:
  `94f9b3980dd446b56acf1dfed649e9b32a9f3813`
- Archive SHA-256:
  `2d3a3dc70e4a44d607051b3e6929fe050f39e5f276343e9d62d399c77c233a34`
- `usemem` SHA-256:
  `cf800a59c614c7994a792580ea7ac8eab954a8ec91ba02a4275c228660873f50`

The attached `15-kmb-support.patch` contains only the four KMB commits above. Apply it to KMB commit `d7a3f5df764b72cb42e5604db1e2107baaa42c10`, or check out the named branch if it is already available locally.

3. Build and stage the five kernels

Use one reviewed config snapshot for all five arms. The VM run used a config whose SHA-256 is:

```text
e7bb64e9f0cb8d24beb079969fd4e2ac26bbaf0389d854d7d9839514147d8570
```

The build-only command shape is:

```sh
/root/kernel-multi-bench/libexec/bench-build-run \
  --config /path/to/kernel-build.config \
  --build-only --enable-recommended --resume-installed -j10 \
  /root/kairui-swap-queue-kmb-src-20260803 \
  bdc38bfc1262e3d1432afadd2aa2ffd83d139dbb \
  4a7d8bd1b6644d139f5aa9074141437aa3b060a3 \
  a438694aa41a39aaaa23926e14d7560c147f6af3 \
  94f9b3980dd446b56acf1dfed649e9b32a9f3813 \
  d5c8964cf19fcffd7eea75dc9f28c928b46503bf \
  -- \
  /root/kernel-multi-bench/testsuites/kairui-swapq-v1-v2-repro-20260807 \
  kairui-swapq-v1-v2-five-kernel-build-20260807
```

The completed VM inventory is:

```text
/var/lib/kmultibench/run-kairui-swapq-v1-v2-five-kernel-build-20260807/kernels.list
```

It contains the A/B/C/D/E images in that exact order. Before testing, verify every image, initrd, `/boot/config-*`, commit suffix, and SHA-256. Do not rely only on a filename.

4. What the suite runs

1. `10-kernel-build-2g-j96.sh`
   - eight equal-priority ZRAM devices;
   - 64 GiB logical ZRAM capacity;
   - fixed Linux source archive;
   - 2 GiB memory cgroup;
   - `make -j96`;
   - one warm-up plus 12 measured builds.

2. `20-kernel-build-3g-j96.sh`
   - the same workload with a 3 GiB memory cgroup;
   - one warm-up plus 12 measured builds.
3. `30-usemem-brd-scaled-12dev.sh`
   - VM-only scaled 12-BRD trend test;
   - 32 workers at 160 MiB each;
   - explicitly not the original 48 GiB workload.

Every kernel-build sample records GNU time, VM counter deltas, build exit status, and per-ZRAM peak usage. The suite restores `/swap.img` and checks the dmesg delta during cleanup.

5. Starting a KMB run

KMB reboots between kernel/test cases. Use only a dedicated VM or test host with console recovery available. Review the inventory and suite before this command:

```sh
/root/kernel-multi-bench/kmultibench run \
  /var/lib/kmultibench/run-kairui-swapq-v1-v2-five-kernel-build-20260807 \
  /root/kernel-multi-bench/testsuites/kairui-swapq-v1-v2-repro-20260807 \
  kairui-swapq-v1-v2-reproduction-20260807
```

Do not launch the full suite on the current 10-CPU/8.2-GiB VM by accident. One E/2-GiB warm-up took 37 minutes, so only the 2-GiB five-arm portion has a greater-than-40-hour lower bound. The full 12-run matrix belongs on the later server work order. A reduced VM trend run must use a separately reviewed suite and must never be described as the original 12-run reproduction.

6. Monitoring and results

Useful read-only checks:

```sh
systemctl status kernel-multibench-runner.service --no-pager -l
cat /var/lib/kmultibench/kmultibench.running.env
find /var/lib/kmultibench/results/NAME -maxdepth 2 -type f -print
cat /proc/swaps
```

Time extraction:

```sh
/root/kernel-multi-bench/bin/kmb-grep-time.sh --real \
  /var/lib/kmultibench/results/NAME/*/10-kernel-build-2g-j96.sh.log
```

BRD/usemem throughput extraction:

```sh
/root/kernel-multi-bench/bin/kmb-grep-usemem.sh --throughput-sum \
  /var/lib/kmultibench/results/NAME/*/30-usemem-brd-scaled-12dev.sh.log
```

For each arm, keep all individual samples and report mean, median, standard deviation, min/max, failures, OOM events, dmesg findings, VM counter deltas, and per-device distribution. Do not report only the average.

7. Server and real multi-SSD phase

The original v1 data used:

- 8 ZRAM devices, `make -j96`, 2-GiB and 3-GiB memcgs, 12 runs; and
- 12 BRD devices with `usemem -n 32 1536M` (48 GiB total).

The exact 48-GiB BRD workload needs a host with at least 64 GiB RAM. Real multi-SSD claims require at least two independent SSD/NVMe devices, with

controller, NUMA, IRQ affinity, queue depth, discard, and temperature recorded.
Partitions or loop devices on one SSD do not provide real multi-device
performance evidence.

Before the server phase, create a separate work order and keep its branch,
config, inventory, results, and cleanup record isolated from the VM run.

==== Attachment 20 (/Volumes/linux/mm/KUNWU_SWAP_QUEUE_V2_PRIVATE_MAIL_2026-08-07/02-review-and-testing/20-summary-for-kunwu.md) ====

```
# Swap priority queue v2: private handoff summary

## Purpose

The series replaces the allocation-time swap priority plist rotation with a
priority queue containing one immutable device ring per priority.  Per-CPU
readers rotate within a ring, reducing global lock contention while preserving
strict priority ordering.

Youngjun's patch restores per-device percpu clusters before the queue is
introduced.  That separation keeps device policy below allocation selection
and avoids the priority inversion possible with one global percpu cluster.

## Important v2 changes

- Stable task-local retry cursor.
- `migrate_disable()` across the sleepable queue walk so retry/accounting and
  per-CPU cluster allocation stay on one CPU.
- Exactly-once peer traversal within the selected priority ring.
- Same-priority peer retry for large folios before split fallback.
- Tagged full/disabled entries with serialized writers and lockless readers.
- Tighter swapon/swapoff publication and disable ordering.
- Bounded synchronous discard work.
- Removal of the transitional active/available plists and unused plist API.

The original authors remain on their patches.  Lian's substantive changes use
`Co-developed-by` and `Signed-off-by`; no automated-tool attribution is added.

## Verified so far

1. Current v2 `d5c8964cf19f` builds and boots.
2. Functional KMB suite: 5/5 passed.
   - exact same-priority distribution;
   - concurrent swapon/swapoff and `/proc/swaps` reading;
   - full/unmask/refill;
   - large-folio same-ring peer retry without fallback;
   - clean dmesg and cleanup.
3. Extreme E/2-GiB smoke passed:
   - `make -j96`, 8 equal-priority ZRAM devices;
   - exit 0, elapsed 37:28.02;
   - system time 18107.36 seconds;
   - all eight ZRAM devices used, peak max/min ratio about 1.0245;
   - no OOM or kernel failure signature;
   - `/swap.img`, cgroup, processes, and workspace restored.
4. A/B/C/D/E build-only gate passed:
   - all five kernels and initrds installed;
   - inventory generated in A/B/C/D/E order;
   - no real build error or fatal message;
   - benchmarks have not started.

The 18107-second VM system time must not be compared directly with Kairui's
original numbers.  The VM has only 10 CPUs and deliberately runs `-j96`, so it
is an extreme contention/correctness environment rather than a matching
performance machine.

## Original v1 reference numbers

Average system time over 12 runs with 8 ZRAM devices and `make -j96`:
```

```
| Memcg | Before | After patch 8 | After patch 13 |
|---|---:|---:|---:|
| 2 GiB | 6633.09 s | 12294.74 s | 6551.27 s |
| 3 GiB | 3312.63 s | 6450.35 s | 3311.38 s |

The original 12-BRD `usemem -n 32 1536M` throughput values were 4347.58,
2773.39, and 4345.04 MB/s.  These are reference targets; the current VM cannot
reproduce the 48-GiB workload safely.

## Still required before an upstream v2

- Review the complete v1-to-v2 range-diff patch by patch.
- Verify five installed image/config/hash mappings.
- Run a reduced, clearly labeled VM A/B/C/D/E trend comparison.
- Run the original 12-sample 2-GiB/3-GiB matrix on an appropriately sized
  server.
- Run the original 48-GiB BRD workload on a host with sufficient RAM.
- Exercise at least two real SSD/NVMe devices before making hardware scaling
  claims.
- Add CPU-migration and ring-mutation stress coverage.
- Review performance ratios, variance, failures, dmesg, and cleanup together
  before deciding whether the series is ready for public RFC v2 posting.

## Requested review focus

Please review especially:

1. whether same priority should remain the definition of a queue/ring;
2. the task-local cursor and CPU migration boundary;
3. lifecycle ordering around queue publication, tag updates, and swapoff;
4. the exactly-once same-ring retry semantics, including large folios;
5. whether the discard bound is appropriate for allocation latency; and
6. the A/B/C/D/E and real multi-SSD validation plan.
```